

文档版本	V1.0
发布日期	2021-01-20

ICE 芯片 SDK 开发者手册



芯片开放社区
Open Chip Community

目录

文档首页	1.1
第一章 开发板烧写与基本使用	1.2
1.1 开发板接线	1.2.1
1.2 Linux 环境下烧写镜像	1.2.2
1.3 Windows 环境下烧写镜像	1.2.3
1.4 开发板Linux系统基本配置	1.2.4
1.5 配置 SSHD 服务与远程登录	1.2.5
1.6 软件包管理工具使用	1.2.6
第二章 功能演示	1.3
2.1 系统性能分析工具集	1.3.1
2.1.1 cpuinfo	1.3.1.1
2.1.2 dhrystone - CPU性能基准测试程序	1.3.1.2
2.1.3 coremark - CPU性能基准测试程序	1.3.1.3
2.1.4 whetstone - 浮点运算测试程序	1.3.1.4
2.1.5 linpack- 线性系统软件包	1.3.1.5
2.1.6 stream - 内存带宽性能测试基准工具	1.3.1.6
2.1.7 sysbench - 系统性能基准测试工具	1.3.1.7
2.1.8 nbench - 存储器性能的基准测试程序	1.3.1.8
2.1.9 calibrator - 高速缓存分析工具	1.3.1.9
2.1.10 iperf3 - 网络性能分析工具	1.3.1.10
2.1.11 stress-ng - 压力测试工具	1.3.1.11
2.1.12 perf - 性能分析工具	1.3.1.12
2.2 GPU 驱动及示例	1.3.2
2.3 小程序应用示例	1.3.3
2.4 Xfce 桌面环境	1.3.4
第三章 Linux 系统定制	1.4
3.1 搭建交叉开发环境	1.4.1
3.2 编译 u-boot	1.4.2
3.3 编译 OpenSBI	1.4.3
3.4 编译 Linux Kernel	1.4.4
3.5 使用 Buildroot 构建根文件系统	1.4.5
3.6 启动镜像构建	1.4.6
3.7 制作 Debian Image	1.4.7
第四章 Linux 应用程序开发	1.5
4.1 安装开发环境	1.5.1
4.2 搭建 NFS 环境	1.5.2
4.3 GPIO 编程	1.5.3
4.4 I2C 编程	1.5.4
4.5 使用 vector 指令	1.5.5
第五章 Linux 程序调试	1.6
5.1 GDB 调试入门	1.6.1
5.2 使用 GDBSERVER 远程调试	1.6.2
5.3 核心转储调试	1.6.3
5.4 Perf 使用指南	1.6.4
5.5 Linux K/Uprobe 使用指南	1.6.5
5.6 KASAN 使用指南	1.6.6
5.7 KGDB 使用指南	1.6.7
5.8 Lockdep 使用指南	1.6.8
5.9 Linux系统性能分析工具集	1.6.9

第六章 Android on RISC-V	1.7
6.1 AOSP 开源	1.7.1
6.2 Android 镜像烧写	1.7.2
FAQ	1.8

ICE EVB 开发板简介

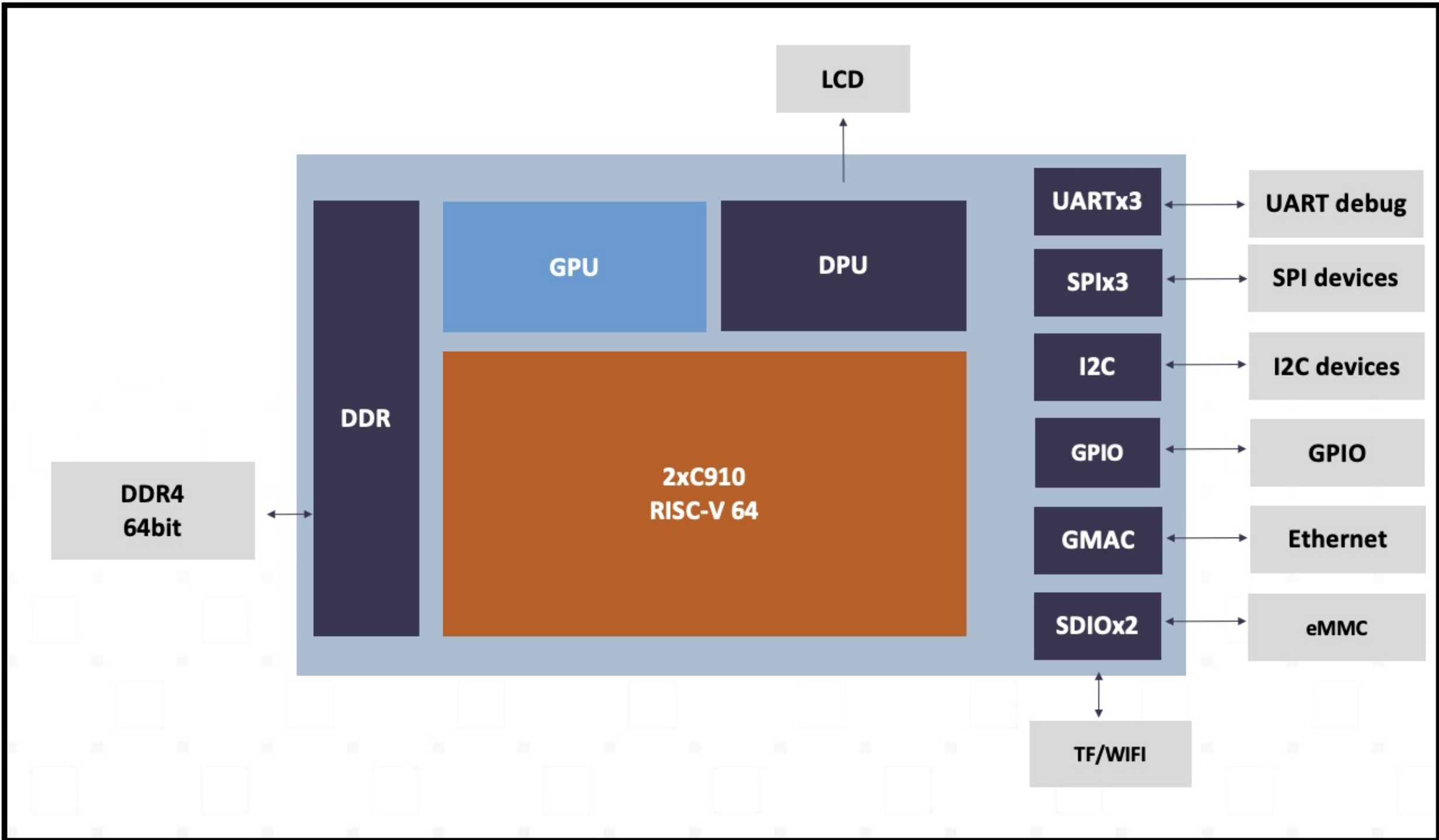
ICE EVB 是基于平头哥自主研发的 RISC-V 架构的 C910 CPU 的 ICE 高性能 SoC 开发板。

- 丰富的常见外设及接口
- 基于 OpenGL ES 的 3D 图形处理能力

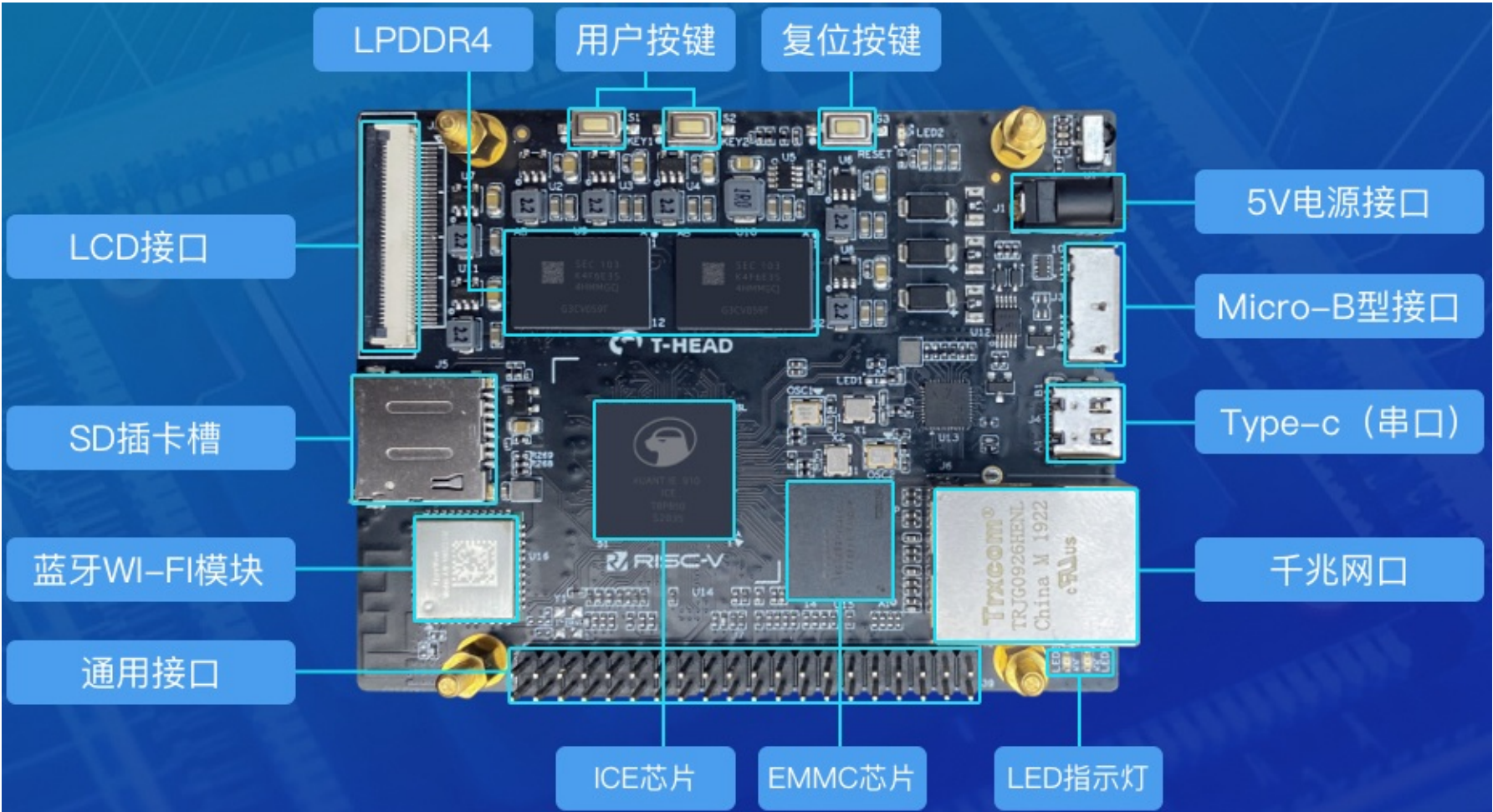
ICE 芯片

ICE 是一款通用智能数字 SoC 芯片，主要集了一个双核RISC-V 64位 C910 处理器，一个 GPU 和 DPU 单元。同时提供了高速接口以及通用外设接口，用于与主控设备间的数据和命令交互。

- 内嵌平头哥双核 C910@1.2GHz
- 支持 DDR4 up to 2400Mbps
- 支持 GMAC 接口
- 支持 GPU，支持 3D
- 支持 RGB888 LCD 显示，最大支持 1080P



ICE-RVB 开发板



ICE-EVB开发板



离线文档

《ICE SDK 使用手册》还提供了最新的离线文档下载：

- PDF 格式：https://yoc.docs.t-head.cn/icebook/ice_book.pdf
- MOBI格式：https://yoc.docs.t-head.cn/icebook/ice_book.mobi
- EPUB格式：https://yoc.docs.t-head.cn/icebook/ice_book.epub

概述

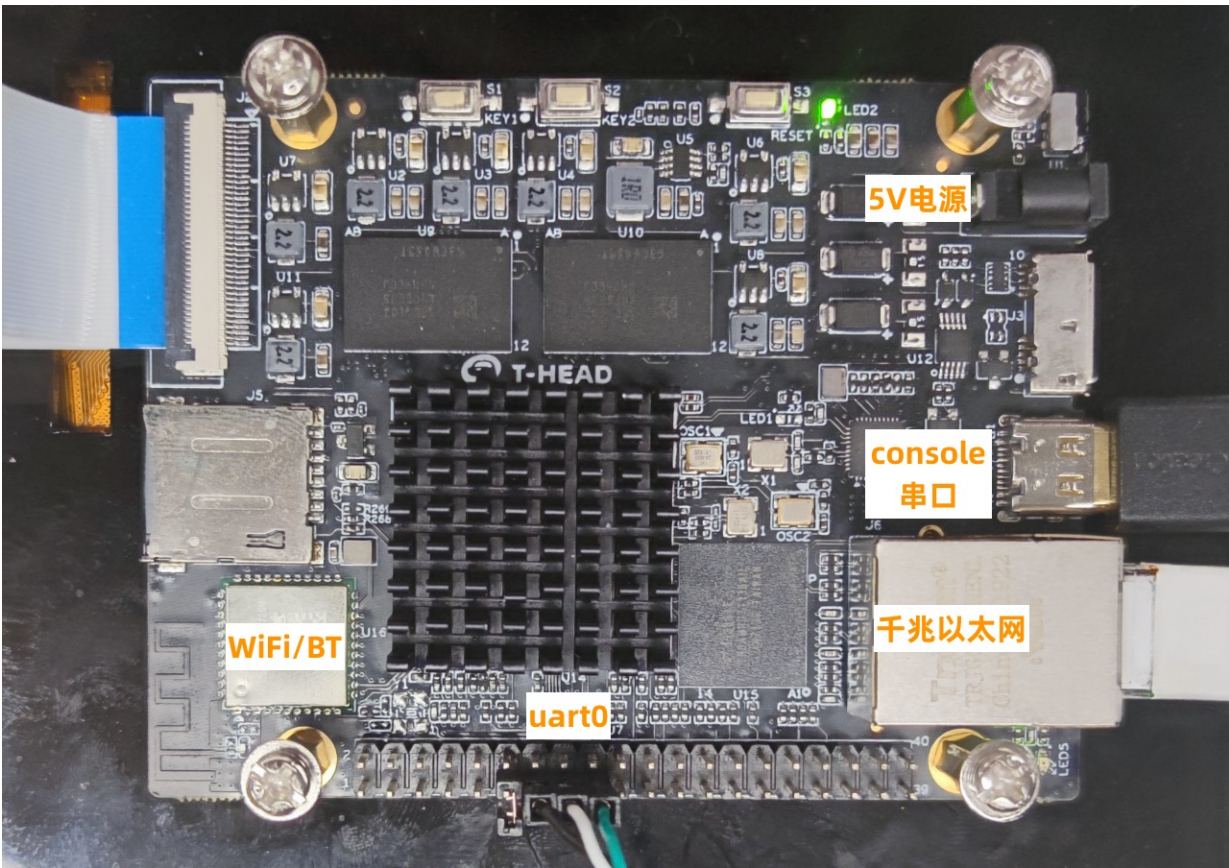
本文介绍镜像烧写、运行预置程序、开发环境搭建、DEMO 程序、内核调试方法等多方面内容，让用户快速上手。

开发板接线

ICE-RVB 开发板硬件连接

烧写、系统连接图

按照系统连接图连接好开发板：



- console 串口：CCT 烧写、Console 控制时使用的通信端口。同时可以通过 USB供电。
- 设备电源：USB type-C 端口 或 5V 电源供电。如果USB 端口供电不足，5V电源也可同时补充供电。
- 网络接口：千兆以太网，连接 RJ45 千兆网线，10M/100M/1000M 自适应。
- WiFi/BT：IEEE 802.11b/g/n，Bluetooth v4.1/BLE

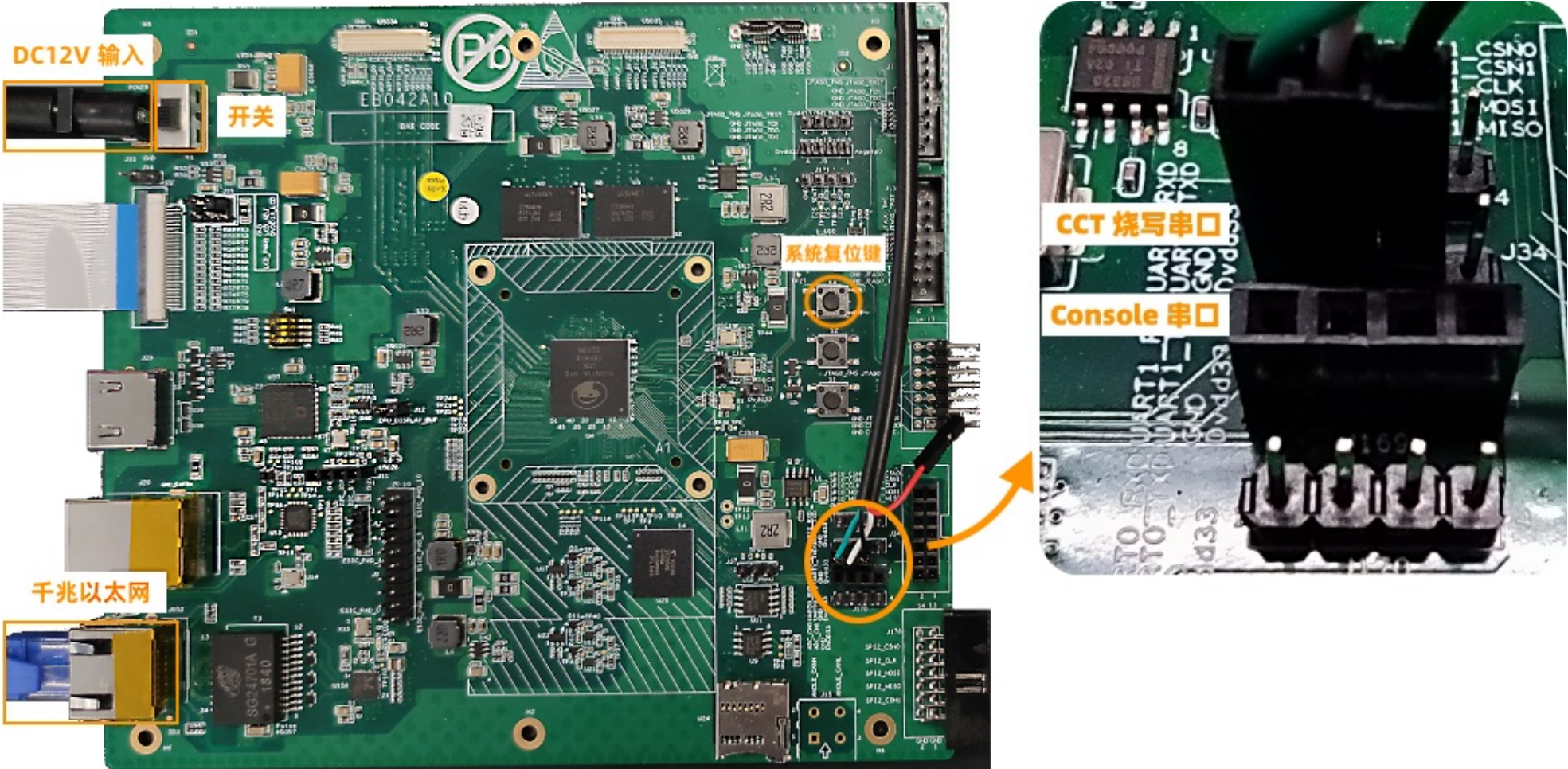
ICE-EVB 开发板硬件连接

先按照系统连接图连接好开发板：

- 设备电源：DC12V 输入，连接 12V 直流电源。
- CCT 烧写：CCT 烧写串口，J5 1-3 PIN，分别为RX、TX、GND，使用 thead-tools 或 CCT 工具烧写 uboot 镜像时使用的通信端口。
- Console 控制：Console 串口，J169 1-3 PIN，分别为RX、TX、GND，log 输出，控制台时使用的通信端口。
- 网络接口：千兆以太网，连接 RJ45 千兆网线，10M/100M/1000M 自适应。

1. CCT 烧写系统连接图：

烧写 u-boot 镜像。

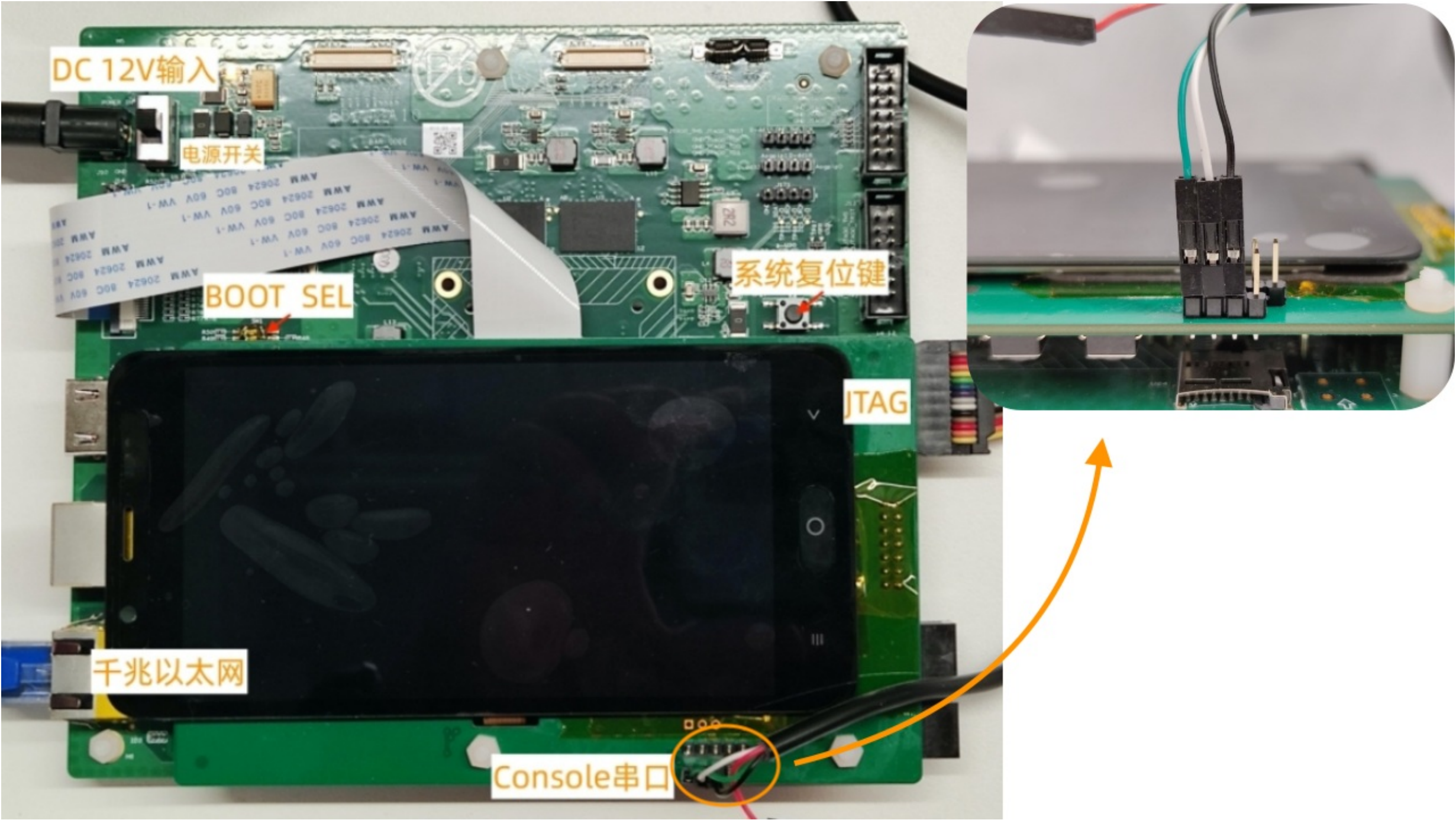


尤其注意串口的连接：

- 有 2个串口，一个用来打印，一个用来烧录 u-boot 镜像
- 连接的方式是 绿，白，黑

2. 系统连接图：

烧写 boot、rootfs 镜像，操作 Linux OS。



3. BOOT SEL 拨码：

BOOT SEL 拨码置为 ON、OFF、OFF、OFF ，eMMC 启动模式。



Linux 环境下烧写镜像

对于第一次使用 ICE-RVB、ICE-EVB 开发板的开发者，需要根据以下几个步骤完成开发板的镜像烧写：

- 第一步：烧录前准备：安装烧录工具(thead-tools、fastboot)、检查开发板与电脑的串口连接情况
- 第二步：下载镜像文件(ice_images)
- 第三步：通过 thead-tools 将 u-boot 镜像烧写到开发板的 eMMC 存储器
- 第四步：在 u-boot 环境下使用 fastboot 工具，将 boot.ext4 (内核、openSBI、dtb)、rootfs.ext4 (根文件系统) 镜像烧写到 eMMC
- 第五步：重启开发板，即可以进入Linux 控制台

推荐开发者使用 ubuntu18.04 或 ubuntu20.04 环境开发。

第一步：烧录前准备

安装 thead-tools

通过 pip 命令来安装 thead-tools 到你的系统中，thead-tools 支持 python2.7+、python3.6+，建议使用 python2 作为默认 python。

```
sudo pip install thead-tools

# 如果使用官方地址下载过慢，可使用国内清华镜像源加速
sudo pip install -i https://pypi.tuna.tsinghua.edu.cn/simple thead-tools
```

如果未找到 pip 命令，请先安装 pythonp-pip，如： sudo apt install python-pip 。

thead-tools 使用

通过 thead-tools 命令可以烧写 u-boot 镜像，更详细的 thead-tools 使用说明可以参见：thead-tools 使用说明，通过命令： thead cct --help 查使用说明：

```
root@linux > thead cct --help
Usage: thead cct <uart|list|download> [param]

Options:
  -h, --help            show this help message and exit
  -u UART               CCT serial port device
  -f FILE               Device start address
  -o OFFSET             Device start address
  -b BLOCK, --block=BLOCK
  -d DEVICE, --device=DEVICE
                        Device name
  -c, --compress
  -D, --debug           Enable debug trace info
```

安装 fastboot

发行版可通过包管理器安装 fastboot 工具，如 ubuntu下通过 sudo apt install fastboot 安装，或者从官网下载二进制文件，下载地址：<https://developer.android.google.cn/studio/releases/platform-tools>。

第二步：下载镜像

访问 https://gitee.com/thead-linux/ice_images/releases 下载最新镜像，可直接使用下面命令行下载 Linux SDK 镜像文件：

```
wget "https://gitee.com/thead-linux/ice_images/repository/archive/master?format=zip" -O ice_images.zip
unzip ice_images.zip
cd ice_images
ls -l
```

需要用到的是如下几个文件：


```
-rw-r--r--  1 lihm staff   30M May  18 21:30 boot.ext4
-rw-r--r--  1 lihm staff  160M May  18 21:30 debian-rootfs.ext4
-rw-r--r--  1 lihm staff  550K May  18 21:30 u-boot-with-spl_ice-evb.bin
-rw-r--r--  1 lihm staff  550K May  18 21:30 u-boot-with-spl_ice-rvb.bin
```

- **u-boot-with-spl_ice-rvb.bin**：ICE-RVB uboot 引导镜像文件
- **u-boot-with-spl_ice-evb.bin**：ICE-EVB uboot 引导镜像文件
- **boot.ext4**：/boot 分区文件系统，ext4格式，包含 fw_jump.bin、uImage、hw.dtb
- **debian-rootfs.ext4**：Debian 基础根文件系统镜像，ext4 格式

第三步：烧写 uboot

查看电脑串口

参考 [开发板接线](#) 《烧写系统连接图》完成开发板的串口连接，通过 thead-tools 烧写 u-boot 时需要使用开发板的 CCT 烧写串口，通过命令 `thead cct uart` 命令看到电脑上安装的串口列表，并确认电脑上串口与开发板串的对应关系：

```
root@linux > thead cct uart
uart device list:
  /dev/ttyUSB0 - USB-Serial Controller
  /dev/ttyUSB1 - USB-Serial Controller
```

查找开发板存储器列表

通过命令：`thead cct -u /dev/ttyUSB0 list` 可以查看开发板支持的烧写存储器列表，如下：

```
root@linux > thead cct -u /dev/ttyUSB0 list
Wait .....
CCT Version: 2
memory device list:
  dev = ram0    , size =  256.0KB
  dev = emmc0   , size =    2.0MB
  dev = emmc1   , size =    2.0MB
  dev = emmc2   , size =   3.7GB
```

注意：使用该命令时，先确定串口的连接是否正确，运行命令前先关闭开发板电源后再运行该命令，等到出现 `wait .....` 信息后再开启开发板电源，正常情况下电源开启后在1秒内会出现如下信息：

```
CCT Version: 2
memory device list:
  dev = ram0    , size =  256.0KB
  dev = emmc0   , size =    2.0MB
  dev = emmc1   , size =    2.0MB
  dev = emmc2   , size =   3.7GB
```

如果未出现上述打印信息，同时电脑是接有多个串口，请更新另一个串口再尝试一次，如 `thead cct -u /dev/ttyUSB1 list` 。如果所有串口都尝试后均未出现上述打印机信息，请检查串口与开发板的连接是否符合，详细连接参见：[开发板接线](#)。

烧写 u-boot

通过命令：`thead cct -u /dev/ttyUSB0 download -f u-boot-with-spl_ice-xxx.bin -d emmc0` 将 u-boot 镜像烧写到 eMMC 的 0号分区，烧写过程信息如下：

```
CCT Version: 2
Send file 'u-boot-with-spl_ice-xxx.bin' to 21:0 ...
Writing at 0x00009800... (3%)
```

待打印 `File u-boot-with-spl_ice-xxx.bin download success.` 后，烧写成功后，可以通过开发板的 Console 串口看到正确的启动信息：


```
U-Boot 2020.01-g6cc5d59b0d (Dec 20 2020 - 08:37:37 +0000)

CPU:   rv64imafdcvsu
Model: T-HEAD c910 ice
DRAM:  4 GiB
GPU ChipDate is:0x20151217
GPU Frequency is:500000KHz
NPU ChipDate is:0x20190514
DPU ChipDate is:0x20161213
MMC:   mmc0@3fffb0000: 0
Loading Environment from MMC... OK
In:     serial@3fff73000
Out:    serial@3fff73000
Err:    serial@3fff73000
Net:
Warning: ethernet@3fffc0000 (eth0) using random MAC address - e6:e2:ea:7a:30:ce
eth0: ethernet@3fffc0000
Hit any key to stop autoboot:  1
```

第四步：烧写开发板镜像

开发板 u-boot 配置

参考 [开发板接线](#) 《系统连接图》完成开发板的串口连接，开发板重新开机后进入 u-boot 命令模式，当 Console 串口出现如下提示时，按任键即可进行命令模式。关于 u-boot 的详细命令介绍可参见：[u-boot](#) 章节中关于命令使用的介绍。

```
Warning: ethernet@3fffc0000 (eth0) using random MAC address - a6:7d:bc:02:7d:4d
eth0: ethernet@3fffc0000
Hit any key to stop autoboot:  3      <--- 出现此信息时按任意键进入uboot
```

u-boot 配置命令列表如下（通过串口终端，在开发板上执行）：

```
# 恢复默认值
env default -a

# 配置 eMMC 分区
setenv uuid_rootfs "80a5a8e9-c744-491a-93c1-4f4194fd690b"
setenv partitions "name=table,size=2031KB"
setenv partitions "$partitions;name=boot,size=60MiB,type=boot"
setenv partitions "$partitions;name=root,size=-,type=linux,uuid=$uuid_rootfs"
gpt write mmc 0 $partitions

# 配置网络

# !!!!!!!!!!!!!!!!!!!!!!! 特别注意 !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
# 如果网络中接入多块 ICE-EVB 开发板，每块开发板的 ethaddr 要唯一
setenv ethaddr 00:a0:a0:a0:a0:a1      # 如果网络中接入多块 ICE-EVB 开发板，ethaddr 要唯一
setenv ipaddr 192.168.1.100           # 开发板 IP 地址
setenv netmask 255.255.255.0
# ping 192.168.1.1                    # ping 主机的IP，检验网络是否正常
saveenv                               # 将配置保存到 eMMC，下次启动时无需再配置

# 开启fastboot模式
fastboot udp
```

烧写 boot 分区

在 Linux 终端上使用 fastboot 命令完成镜像烧写，192.168.1.100 是开发板的运行 u-boot 时配置的 IP 地址，使用 `fastboot` 命令前确认电脑与开发板在同一个子网。在电脑的终端上执行：

```
fastboot -s udp:192.168.1.100 flash boot boot.ext4
```

烧写镜像

ICE-EVB 开发板提供三种根文件系统镜像，开发者可以根据自己的需要选择下载：

镜像	下载地址	大小
基本镜像(debian-base)	http://mirrors.aliyun.com/thead/images/riscv64-ice/debian-rootfs.ext4.gz	59MB
开发镜像 (debian-dev)	http://mirrors.aliyun.com/thead/images/riscv64-ice/debian-dev-rootfs.ext4.gz	197MB
xfce 桌面(debian-desktop)	http://mirrors.aliyun.com/thead/images/riscv64-ice/debian-desktop-rootfs.ext4.gz	517MB

选项一：烧写基本系统镜像

在下载的冰_images 中默认提供 debian-rootfs.ext4 镜像，使用 fastboot 命令烧写：

```
fastboot -s udp:192.168.1.100 flash root debian-rootfs.ext4
```

选项二：烧写开发者镜像

需要另外下载 debian-dev-rootfs.ext4 镜像，再用 fastboot 命令烧写：

```
wget "http://mirrors.aliyun.com/thead/images/riscv64-ice/debian-dev-rootfs.ext4.gz"
gzip -d debian-dev-rootfs.ext4.gz
fastboot -s udp:192.168.1.100 flash root debian-dev-rootfs.ext4
```

选项三：烧写 xFce 桌面系统镜像

需要另外下载 debian-desktop-rootfs.ext4 镜像，再用 fastboot 命令烧写：

```
wget "http://mirrors.aliyun.com/thead/images/riscv64-ice/debian-desktop-rootfs.ext4.gz"
gzip -d debian-desktop-rootfs.ext4.gz
fastboot -s udp:192.168.1.100 flash root debian-desktop-rootfs.ext4
```

第五步：启动开发板 Linux 系统

完成 Linux 镜像烧写后，重新启动开发板，Console 串口显示如下：

```
[ OK ] Started Serial Getty on hvc0.
[ OK ] Started Serial Getty on ttyS0.
[ OK ] Reached target Login Prompts.
[ OK ] Started System Logging Service.
[ OK ] Finished Remove Stale Onli...ext4 Metadata Check Snapshots.
[ OK ] Reached target Multi-User System.
[ OK ] Reached target Graphical Interface.
        Starting Update UTMP about System Runlevel Changes...
[ OK ] Finished Update UTMP about System Runlevel Changes.

Debian GNU/Linux bullseye/sid thead-910 ttyS0

thead-910 login: [ 94.362252] random: crng init done
[ 94.365674] random: 6 urandom warning(s) missed due to ratelimiting

thead-910 login:
```

登录用户名 root ，初始无密码，开发者可根据需要更改密码。

完成开发板的镜像烧写后，下一步进入 系统配置 章节，进行开发板的基本系统配置。

Windows 环境下烧写镜像

对于第一次使用 ICE-RVB、ICE-EVB 开发板的开发者，需要根据以下几个步骤完成开发板的镜像烧写：

- 第一步：烧录前准备：下载烧录工具(C-SKY Chip Tool)、检查开发板与电脑的串口连接情况
- 第二步：下载镜像文件(ice_images)
- 第三步：通过 C-SKY Chip Tool (CCT) 将 u-boot 镜像烧写到开发板的 eMMC 存储器
- 第四步：在 u-boot 环境下使用 fastboot 工具，将 boot.ext4 (内核、openSBI、dtb)、rootfs.ext4 (根文件系统) 镜像烧写到 eMMC
- 第五步：重启开发板，即可以进入Linux 控制台

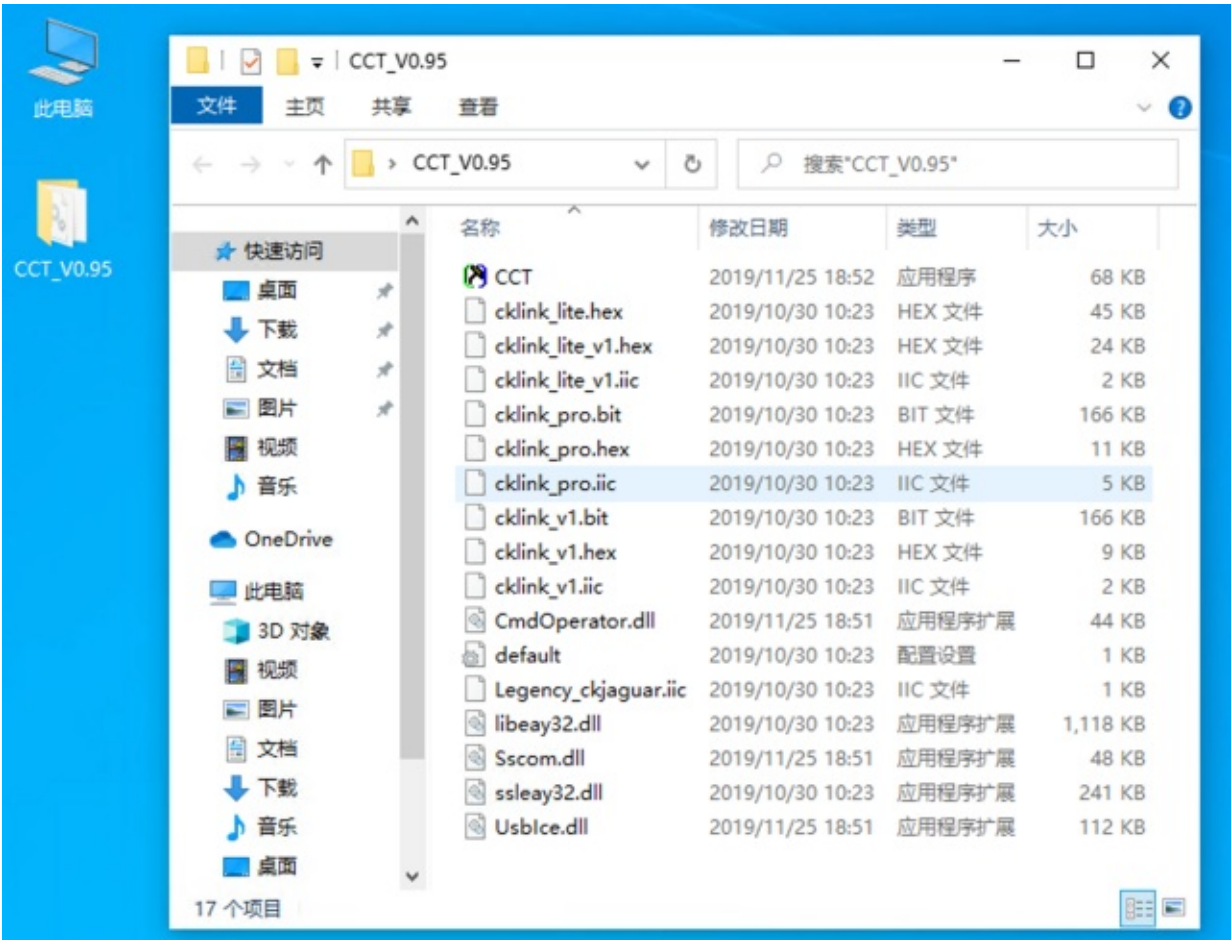
Windows 下有很多串口工具，推荐使用开源工具 PuTTY 进行串口操作。

第一步：烧录前准备

下载烧写工具

在此连接下载 CCT_V0.95 工具：https://yoc.docs.t-head.cn/tools/CCT_V0.95.zip

将下载的 zip 文件中 CCT_V0.95 文件夹解压到电脑上，下图为解压到桌面：

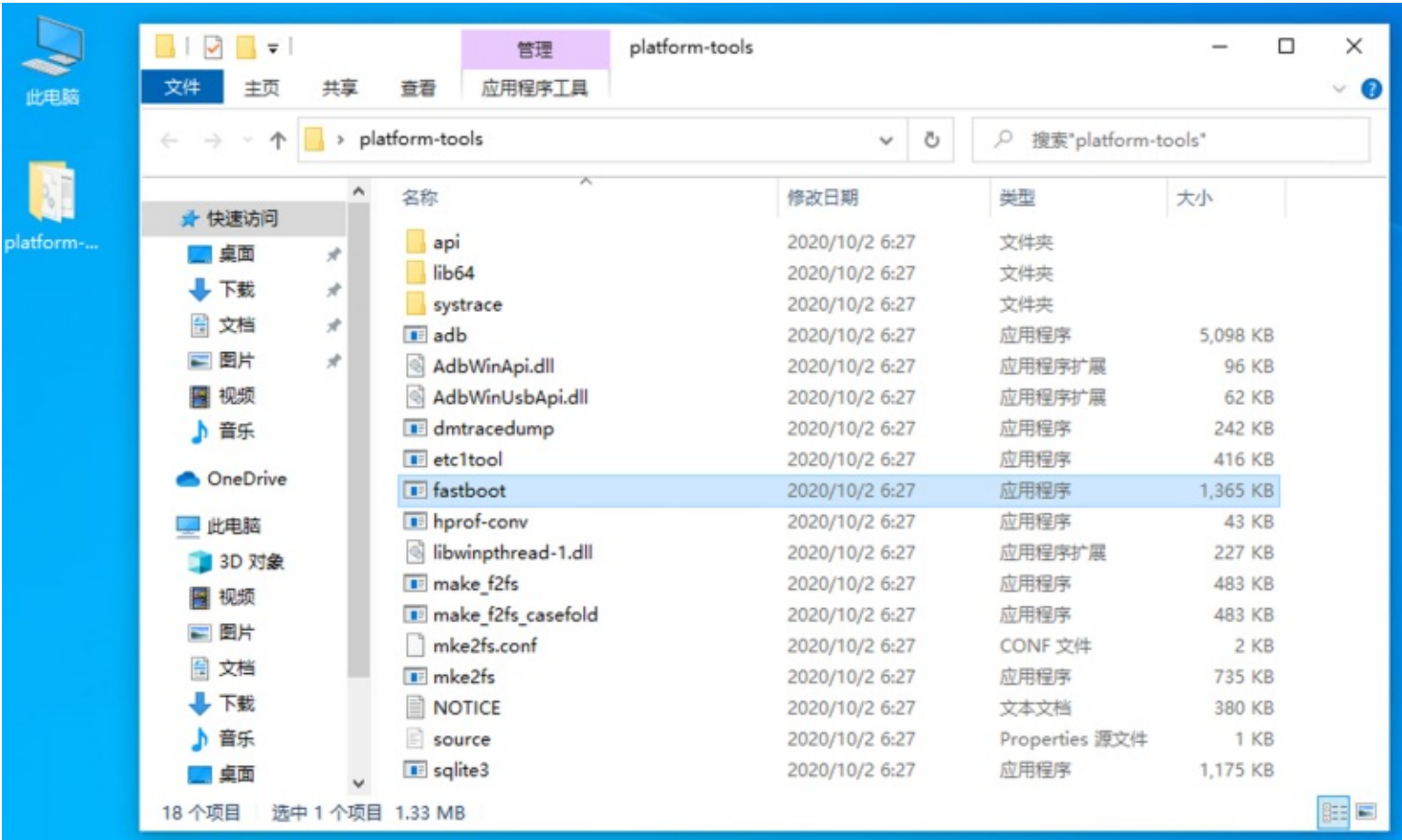


连接镜像传输串口

参考 [开发板接线](#) 《烧写系统连接图》完成开发板的串口连接，烧写 u-boot 需要使用开发板的 CCT 烧写串口。运行 CCT.exe 启动 CCT 烧写工具。

安装 fastboot

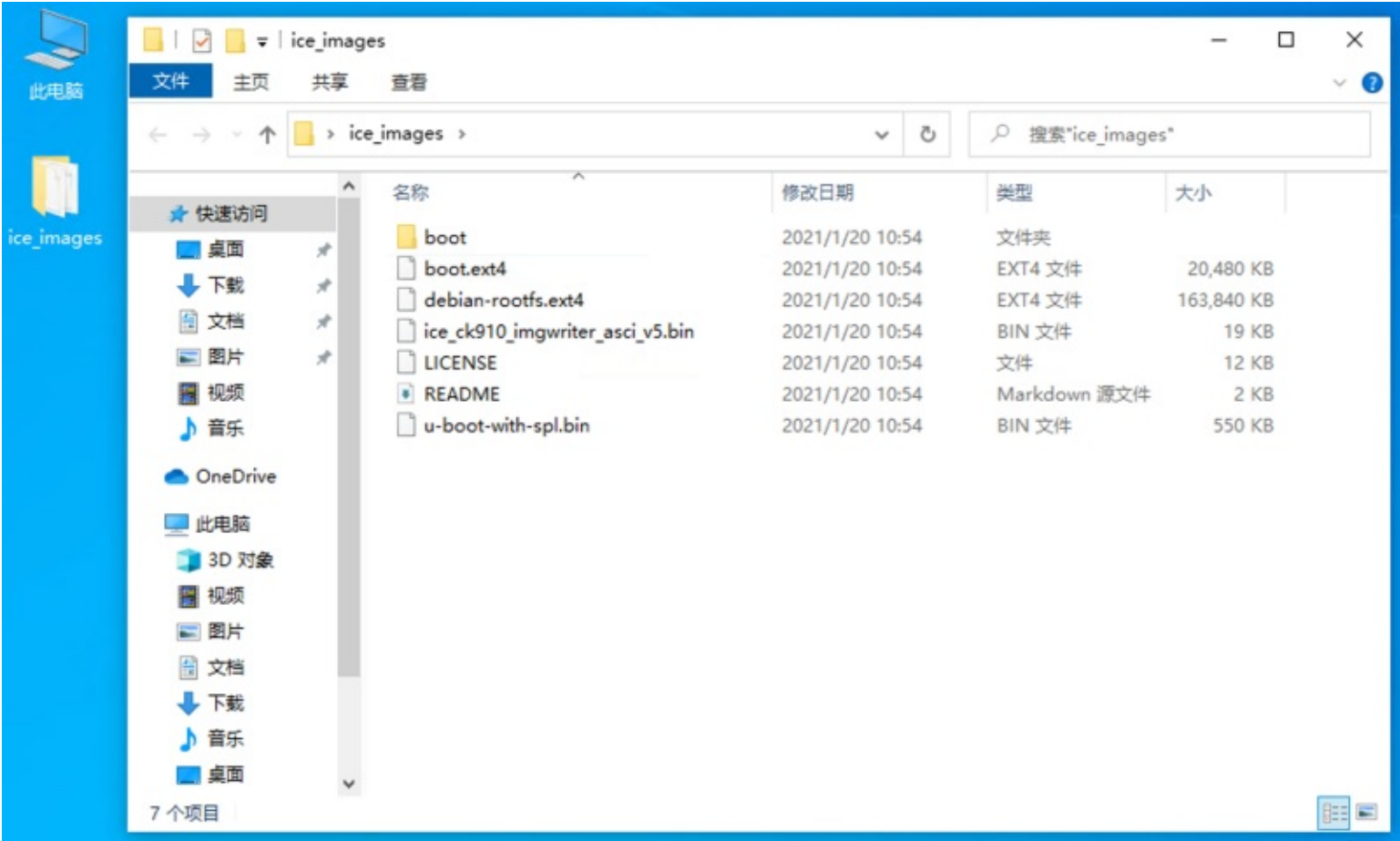
最新 fastboot 工具可从官网下载，下载地址：<https://developer.android.google.cn/studio/releases/platform-tools>，点击下载适用于 Windows 的 SDK Platform-Tools，下载 zip 文件为 platform-tools_rxx.x.x-windows.zip，将 zip 文件中 platform-tools 文件夹解压到电脑上，下图为解压到桌面：



使用其中 `fastboot.exe` 和 `AdbWinApi.dll` 两个文件，fastboot 工具需要在 cmd 终端下使用命令行执行。

第二步：下载镜像

访问 http://mirrors.aliyun.com/thead/images/riscv64-ice/ice_images.zip 下载最新镜像并解压到电脑上，如 `D:\` 下：



需要用到的是如下几个文件：

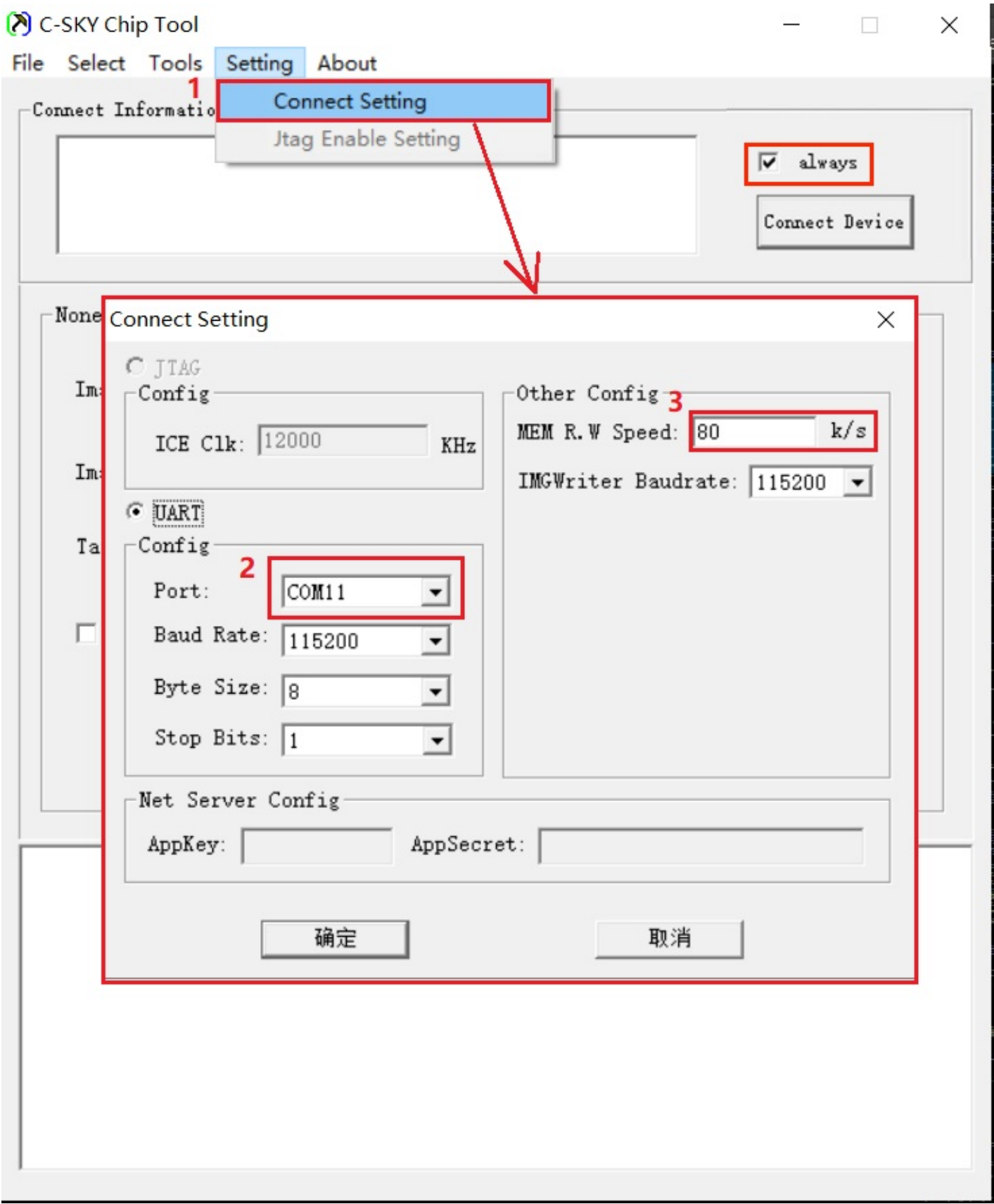
- `ice_ck910_imgwriter_asci_v5.bin`：CCT 烧写 u-boot 镜像工具
- `u-boot-with-spl_ice-rvb.bin`：ICE-RVB uboot 引导镜像文件
- `u-boot-with-spl_ice-evb.bin`：ICE-EVB uboot 引导镜像文件
- `boot.ext4`：/boot 分区文件系统，ext4 格式，包含 fw_jump.bin、uImage、hw.dtb
- `debian-rootfs.ext4`：Debian 基础根文件系统镜像，ext4 格式

第三步：烧写 u-boot

CCT 连接配置

CCT 软件运行界面如下图所示，首先配置 CCT 的参数，其中：

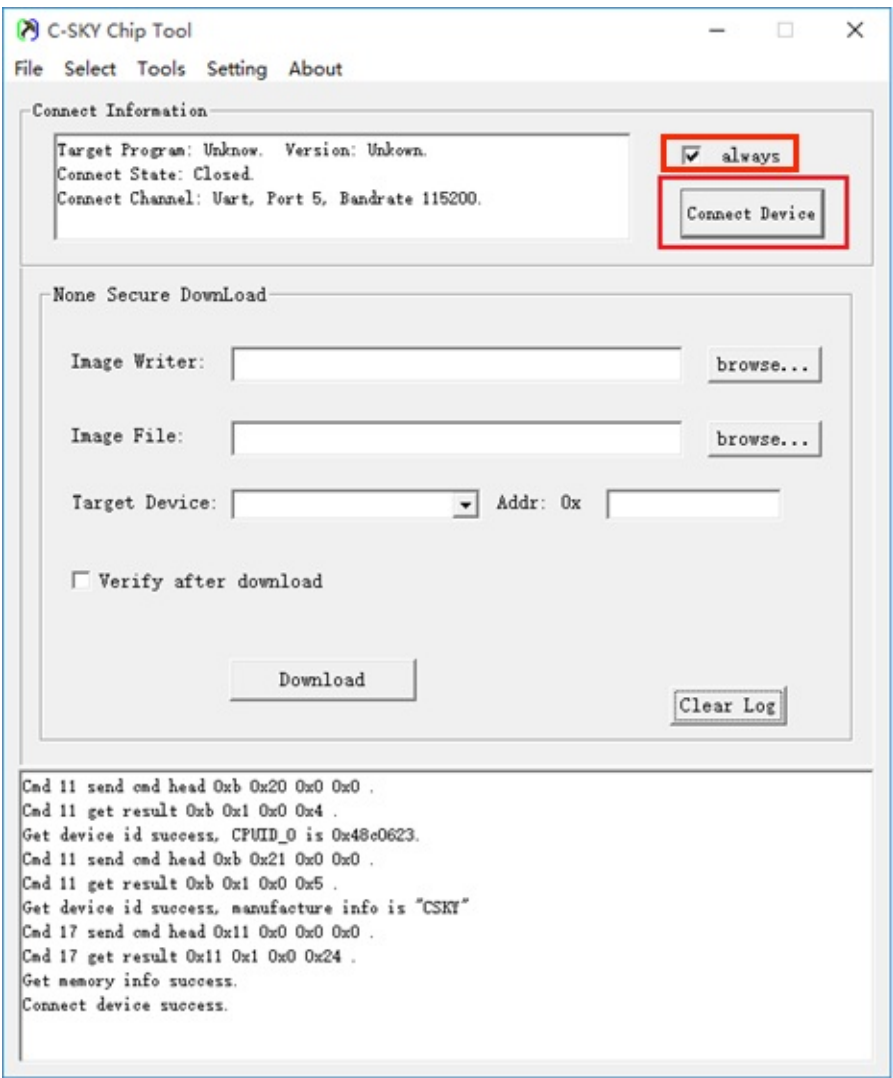
- Port 配置的是 CCT 烧写串口：根据实际电脑情况设置相应串口 Port
- Mem R.W Speed 是镜像传输协议速度：使用默认值



连接设备

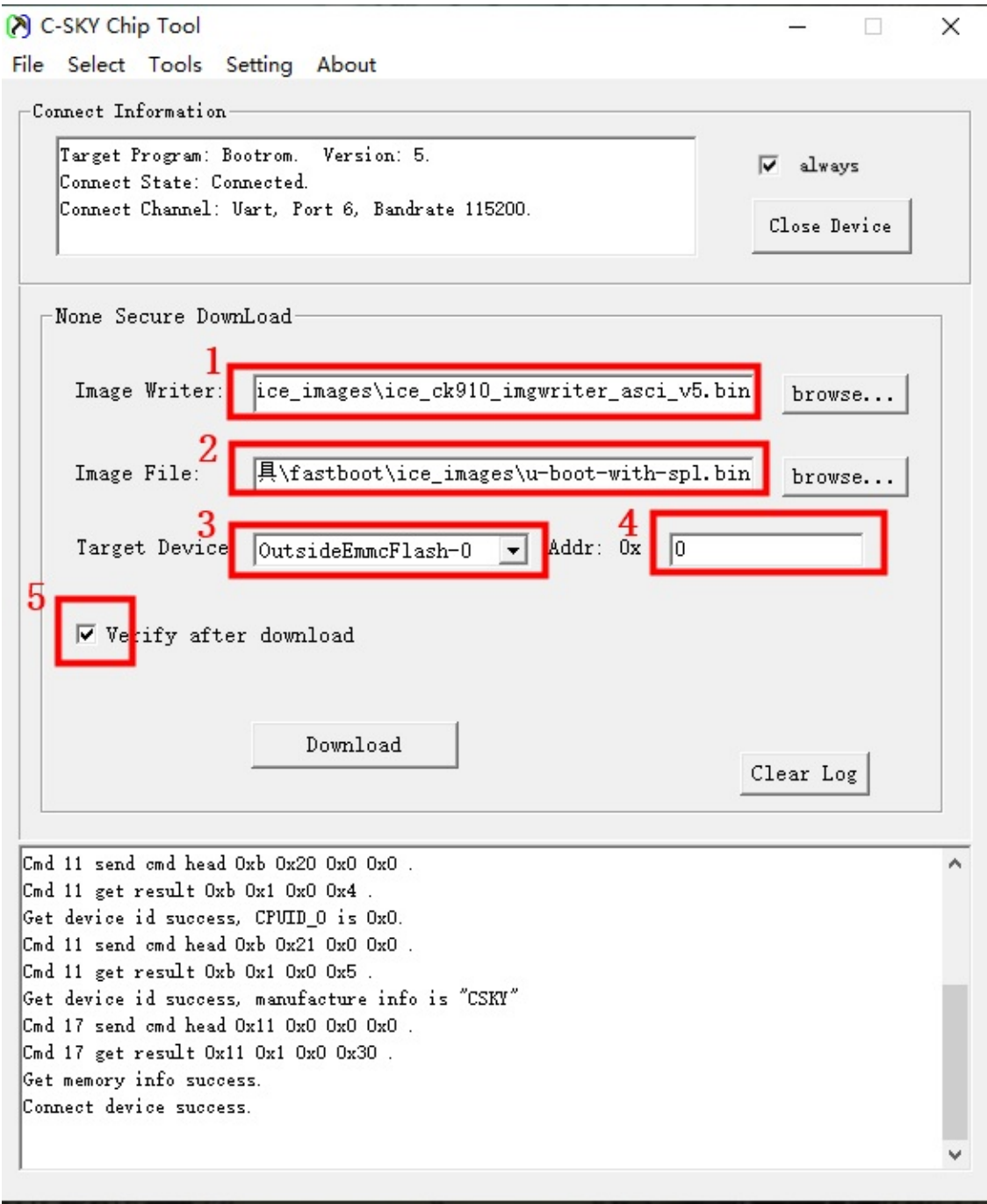
操作步骤：

EVB 开发板上电 -> 确认勾选 `always` ，点击 `Connect Device` —> 按 EVB 开发板上 `系统复位键` ，复位开发板，出现 "Connect device success." 即表示连接成功，进入下一步镜像烧写步骤。



配置烧写目标参数

连接成功之后，需要配置镜像烧写参数，配置方法如下图所示：



1. Image Writer: 设置 ImageWriter 固件文件
2. Image File: 设置将要烧写到开发板的镜像文件
3. Target Device: 镜像烧写的目标设备（常见选择为“OutsideEmmcFlash-0”，意为 eMMC 的 boot0 区域）
4. Addr: 镜像烧写到存储设备的偏移值。（FYI，图中文件名为示意文件名，具体视产品项目而定）

开始烧写

点击用户界面上的 `Download` 按钮，镜像就会开始自动下载与烧写。完成后，在完成之后，在用户界面的日志窗口中，将会出现 “Verify data success.” 的内容提示，表示镜像完成了烧写。

第四步：烧写开发板镜像

开发板 u-boot 配置

参考 [开发板接线](#) 《系统连接图》完成开发板的串口连接，使用Console 串口进行控制。

串口终端工具配置

PuTTY是一款集成虚拟终端、系统控制台和网络文件传输为一体的自由及开放源代码的程序，它支持多种网络协议，可从官网下载：

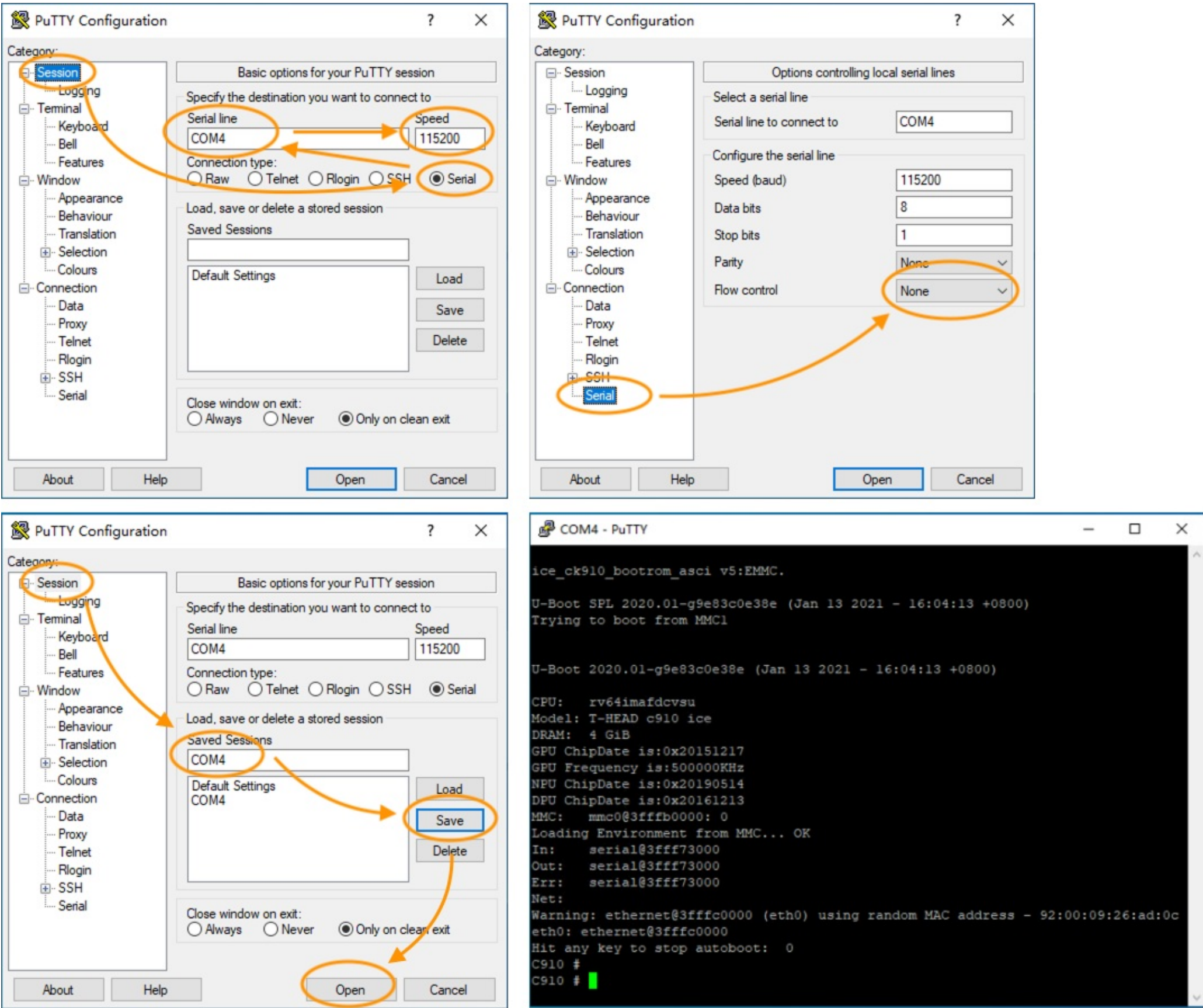
<https://www.putty.org/>，根据实际电脑系统下载 32位或 64位版本。

32位版本(<https://the.earth.li/~sgtatham/putty/latest/w32/putty-0.74-installer.msi>)

64位版本(<https://the.earth.li/~sgtatham/putty/latest/w64/putty-64bit-0.74-installer.msi>)

启动 PuTTY，进行串口参数设置。

1. 按照实际电脑系统情况，在 `Serial line` 设置串口号，`Speed` 设为 `115200`。
2. 将 `Flow control` 设置为 `None`。注意：此步骤一定要设置正确，否则会造成串口终端无法输入。
3. 保存配置，下次启动直接在 `Saved Sessions` 中点击相应配置即可连接。
4. 启动 PuTTY 串口终端。



开发板重新开机后进入 u-boot 命令模式，当 Console 串口出现如下提示时，按任意键即可进行命令模式。关于 u-boot 的详细命令介绍可参见：[u-boot](#) 章节中关于命令使用的介绍。

```
Warning: ethernet@3fffc0000 (eth0) using random MAC address - a6:7d:bc:02:7d:4d
eth0: ethernet@3fffc0000
Hit any key to stop autoboot:  3    <--- 出现此信息时按任意键进入uboot
```

u-boot 配置命令列表如下：

```
# 恢复默认值
env default -a

# 配置 eMMC 分区
setenv uuid_rootfs "80a5a8e9-c744-491a-93c1-4f4194fd690b"
setenv partitions "name=table,size=2031KB"
setenv partitions "$partitions;name=boot,size=60MiB,type=boot"
setenv partitions "$partitions;name=root,size=-,type=linux,uuid=$uuid_rootfs"
gpt write mmc 0 $partitions

# 配置网络

# !!!!!!!!!!!!!!!!!!!!!!! 特别注意 !!!!!!!!!!!!!!!!!!!!!!!
# 如果网络中接入多块 ICE-EVB 开发板，每块开发板的 ethaddr 要唯一
setenv ethaddr 00:a0:a0:a0:a0:a1 # 如果网络中接入多块 ICE-EVB 开发板，ethaddr 要唯一
setenv ipaddr 192.168.1.100 # 开发板 IP 地址
setenv netmask 255.255.255.0
# ping 192.168.1.1 # ping 主机的IP，检验网络是否正常
saveenv # 将配置保存到 eMMC，下次启动时无需再配置

# 开启fastboot模式
fastboot udp
```

烧写 boot 分区

在电脑上使用使用 fastboot 命令完成镜像烧写，192.168.1.100 是开发板的运行 u-boot 时配置的 IP 地址，使用 fastboot 命令前确认电脑与开发板在同一个子网。

为方便操作，可将 fastboot.exe、AdbWinApi.dll 两个文件拷贝到解压后的 ice_images 目录，启动 windows 的 cmd 终端，如解压到D盘，进入 D:\ice_images 目录（下载目录根据实际电脑目录填写）：

```
cd D:\ice_images
fastboot.exe -s udp:192.168.1.100 flash boot boot.ext4
```

烧写镜像

ICE-EVB 开发板提供三种根文件系统镜像，开发者可以根据自己的需要选择下载：

镜像	下载地址	大小
基本镜像(debian-base)	http://mirrors.aliyun.com/thead/images/riscv64-ice/debian-rootfs.ext4.gz	59MB
开发镜像 (debian-dev)	http://mirrors.aliyun.com/thead/images/riscv64-ice/debian-dev-rootfs.ext4.gz	197MB
xfce 桌面(debian-desktop)	http://mirrors.aliyun.com/thead/images/riscv64-ice/debian-desktop-rootfs.ext4.gz	517MB

解压 gz 文件，可下载安装开源数据压缩程序 7-Zip，官网 <https://www.7-zip.org/>，根据实际电脑情况下载：

32位版本 (<https://www.7-zip.org/a/7z1900.exe>)

64位版本 (<https://www.7-zip.org/a/7z1900-x64.exe>)

选项一：烧写基本系统镜像

镜像文件(ice_images)中带有 debian-rootfs.ext4 镜像，可直接使用 fastboot 命令烧写：

```
fastboot.exe -s udp:192.168.1.100 flash root debian-rootfs.ext4
```

选项二：烧写开发者镜像

下载文件 <http://mirrors.aliyun.com/thead/images/riscv64-ice/debian-dev-rootfs.ext4.gz> 并解压，得到 debian-dev-rootfs.ext4 镜像，再用 fastboot 命令烧写：

```
fastboot.exe -s udp:192.168.1.100 flash root debian-dev-rootfs.ext4
```

选项三：烧写 XFCE 镜像

下载文件<http://mirrors.aliyun.com/thead/images/riscv64-ice/debian-desktop-rootfs.ext4.gz> 并解压，得到 debian-desktop-rootfs.ext4 镜像，再用 fastboot 命令烧写：

```
fastboot.exe -s udp:192.168.1.100 flash root debian-desktop-rootfs.ext4
```

第五步：启动 Linux

完成 Linux 烧写后，重新启动 ICE EVB 开发板，开发板 console 串口显示如下：

```
[ OK ] Started Serial Getty on hvc0.
[ OK ] Started Serial Getty on ttyS0.
[ OK ] Reached target Login Prompts.
[ OK ] Started System Logging Service.
[ OK ] Finished Remove Stale Onli...ext4 Metadata Check Snapshots.
[ OK ] Reached target Multi-User System.
[ OK ] Reached target Graphical Interface.
        Starting Update UTMP about System Runlevel Changes...
[ OK ] Finished Update UTMP about System Runlevel Changes.

Debian GNU/Linux bullseye/sid thead-910 ttyS0

thead-910 login: [ 94.362252] random: crng init done
[ 94.365674] random: 6 urandom warning(s) missed due to ratelimiting
```



```
thead-910 login:
```

登录用户名 `root` ，无密码，开发者可根据需要更改密码。

完成 ICE-EVB 开发板的镜像烧写后，下一步进入 [系统配置](#) 章节，进行开发板的基本系统配置。

系统配置

1. 调整分区大小：

注意：此步骤一定执行，否则在执行后面操作的时候，会报没有磁盘空间的错误。

通过 uboot 对 eMMC 的分区设置，eMMC 被分成三个区：

- table 分区对应 Linux 设备：`/dev/mmcblk0p1`
- boot 分区对应 Linux 设备：`/dev/mmcblk0p2`
- root 分区对应 Linux 设备：`/dev/mmcblk0p3`

`debian-rootfs.ext4` 与 `boot.ext4` 烧写到 eMMC后，需要通过 `resize2fs` 命令调整分区大小，命令如下：

```
# 调整根文件系统分区
resize2fs /dev/mmcblk0p3
# 调用 boot 分区
resize2fs /dev/mmcblk0p2
```

调整后，显示如下：

```
Filesystem at /dev/mmcblk0p3 is mounted on /; on-line resizing required
old_desc_blocks = 2, new_desc_blocks = 30
[ 209.862639] EXT4-fs (mmcblk0p3): resized filesystem to 3803136
The filesystem on /dev/mmcblk0p3 is now 3803136 (1k) blocks long.
```

可通过 `df -h` 命令查看磁盘空间使用情况。

2. 配置网络

动态分配IP

如果开发板接入的网络中存在DHCP 服务，可以通过命令 `dhclient` 来手动获取 IP地址，如需启动系统自动配置网络信息，需要编辑配置文件 `/etc/network/interfaces`：

配置 DHCP 动态获取 IP：

```
# 配置网卡
cat > /etc/network/interfaces <<EOF
source-directory /etc/network/interfaces.d
auto lo
iface lo inet loopback

auto eth0
iface eth0 inet dhcp
EOF

# 重启网络服务
systemctl restart networking.service
```

配置指定静态 IP：

如果网络中没有 DHCP服务，或者需要指定固定的 IP 地址，可以将网络配置成静态IP：

```
# 配置网卡
cat > /etc/network/interfaces <<EOF
source-directory /etc/network/interfaces.d
auto lo
iface lo inet loopback
```



```
auto eth0
iface eth0 inet static
    address 192.168.1.100
    netmask 255.255.255.0
    gateway 192.168.1.1
EOF

# 重启网络服务
systemctl restart networking.service
```

3. 设置时区

```
$ ln -sf /usr/share/zoneinfo/Asia/Shanghai /etc/localtime
```

4. 控制台配置

PC 串口工具：

在 Linux 系统下，可以使用 minicom 连接到 ICE-EVB 控制台，命令如下：

```
minicom -c on -b 115200 -D /dev/ttyUSB0
```

- -c on ：打开颜色显示
- -b 115200 ：ICE-EVB 串口控制台的默认波特率为：115200
- -D ：指令PC上与ICE-EVB 开发板连接的串口设备，使用时要注意串口设备是否正确

在 windows 系统下，可以使用 PuTTY 连接到 ICE-EVB 控制台。

可以使用下列命令，

ICE-EVB 配置:

- 修改控制行数：串口控制台默认每行为80个字符，改变每行字符数命令：`stty cols 120`
- 修改控制台列数：串口控制台默认每行为20行，改变行数命令：`stty rows 60`

5. 设置语言环境

此操作需要网络连接，确保开发板可以连接互联网。

```
# 安装
apt update
apt install -y locales
# 编辑 /etc/locale.gen
sed -i 's/# en_US.UTF-8 UTF-8/ en_US.UTF-8 UTF-8/g' /etc/locale.gen
sed -i 's/# zh_CN.UTF-8 UTF-8/ zh_CN.UTF-8 UTF-8/g' /etc/locale.gen
locale-gen

# 显示正在使用的 locale 和相关的环境变量
$ locale
LANG=en_US.utf8
LANGUAGE=
LC_CTYPE="en_US.utf8"
LC_NUMERIC="en_US.utf8"
LC_TIME="en_US.utf8"
LC_COLLATE="en_US.utf8"
LC_MONETARY="en_US.utf8"
LC_MESSAGES="en_US.utf8"
LC_PAPER="en_US.utf8"
LC_NAME="en_US.utf8"
LC_ADDRESS="en_US.utf8"
LC_TELEPHONE="en_US.utf8"
LC_MEASUREMENT="en_US.utf8"
LC_IDENTIFICATION="en_US.utf8"
LC_ALL=
```

```
# 查看已经生产的 locale
$ localedef --list-archive
en_US.utf8
zh_CN.utf8

# 设置默认语言环境
# 编辑 /etc/locale.conf
LANG=zh_CN.UTF-8
LC_COLLATE=zh_CN.UTF-8
LC_TIME=zh_CN.UTF-8
```

6. 安装开发环境

此操作需要网络连接，确保开发板可以连接互联网。

```
apt update
apt install -y build-essential autoconf automake
```


开发板网络配置与 SSHD

如果开发板上通过以太网卡与路由器连接，与其他电脑共在同一个网络，可以开启 SSHD 服务，通过远程 ssh 登录，来远程操作系统开发板。

检查开发板上网络

根据 [系统配置](#) 章节讲到的网络配置方式来进行网络配置，使用 `ifconfig` 命令查看开发板的IP地址：

```
root@thead-910:~# ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
    inet 192.168.1.100  netmask 255.255.255.0  broadcast 192.168.1.255
    inet6 fe80::68af:e8ff:fec6:23ff  prefixlen 64  scopeid 0x20<link>
    ether 6a:af:e8:c6:23:ff  txqueuelen 1000  (Ethernet)
    RX packets 2   bytes 410 (410.0 B)
    RX errors 0    dropped 0  overruns 0  frame 0
    TX packets 10  bytes 1040 (1.0 KiB)
    TX errors 0    dropped 0 overruns 0  carrier 0  collisions 0
    device interrupt 7   base 0xa000

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
    inet 127.0.0.1  netmask 255.0.0.0
    inet6 ::1  prefixlen 128  scopeid 0x10<host>
    loop  txqueuelen 1000  (Local Loopback)
    RX packets 0   bytes 0 (0.0 B)
    RX errors 0    dropped 0  overruns 0  frame 0
    TX packets 0   bytes 0 (0.0 B)
    TX errors 0    dropped 0 overruns 0  carrier 0  collisions 0
```

可以看到 eth0 网卡的 IP 地址为：`192.168.1.100`

在HOST 机上通过 `ping` 命令检查网络是否正常：

```
ping 192.168.1.100
```

如果看到类似下面的数据，表示网络正常：

```
root@thead-910:~# ping 192.168.1.100
PING 192.168.1.100 (192.168.1.100) 56(84) bytes of data.
 64 bytes from 192.168.1.100: icmp_seq=1 ttl=64 time=0.723 ms
 64 bytes from 192.168.1.100: icmp_seq=2 ttl=64 time=0.486 ms
 64 bytes from 192.168.1.100: icmp_seq=3 ttl=64 time=0.493 ms
 64 bytes from 192.168.1.100: icmp_seq=4 ttl=64 time=0.514 ms
 64 bytes from 192.168.1.100: icmp_seq=5 ttl=64 time=0.498 ms
 64 bytes from 192.168.1.100: icmp_seq=6 ttl=64 time=0.492 ms
 64 bytes from 192.168.1.100: icmp_seq=7 ttl=64 time=0.498 ms
```

配置开发板上的 SSH Server

安装 openssh-server

```
# 安装
apt update
apt install -y ssh openssh-server
```

配置 sshd_config

```
# 配置
vi /etc/ssh/sshd_config
```

```
# To disable tunneled clear text passwords, change to no here!  
PasswordAuthentication yes  
#PermitEmptyPasswords no  
  
PermitRootLogin yes
```

重启 sshd

```
# 重启 SSH 服务  
systemctl restart sshd
```

SSH 客户端

在主机为 Linux 环境下， `sudo apt-get install openssh-client` 安装 SSH 客户端。

生成秘钥

```
root@thead-910:~# ssh-keygen  
Generating public/private rsa key pair.  
Enter file in which to save the key (/root/.ssh/id_rsa):  
Created directory '/root/.ssh'.  
Enter passphrase (empty for no passphrase):  
Enter same passphrase again:  
Your identification has been saved in /root/.ssh/id_rsa  
Your public key has been saved in /root/.ssh/id_rsa.pub  
The key fingerprint is:  
SHA256:Yh1rjgGgVUHmK26d8A6Id2U54enDCd7cI1hg6i/rL6I root@thead-910  
The key's randomart image is:  
+---[RSA 3072]-----+  
|  oo=.                |  
| o +                  |  
|.   = . .            |  
|   o = = o           |  
|  + o % S            |  
|. + * & X             |  
|o * B X +            |  
|.oo=   o .           |  
|Eoo=+                |  
+----[SHA256]-----+  
root@thead-910:~# ls ~/.ssh/  
id_rsa  id_rsa.pub
```

- id_rsa 私钥
- id_rsa.pub 公钥

将公钥上传到 ICE 开发板

```
root@thead-910:~# ssh-copy-id root@192.168.1.100  
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/root/.ssh/id_rsa.pub"  
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are already installed  
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is to install the new keys  
root@192.168.1.100's password: <----- 输入 root 的密码  
  
Number of key(s) added: 1  
  
Now try logging into the machine, with:  "ssh 'root@192.168.1.100'"  
and check to make sure that only the key(s) you wanted were added.
```

远程登录


```
ssh root@192.168.1.100
```

登录成功后显示信息如下：

```
Linux thead-910 5.10.4 #2 SMP Sun Jan 3 09:04:16 CST 2021 riscv64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Tue Jan 12 03:38:48 2021
root@thead-910:~#
```

上传文件

可以使用 `scp` 命令将PC 上的文件拷贝到 ICE-EVB 上，也可以将 ICE-EVB 的文件拷贝到 PC上。

上传文件到 ICE-EVB

```
# 将电脑上的文件拷贝到 ICE-EVB 开发板的 root 账号的 $HOME 目录 (/root) 下：
scp helloworld root@192.168.1.100:~/
```

从开发板上下载文件

```
# 将开发板上 root 账号的 $HOME 下的 helloworld 文件下载到电脑上
scp root@192.168.1.100:~/helloworld .
```

更详细的 `ssh` 的使用方法，可以通过 <http://www.openssh.com> 学习。

- <https://man.openbsd.org/ssh.1>
- <https://man.openbsd.org/scp.1>

Linux 包管理

目前，大多数类 Unix 操作系统都提供提供了一种中心化的软件包管理机制，可以用来在操作系统上安装、删除、升级、配置和管理软件包。软件包管理器可以是像“软件中心”，在 Linux 操作系统中，最常见的两种包分别是 deb 和 rpm。本指南旨在快速地介绍如何使用包管理工具来安装软件包。

ICE-EVB 开发板使用的是 debian 环境，采用 DEB 的包管理机制。

apt 包工具

在 Debian 系统中，采用deb(debian) 形式的软件包来进行管理的。安装软件可以使用如下命令，，其中 xxxx.deb 为要安装的软件包的名字：

```
sudo dpkg -i xxxx.deb
```

命令中的 dpkg(Debian Packager) 是 Debian 专门开发的包管理工具，它可以用来安装、更新和移除软件，安装时它需要使用 deb 类型的软件包。

安装软件包

```
apt-get install PackageName # 普通安装
apt-get install PackageName=VersionName # 安装指定包的指定版本
apt-get --reinstall install PackageName # 重新安装

apt-get build-dep PackageName # 安装源码包所需要的编译环境
apt-get -f install # 修复依赖关系

apt-get source PackageName # 下载软件包的源码
```

卸载软件包

```
apt-get remove PackageName # 删除软件包，保留配置文件

apt-get --purge remove PackageName # 删除软件包，同时删除配置文件
apt-get purge PackageName # 删除软件包，同时删除配置文件

apt-get autoremove PackageName # 删除软件包，同时删除为满足依赖
# 而自动安装且不再使用的软件包

apt-get --purge autoremove PackageName # 删除软件包，删除配置文件，
# 删除不再使用的依赖包

apt-get clean && apt-get autoclean # 清除已下载的软件包和旧软件包
```

更新软件包

```
apt-get update # 更新安装源（Source）
apt-get upgrade # 更新已安装的软件包
apt-get dist-upgrade # 更新已安装的软件包（识别并处理依赖关系的改变）
```

查询软件包

```
dpkg -l # 列出已安装的所有软件包

apt-cache search PackageName # 搜索软件包
apt-cache show PackageName # 获取软件包的相关信息，如说明、大小、版本等

apt-cache depends PackageName # 查看该软件包需要哪些依赖包
apt-cache rdepends PackageName # 查看该软件包被哪些包依赖
```



```
apt-get check                                # 检查是否有损坏的依赖
```

apt-get --help

执行命令: `apt-get --help` , 输出:

```
apt 1.0.1ubuntu2, 用于 amd64 构架, 编译于 Dec  8 2016 16:23:38
用法: apt-get [选项] 命令
      apt-get [选项] install|remove 软件包1 [软件包2 ...]
      apt-get [选项] source 软件包1 [软件包2 ...]

apt-get 是一个用于下载和安装软件包的简易命令行界面。
最常用命令是 update 和 install。

命令:
  update - 取回更新的软件包列表信息
  upgrade - 进行一次升级
  install - 安装新的软件包(注: 软件包名称是 libc6 而非 libc6.deb)
  remove - 卸载软件包
  autoremove - 卸载所有自动安装且不再使用的软件包
  purge - 卸载并清除软件包的配置
  source - 下载源码包文件
  build-dep - 为源码包配置所需的编译依赖关系
  dist-upgrade - 发布版升级, 见 apt-get(8)
  dselect-upgrade - 根据 dselect 的选择来进行升级
  clean - 删除所有已下载的包文件
  autoclean - 删除已下载的旧包文件
  check - 核对以确认系统的依赖关系的完整性
  changelog - 下载指定软件包, 并显示其changelog
  download - 下载指定的二进制包到当前目录

选项:
  -h  本帮助文档。
  -q  让输出可作为日志 - 不显示进度
  -qq 除了错误外, 什么都不输出
  -d  仅仅下载 - 【不】安装或解开包文件
  -s  不作实际操作。只是依次模拟执行命令
  -y  对所有询问都回答是(Yes), 同时不作任何提示
  -f  当出现破损的依赖关系时, 程序将尝试修正系统
  -m  当有包文件无法找到时, 程序仍尝试继续执行
  -u  显示已升级的软件包列表
  -b  在下载完源码包后, 编译生成相应的软件包
  -V  显示详尽的版本号
  -c=? 读取指定配置文件
  -o=? 设置任意指定的配置选项, 例如 -o dir::cache=/tmp

请查阅 apt-get(8)、sources.list(5) 和 apt.conf(5)的参考手册
以获取更多信息和选项。

      本 APT 具有超级牛力
```

软件源

使用 `apt-get install` 安装软件时, 可注意到它是从官方源地址下载的, 但默认官方的软件仓库都设置在国外, 这种情况下经常出现安装更新缓慢甚至下载超时等异常。实际上, 国内有许多第三方软件仓库镜像源, 可以从这些国内仓库安装软件工具, 加快下载速度。支持 RISC-V 的 debian 系统的国内镜像源有:

- 阿里云镜像源: <http://mirrors.aliyun.com/debian-ports>
- 腾讯软件源: <http://mirrors.cloud.tencent.com/debian-ports>

软件源修改

在 Debian 系统中, 软件源的配置是记录在文件 `/etc/apt/sources.list` 中的, 我们可以通过 `cat` 命令输出该文件的内容, 查看自己电脑上的源配置:

```
# 查找软件源
cat /etc/apt/sources.list

deb http://mirrors.aliyun.com/debian-ports sid main
deb http://mirrors.aliyun.com/debian-ports unreleased main
deb http://mirrors.aliyun.com/thead/debian-riscv64 sid main
```

在 /etc/apt/sources.list 文件中，每一行都代表了一个软件源，每行的软件源描述又可以分为四个部分，见下表：

软件包格式	镜像源地址	版本代号	软件包类型
deb/deb-src	http://mirrors.aliyun.com/debian-ports	sid	main

可以通过浏览 <http://mirrors.aliyun.com/debian-ports/dists> 可以查看到对应 版本代号：

```
Index of /debian-ports/dists/
../
experimental/      11-Jan-2021 14:07      -
sid/                11-Jan-2021 14:06      -
unreleased/        11-Jan-2021 14:07      -
unstable/          11-Jan-2021 14:06      -
```

概述

本章介绍如何 ICE-SDK 是提供的一些软件，这些软件包括针对 CPU 进行的性能测试与分析，如：

- [cpuinfo](#)
- [dhrystone](#)
- [coremark](#)
- [whetstone](#)
- [cache_calibrator](#)
- [stress_ng](#)
- [perf](#)

还提供 GPU 显卡的应用示例，展现 ICE 芯片的 GPU 的性能，XFce 的桌面系统的安全与体验。另外，在第六章中还提供了在 ICE-EVB 开发板上运行 Android 10 的演示，可以跳到 [Android 烧写](#) 开始玩转 Android。

概述

Linux 系统中提供CPU 性能分析工具，常见的工具如下表：

软件包名	ICE EVB数据	简介
dhrystone	5.8 DMIPS/MHz	常用于处理器的整型运算性能的测量。把在VAX-11/780机器上的测试结果1757 Dhrystones/s定义为1 Dhrystone MIPS(百万条指令每秒)
coremark	6.948	Coremark是一个用来计算整数运算能力。一般按coremark score/cpu mhz 给出性能数据
whetstone	4.2 MIPS/MHz	主要用于对浮点运算、整数运算、涉及到数组下标索引、子程序调用、参数传递、条件转移和三角/超越函数等。一般按whetstone score/cpu mhz 给出性能数据
linpack	542.5 KFLOPS/MHz	通过利用高性能计算机，用高斯消元法求解N元一次稠密线性代数方程组的测试，评价高性能计算机的浮点性能。测试结果以浮点运算每秒（Flops）给出
stream	Copy: 6965.0 Scale: 7020.0 Add: 6095.7 Triad: 6105.9 (MB/S/GHz)	Stream测试是内存测试中业界公认的内存带宽性能测试基准工具
Sysbench	test=threads 5.1	Sysbench是一个开源的、模块化的、跨平台的多线程性能测试工具，可以用来进行CPU、内存、磁盘 I/O、线程、数据库的性能测试。
nbench	MEMORY : 10.186 INTEGER : 12.479 FLOATING-POINT : 15.010	nbench是一个简单的用于测试处理器、存储器性能的基准测试程序
iperf3	3.77Gb/S	测量网络性能
cache_calibrator	----	测量 cache 相关的性能参数
stress_ng	----	压力测试神器

查看CPU信息

在linux操作系统中，CPU的信息在启动的过程中被装载到虚拟目录/proc下的cpufreq文件中，我们可以通过 cat /proc/cpufreq 查看一下：

```
cat /proc/cpufreq
```

显示如下：

```
root@thead-910:~# cat /proc/cpufreq
processor      : 0
hart          : 0
isa           : rv64imafdcsv
mmu           : sv39
model name    : T-HEAD C910
freq          : 1.2GHz
icache       : 64kB
dcache       : 64kB
l2cache      : 2MB
tlb          : 1024 4-ways
cache line    : 64Bytes
address sizes : 40 bits physical, 39 bits virtual
vector version : 0.7.1

processor      : 1
hart          : 1
isa           : rv64imafdcsv
mmu           : sv39
model name    : T-HEAD C910
freq          : 1.2GHz
icache       : 64kB
dcache       : 64kB
l2cache      : 2MB
tlb          : 1024 4-ways
cache line    : 64Bytes
address sizes : 40 bits physical, 39 bits virtual
vector version : 0.7.1
```

通过 cpufreq 可以看到主要信息：

- 两颗 RISC-V 64位处理器，型号 T-HEAD C910
- CPU 主频：1.2GHz
- 支持的 ISA 特性：
 - i: 基础整形指令集
 - m: 整型乘除法指令扩展
 - a: 原子操作指令扩展
 - f: 单精度指令扩展
 - d: 双精度指令扩展
 - c: 16位压缩指令扩展
 - s: 特权态支持
 - u: 用户态支持
- I/D CACHE: 64KB
- 二级 CACHE：2MB
- MMU TLB：4路1024个表项
- 物理地址位宽：40位
- 虚拟地址位宽: 39位
- 矢量加速指令集版本: 0.7.1

测量处理器运算能力 dhrystone

Dhrystone是测量处理器运算能力的最常见基准程序之一，常用于处理器的整型运算性能的测量。Dhrystone的计量单位为每秒计算多少次Dhrystone，后来把在VAX-11/780机器上的测试结果1757 Dhrystones/s定义为1 Dhrystone MIPS(百万条指令每秒)。但其也有许多不足，Dhrystone不仅不适合于作为嵌入式系统的测试向量，甚至在其大多数场合下都不适合进行应用。

安装 **dhrystone**：

```
apt install -y dhrystone
```

ICE-EVB 开发板测试：

```
dhrystone
```

在 **ICE-EVB** 开发板上运行结果如下：

```
Dhrystone Benchmark, Version 2.1 (Language: C)

Execution starts, 600000000 runs through Dhrystone
Execution ends

Final values of the variables used in the benchmark:

Int_Glob:          5
    should be:    5
Bool_Glob:         1
    should be:    1
Ch_1_Glob:         A
    should be:    A
Ch_2_Glob:         B
    should be:    B
Arr_1_Glob[8]:     7
    should be:    7
Arr_2_Glob[8][7]: 600000010
    should be:    Number_Of_Runs + 10
Ptr_Glob->
  Ptr_Comp:        481216
    should be:    (implementation-dependent)
  Discr:          0
    should be:    0
  Enum_Comp:       2
    should be:    2
  Int_Comp:        17
    should be:    17
  Str_Comp:        DHRYSTONE PROGRAM, SOME STRING
    should be:    DHRYSTONE PROGRAM, SOME STRING
Next_Ptr_Glob->
  Ptr_Comp:        481216
    should be:    (implementation-dependent), same as above
  Discr:          0
    should be:    0
  Enum_Comp:       1
    should be:    1
  Int_Comp:        18
    should be:    18
  Str_Comp:        DHRYSTONE PROGRAM, SOME STRING
    should be:    DHRYSTONE PROGRAM, SOME STRING
Int_1_Loc:         5
    should be:    5
Int_2_Loc:        13
    should be:    13
Int_3_Loc:         7
    should be:    7
```



```
Enum_Loc:          1
    should be:      1
Str_1_Loc:          DHRYSTONE PROGRAM, 1'ST STRING
    should be:      DHRYSTONE PROGRAM, 1'ST STRING
Str_2_Loc:          DHRYSTONE PROGRAM, 2'ND STRING
    should be:      DHRYSTONE PROGRAM, 2'ND STRING

Microseconds for one run through Dhrystone:      0.1
Dhrystones per Second:                            12329495.0
```

换算：

$12329495/1757/1200=5.8 \text{ DMIPS/MHz}$

衡量CPU性能 coremark

CoreMark是用来衡量CPU性能的标准。包含如下的运算法则：列举（寻找并排序），数学矩阵操作（普通矩阵运算）和状态机（用来确定输入流中是否包含有效数字），最后还包括CRC（循环冗余校验）。

安装 **coremark**：

```
apt install -y coremark
```

ICE-EVB 开发板测试命令：

```
coremark
```

在 **ICE-EVB** 开发板上运行结果如下：

```
2K performance run parameters for coremark.
CoreMark Size      : 666
Total ticks        : 13193
Total time (secs): 13.193000
Iterations/Sec     : 8337.754870
Iterations         : 110000
Compiler version  : GCC8.4.0
Compiler flags    : -O2 -O3 -static -funroll-all-loops -finline-limit=500 -fgcse-sm -fno-schedule-insns --param max-rtl-if-conversion-unpredictable-cost=100 -msignedness-cmpiv -fno-code-hoisting -mno-thread-jumps1 -mno-iv-adjust-addr-cost -mno-expand-split-imm -lrt
Memory location   : Please put data memory location here
                   (e.g. code in flash, data on heap etc)
seedcrc           : 0xe9f5
[0]crclist        : 0xe714
[0]crcmatrix      : 0x1fd7
[0]crcstate       : 0x8e3a
[0]crcfinal       : 0x33ff
Correct operation validated. See readme.txt for run and reporting rules.
CoreMark 1.0 : 8337.754870 / GCC8.4.0 -O2 -O3 -static -funroll-all-loops -finline-limit=500 -fgcse-sm -fno-schedule-insns --param max-rtl-if-conversion-unpredictable-cost=100 -msignedness-cmpiv -fno-code-hoisting -mno-thread-jumps1 -mno-iv-adjust-addr-cost -mno-expand-split-imm -lrt / Heap
```

换算：

$8337.754870 / 1200 = 6.948$

其中 1200表示 1200MHz，也就是 1.2GHz，也即 ICE 开发板的 CPU 频率。

whetstone

whetstone是一个用于测量CPU浮点计算性能的工具。

安装 **whetstone**：

```
apt install -y whetstone
```

ICE-EVB 开发板测试：

```
root@thead-910:~# whetstone 1000000
Loops: 1000000, Iterations: 1, Duration: 19.778 sec.
C Converted Double Precision Whetstones: 5056.029 MIPS
```

换算：

$5056.029 / 1200 = 4.2 \text{ MIPS/MHz}$

其中 1200表示 1200MHz，也就是 1.2GHz，也即 ICE 开发板的 CPU 频率。

linpack

通过利用高性能计算机，用高斯消元法求解N元一次稠密线性代数方程组的测试，评价高性能计算机的浮点性能。

安装 **linpack**：

```
apt install -y linpack
```

ICE-EVB 开发板测试：

```
root@thead-910:~# linpack
Unrolled Double Precision Linpack

Unrolled Double Precision Linpack

      norm. resid      resid      machep      x[0]-1      x[n-1]-1
      2.7      2.40728912e-13  2.22044605e-16  4.08562073e-14 -2.94209102e-14
times are reported for matrices of order  200
      dgefa      dgesl      total      kflops      unit      ratio
times for array with leading dimension of  201
      0.01      0.00      0.01      644521      0.00      0.15
      0.01      0.00      0.01      646445      0.00      0.15
      0.01      0.00      0.01      647683      0.00      0.15
      0.01      0.00      0.01      651032      0.00      0.15
times for array with leading dimension of 200
      0.01      0.00      0.01      691623      0.00      0.14
      0.01      0.00      0.01      693661      0.00      0.14
      0.01      0.00      0.01      696428      0.00      0.14
      0.01      0.00      0.01      696383      0.00      0.14
Unrolled Double Precision 651032 Kflops ; 10 Reps
```

换算：

651032/1200=542.5 KFLOPS/MHz

其中 1200表示 1200MHz，也就是 1.2GHz，也即 ICE 开发板的 CPU 频率。

stream

Stream测试是内存测试中业界公认的内存带宽性能测试基准工具。

安装 **stream**：

```
apt install -y stream
```

ICE-EVB 开发板测试：

```
root@thead-910:~# stream
-----
STREAM version $Revision: 5.10 $
-----
This system uses 8 bytes per array element.
-----
Array size = 10000000 (elements), Offset = 0 (elements)
Memory per array = 76.3 MiB (= 0.1 GiB).
Total memory required = 228.9 MiB (= 0.2 GiB).
Each kernel will be executed 10 times.
  The *best* time for each kernel (excluding the first iteration)
  will be used to compute the reported bandwidth.
-----
Your clock granularity/precision appears to be 1 microseconds.
Each test below will take on the order of 30088 microseconds.
  (= 30088 clock ticks)
Increase the size of the arrays if this shows that
you are not getting at least 20 clock ticks per test.
-----
WARNING -- The above is only a rough guideline.
For best results, please be sure you know the
precision of your system timer.
-----
Function      Best Rate MB/s  Avg time     Min time     Max time
Copy:          6965.0    0.022986     0.022972     0.023003
Scale:         7020.0    0.022831     0.022792     0.022890
Add:           6095.7    0.039484     0.039372     0.039601
Triad:         6105.9    0.040019     0.039306     0.042522
-----
Solution Validates: avg error less than 1.000000e-13 on all three arrays
-----
```

sysbench

sysbench是一个开源的、模块化的、跨平台的多线程性能测试工具，可以用来进行CPU、内存、磁盘I/O、线程、数据库的性能测试。

安装 sysbench：

```
apt install -y sysbench
```

ICE-EVB 开发板测试：

- 测试线程：

```
```bash root@thead-910:/# ./sysbench --test=threads --num-threads=32 --thread-yields=100 --thread-lock=8 run sysbench 0.5: multi-threaded system evaluation benchmark
```

```
Running the test with following options: Number of threads: 32 Random number generator seed is 0 and will be ignored
```

Threads started!

General statistics: total time: 2.9138s total number of events: 10000 total time taken by event execution: 93.1165s response time: min: 0.14ms avg: 9.31ms max: 82.38ms approx. 95 percentile: 32.83ms

Threads fairness: events (avg/stddev): 312.5000/14.83 execution time (avg/stddev): 2.9099/0.00

换算：

14.83/2.9099=5.1

其他测试项目：

- 测试CPU：

```
```bash root@thead-910:~# ./sysbench --test=cpu --cpu-max-prime=20000 --num-threads=2 run sysbench 0.5: multi-threaded system evaluation benchmark
```

```
Running the test with following options: Number of threads: 2 Random number generator seed is 0 and will be ignored
```

```
Primer numbers limit: 20000
```

Threads started!

```
General statistics: total time: 15.0576s total number of events: 10000 total time taken by event execution: 30.1038s response time: min: 3.00ms avg: 3.01ms max: 3.12ms approx. 95 percentile: 3.01ms
```

```
Threads fairness: events (avg/stddev): 5000.0000/0.00 execution time (avg/stddev): 15.0519/0.00
```

- 测试IO：


```
root@thead-910:~# .sysbench --test=fileio --num-threads=16 --file-total-size=2G --file-test-mode=rndrw prepare
sysbench 0.5: multi-threaded system evaluation benchmark

128 files, 16384Kb each, 2048Mb total
Creating files for the test...
Extra file open flags: 0
Creating file test_file.0
Creating file test_file.1
...
Creating file test_file.125
Creating file test_file.126
Creating file test_file.127
2147483648 bytes written in 110.50 seconds (18.53 MB/sec).
```

```
```bash root@thead-910:~# sysbench --test=fileio --num-threads=20 --file-total-size=2G --file-test-mode=rndrw run sysbench 0.5: multi-threaded system
evaluation benchmark
```

Running the test with following options: Number of threads: 20 Random number generator seed is 0 and will be ignored

Extra file open flags: 0 128 files, 16Mb each 2Gb total file size Block size 16Kb Number of IO requests: 10000 Read/Write ratio for combined random IO test: 1.50 Periodic FSYNC enabled, calling fsync() each 100 requests. Calling fsync() at the end of test, Enabled. Using synchronous I/O mode Doing random r/w test Threads started!

Operations performed: 6000 reads, 4000 writes, 12800 Other = 22800 Total Read 93.75Mb Written 62.5Mb Total transferred 156.25Mb (34.834Mb/sec) 2229.38 Requests/sec executed

General statistics: total time: 4.4856s total number of events: 10000 total time taken by event execution: 0.8171s response time: min: 0.01ms avg: 0.08ms max: 20.83ms approx. 95 percentile: 0.08ms

Threads fairness: events (avg/stddev): 500.0000/179.75 execution time (avg/stddev): 0.0409/0.02

```
```bash
root@thead-910:~# ./sysbench --test=fileio --num-threads=20 --file-total-size=2G --file-test-mode=rndrw cleanup
sysbench 0.5: multi-threaded system evaluation benchmark

Removing test files...
```

• 测试内存：

```
```bash root@thead-910:~# sysbench --test=memory --memory-block-size=8k --memory-total-size=1G run sysbench 0.5: multi-threaded system
evaluation benchmark
```

Running the test with following options: Number of threads: 1 Random number generator seed is 0 and will be ignored

Threads started!

Operations performed: 131072 (374615.49 ops/sec)

1024.00 MB transferred (2926.68 MB/sec)

General statistics: total time: 0.3499s total number of events: 131072 total time taken by event execution: 0.2768s response time: min: 0.00ms avg: 0.00ms max: 0.06ms approx. 95 percentile: 0.00ms

Threads fairness: events (avg/stddev): 131072.0000/0.00 execution time (avg/stddev): 0.2768/0.00

- 测试mutex：

```
```bash
root@thead-910:~# sysbench --test=mux --num-threads=64 run
sysbench 0.5: multi-threaded system evaluation benchmark
```

Running the test with following options:
Number of threads: 64

```
Random number generator seed is 0 and will be ignored

Threads started!

General statistics:
  total time:                1.2616s
  total number of events:    64
  total time taken by event execution: 76.6870s
  response time:
    min:                    1066.46ms
    avg:                    1198.23ms
    max:                    1254.10ms
    approx. 95 percentile: 1251.30ms

Threads fairness:
  events (avg/stddev):      1.0000/0.00
  execution time (avg/stddev): 1.1982/0.05
```

nbench

nbench是一个简单的用于测试处理器、存储器性能的基准测试程序。

安装 **nbench**：

```
apt install -y nbench
```

ICE-EVB 开发板测试：

```
root@thead-910:~# nbench

BYTEmark* Native Mode Benchmark ver. 2 (10/95)
Index-split by Andrew D. Balsa (11/97)
Linux/Unix* port by Uwe F. Mayer (12/96,11/97)

TEST                : Iterations/sec.   : Old Index   : New Index
                    :                               : Pentium 90* : AMD K6/233*
-----:-----:-----:-----
NUMERIC SORT        :          676.01 :          17.34 :          5.69
STRING SORT          :          148.55 :          66.38 :         10.27
BITFIELD             :       1.6208e+08 :          27.80 :          5.81
FP EMULATION         :          154.38 :          74.08 :         17.09
FOURIER              :          21743 :          24.73 :         13.89
ASSIGNMENT           :          17.95 :          68.30 :         17.72
IDEA                 :          3738.9 :          57.19 :         16.98
HUFFMAN              :          1657.3 :          45.96 :         14.68
NEURAL NET           :          14.84 :          23.84 :         10.03
LU DECOMPOSITION     :          649.13 :          33.63 :         24.28

=====ORIGINAL BYTEMARK RESULTS=====
INTEGER INDEX        : 45.842
FLOATING-POINT INDEX: 27.063
Baseline (MSDOS*)    : Pentium* 90, 256 KB L2-cache, Watcom* compiler 10.0
=====LINUX DATA BELOW=====
CPU                  : Dual T-HEAD C910
L2 Cache             :
OS                   : Linux 5.10.4
C compiler           : riscv64-unknown-linux-gnu-gcc
libc                 : static
MEMORY INDEX         : 10.186
INTEGER INDEX        : 12.479
FLOATING-POINT INDEX: 15.010
Baseline (LINUX)     : AMD K6/233*, 512 KB L2-cache, gcc 2.7.2.3, libc-5.4.38
* Trademarks are property of their respective holder.
```


cache_calibrator

cache_calibrator 可以测量 cache 相关的性能参数。

安装 **cache_calibrator**：

```
apt install -y cache-calibrator
```

ICE-EVB 开发板测试：

```
cache_calibrator 1200 10M file
```

在 **ICE-EVB** 开发板上运行结果如下：

```
Calibrator v0.9e
(by Stefan.Manegold@cwil.nl, http://www.cwil.nl/~manegold/)
e16a9010 274364796944 4096    16
e16a9fff 274364801023 4096   4095
e16aa000 274364801024 4096     0

MINTIME = 10000

analyzing cache throughput...
      range      stride      spots      brutto-  netto-time
    10485760         8    1310720      17472      273

analyzing cache latency...
      range      stride      spots      brutto-  netto-time
    10485760         8    1310720     139776      273

analyzing TLB latency...
      range      stride      spots      brutto-  netto-time
    1474560     1152      1280      18176     1136

CPU loop + L1 access:      2.50 ns =   3 cy
      ( delay:      0.00 ns =   0 cy )

caches:
level  size    linesize  miss-latency      replace-time
   1    64 KB   128 bytes   9.19 ns = 11 cy   9.16 ns = 11 cy
   2    2 MB   128 bytes  19.05 ns = 23 cy  19.09 ns = 23 cy

TLBs:
level #entries  pagesize  miss-latency
   1      20      4 KB    3.34 ns =   4 cy

# ls
file.cache-replace-time.gp
file.TLB-miss-latency.data
file.TLB-miss-latency.gp
file.cache-miss-latency.data
file.cache-miss-latency.gp
file.cache-replace-time.data
```

其中 1200表示 1200MHz，也就是 1.2GHz，也即 ICE 开发板的 CPU 频率；跑完后生成了很多以“file”开头的文件，可以观测 cache 各个方面的性能。

iperf3

iperf 用于测量网络性能。

安装**iperf3**：

```
apt install -y iperf3
```

启动 **perf3 server**：

```
iperf3 -s > /dev/null &
```

开始测试：

```
iperf3 -c 127.0.0.1
```

在ICE 开发板上，测试数组如下：

```
root@thead-910:~# iperf3 -c 127.0.0.1
Connecting to host 127.0.0.1, port 5201
[ 5] local 127.0.0.1 port 51114 connected to 127.0.0.1 port 5201
[ ID] Interval            Transfer        Bitrate          Retr   Cwnd
[ 5]  0.00-1.00    sec    454 MBytes    3.80 Gbits/sec     0   1.12 MBytes
[ 5]  1.00-2.00    sec    452 MBytes    3.79 Gbits/sec     0   1.12 MBytes
[ 5]  2.00-3.00    sec    430 MBytes    3.61 Gbits/sec     0   1.12 MBytes
[ 5]  3.00-4.00    sec    445 MBytes    3.73 Gbits/sec     0   1.12 MBytes
[ 5]  4.00-5.00    sec    452 MBytes    3.79 Gbits/sec     0   1.50 MBytes
[ 5]  5.00-6.00    sec    442 MBytes    3.71 Gbits/sec     0   1.50 MBytes
[ 5]  6.00-7.00    sec    448 MBytes    3.76 Gbits/sec     0   1.94 MBytes
[ 5]  7.00-8.00    sec    451 MBytes    3.78 Gbits/sec     0   2.12 MBytes
[ 5]  8.00-9.00    sec    464 MBytes    3.90 Gbits/sec     0   2.81 MBytes
[ 5]  9.00-10.00   sec    458 MBytes    3.84 Gbits/sec     0   3.00 MBytes
- - - - -
[ ID] Interval            Transfer        Bitrate          Retr
[ 5]  0.00-10.00   sec    4.39 GBytes    3.77 Gbits/sec     0
[ 5]  0.00-10.00   sec    4.39 GBytes    3.77 Gbits/sec
                                     sender
                                     receiver
```

如果需要测试物理网卡的速度，可以在远程电脑上启动 iperf3 服务，然后再通过 `iperf3 -c <host_ip>` 来测试网速。

压力测试神器 stress_ng

stress-ng是stress的加强版，完全兼容stress，并在此基础上增加了几百个参数，堪称压测工具中的瑞士军刀。

安装 stress-ng：

```
apt install -y stress-ng
```

应用场景1：CPU 密集型进程（使用CPU的进程）

```
stress-ng --cpu 2 --cpu-method all
```

应用场景2：I/O 密集型进程（等待IO的进程）

```
stress-ng -i 4 --hdd 1 --timeout 600
```

应用场景3：大量进程的场景（等待CPU的进程->进程间会争抢CPU）

```
stress-ng -c 16 --timeout 600
```

性能分析工具 -- perf

Perf 是内置于 Linux 内核源码树中的性能剖析(profiling)工具，它基于事件采样原理，以性能事件为基础，支持针对处理器相关性能指标与操作系统相关性能指标的性能剖析，常用于性能瓶颈的查找与热点代码的定位。

通过它，应用程序可以利用 PMU，tracepoint 和内核中的特殊计数器来进行性能统计。它不但可以分析指定应用程序的性能问题 (per thread)，也可以用来分析内核的性能问题，当然也可以同时分析应用代码和内核，从而全面理解应用程序中的性能瓶颈。

使用 perf，您可以分析程序运行期间发生的硬件事件，比如 instructions retired ，processor clock cycles 等；您也可以分析软件事件，比如 Page Fault 和进程切换。这使得 Perf 拥有了众多的性能分析能力，举例来说，使用 Perf 可以计算每个时钟周期内的指令数，称为 IPC，IPC 偏低表明代码没有很好地利用 CPU。Perf 还可以对程序进行函数级别的采样，从而了解程序的性能瓶颈究竟在哪里等等。Perf 还可以替代 strace，可以添加动态内核 probe 点，还可以做 benchmark 衡量调度器的好坏。

安装perf

```
# 安装 perf 命令
apt install perf
```

装完之后就可以使用perf命令了。

```
usage: perf [--version] [--help] [OPTIONS] COMMAND [ARGS]

The most commonly used perf commands are:
  annotate      Read perf.data (created by perf record) and display annotateddd code
  archive      Create archive with object files with build-ids found in perfff.data file
  bench        General framework for benchmark suites
  builddid-cache  Manage build-id cache.
  builddid-list  List the buildids in a perf.data file
  c2c          Shared Data C2C/HITM Analyzer.
  config       Get and set variables in a configuration file.
  data        Data file related processing
  diff        Read perf.data files and display the differential profile
  evlist       List the event names in a perf.data file
  ftrace       simple wrapper for kernel's ftrace functionality
  inject       Filter to augment the events stream with additional informatiion
  kallsyms     Searches running kernel for symbols
  kmem        Tool to trace/measure kernel memory properties
  kvm         Tool to trace/measure kvm guest os
  list        List all symbolic event types
  lock        Analyze lock events
  mem         Profile memory accesses
  record       Run a command and record its profile into perf.data
  report       Read perf.data (created by perf record) and display the profiile
  sched       Tool to trace/measure scheduler properties (latencies)
  script       Read perf.data (created by perf record) and display trace outtput
  stat        Run a command and gather performance counter statistics
  test        Runs sanity tests.
  timechart   Tool to visualize total system behavior during a workload
  top         System profiling tool.
  version     display the version of perf binary

See 'perf help COMMAND' for more information on a specific command.
```

安装flamegraph

火焰图是脚本，只需要下载:

```
git clone https://github.com/brendangregg/FlameGraph.git
```


查看当前软硬件环境、支持的性能事件

查看所有分类事件的个数:

```
perf list | awk -F: '/Tracepoint event/ { lib[$1]++ } END { for (l in lib) { printf " %-16s %d\n", l, lib[l] } }' | sort | column
```

在 ICE 芯片运行结果如下：

```
root@thead-910:~# perf list | awk -F: '/Tracepoint event/ { lib[$1]++ } END { for (l in lib) { printf " %-16s %d\n", l, lib[l] } }' | sort | column
9p                3          iomap          9          random          16
alarmtimer        4          irq            5          ras              4
block             19         jbd2           17         raw_syscalls     2
bpf_test_run      1          kmem           13         rcu               1
bpf_trace         1          kyber          3          regmap           15
cgroup            13         mdio           1          rpcgss            26
clk               16         migrate        1          sched             20
compaction        14         mmap           1          signal           2
cpuhp             3          mmc            2          skb               3
dma_fence         7          module         5          smbus             4
dwc3              15         napi           1          sock              3
ext4              117        neigh          7          spi               7
fib               1          net            18         sunrpc            127
fib6              1          nfs            58         swiotlb           1
filelock          12         nfs4           87         task              2
filemap           4          oom            8          tcp               7
ftrace            1          page_pool      4          timer             12
gadget            24         pagemap        2          udp               1
gpio              2          percpu         5          vmscan            14
i2c               4          power          22         workqueue         4
initcall          3          printk         1          writeback         30
io_uring          14         qdisc          4          xdp               12
```

flamegraph火焰图

如果希望了解CPU在一段时间内的都运行了哪些函数以及各函数都消耗了多少时间，就可以使用On CPU火焰图，这种火焰图基于cpu-cycles事件进行采样，然后通过svg图片格式展现出来

```
dd if=/dev/zero of=/tmp/testfile bs=4K count=102400 &
perf record -e cpu-cycles -a -g sleep 1
perf script > perf.unfold
cd FlameGraph
./stackcollapse-perf.pl < ../perf.unfold | ./flamegraph.pl > ../perf.svg
```

首先在后台启动一个dd命令，让它持续运行一段时间，然后开启perf record，记录一秒钟内cpu都运行了多少个cpu-cycles，也就是时间（同时使用-g，就会一并记录运行的函数以及调用关系），再利用perf script命令将perf.data转成perf.unfold，最后利用FlameGraph工具将其转换成一个perf.svg，这是一个图形文件，用浏览器打开后会得到这样一幅图：



此图记录着函数调用关系及其cpu-cycles（时间）占比，就像一缕缕升起的火苗，所以被称之为火焰图。

火焰图还可以通过鼠标点击放大，观察其细节，如下：



安装 GPU 驱动

安装驱动

```
apt update
apt install -y ice-drivers
```

运行测试程序

```
cd /opt/offical_tests/vdk/
./tutorial4_es20
```

GPU Demo [运行效果](#)

Linux 小程序应用开发环境

下载并安装运行环境、Demo

```
apt update
apt install -y cube-miniapp
```

运行 Demo 程序

```
# 运行 Cube 小程序 Demo
cube-test 2021001142699009 pages/index/index
```

- cube-test：cube 小程序框架
- 2021001142699009：小程序 appid，这个是由 cube-test 确定入口程序 id
- pages/index/index：初始页面

css文本
背景
盒子模型
flex布局
css动画
图片测试
基础组件
swiper测试

注意：

1. 此 Demo 为离线方式小程序，实际小程序需要联网下载。
2. 小程序需要一个 HomeApp 调用，因此 Demo 直接调用，在初始页面再返回时，会返回到黑屏。

Cube 小程序简介

Cube 是一个可独立发布的跨平台 渲染引擎，在小程序这一技术体系内，仅作为一种不同于 Web(WebView) 的高性能渲染方式存在。

1. Cube 小程序特点
 - 支持使用 "类Web" 语言进行业务开发，快速开发体验
 - 兼顾性能和体验的跨平台 Native 渲染引擎，与 Native 应用相当的使用体验
2. 与 Web 的关系
 - 渲染能力支持上，基本实现了 w3c 样式规范的子集
 - 在运行架构上，不基于 HTML 语言表示, 而基于节点级别的批量指令操作
 - 在逻辑执行上，均通过 JavaScript 进行逻辑执行

示例：编译、运行 helloworld

1. 安装编译工具

```
# 全局安装
```

```
tnpm install -g @ali/mini

# 如无权限或permission denied, 请加sudo尝试安装
sudo tnpm install -g @ali/mini
```

2. 初始化 helloworld 小程序项目

```
mkdir helloworld
cd helloworld
mini init
# 选择 Cube 小程序
? 请输入项目名称: helloTinyApp
? 选择你的项目类型: Cube 小程序

# 生成后的小程序项目目录
helloTinyApp
├─ README.md
├─ app.acss
├─ app.js
├─ app.json
├─ mini.project.json
├─ pages
│   └─ index
│       ├── index.acss
│       ├── index.axml
│       ├── index.js
│       └─ index.json
```

3. 构建、打包小程序

```
# helloTinyApp Cube 小程序项目根目录下执行
mini build ./

...
Inject code for ./helloworld/helloTinyApp/miniapptools_dist/ng-main/index.html failed because of file not exist
Inject code for ./helloworld/helloTinyApp/miniapptools_dist/ng-main/index.worker.js failed because of file not exist
>>> 构建完成
小程序产物包: ./helloworld/helloTinyApp/miniapptools_dist/dist.tar

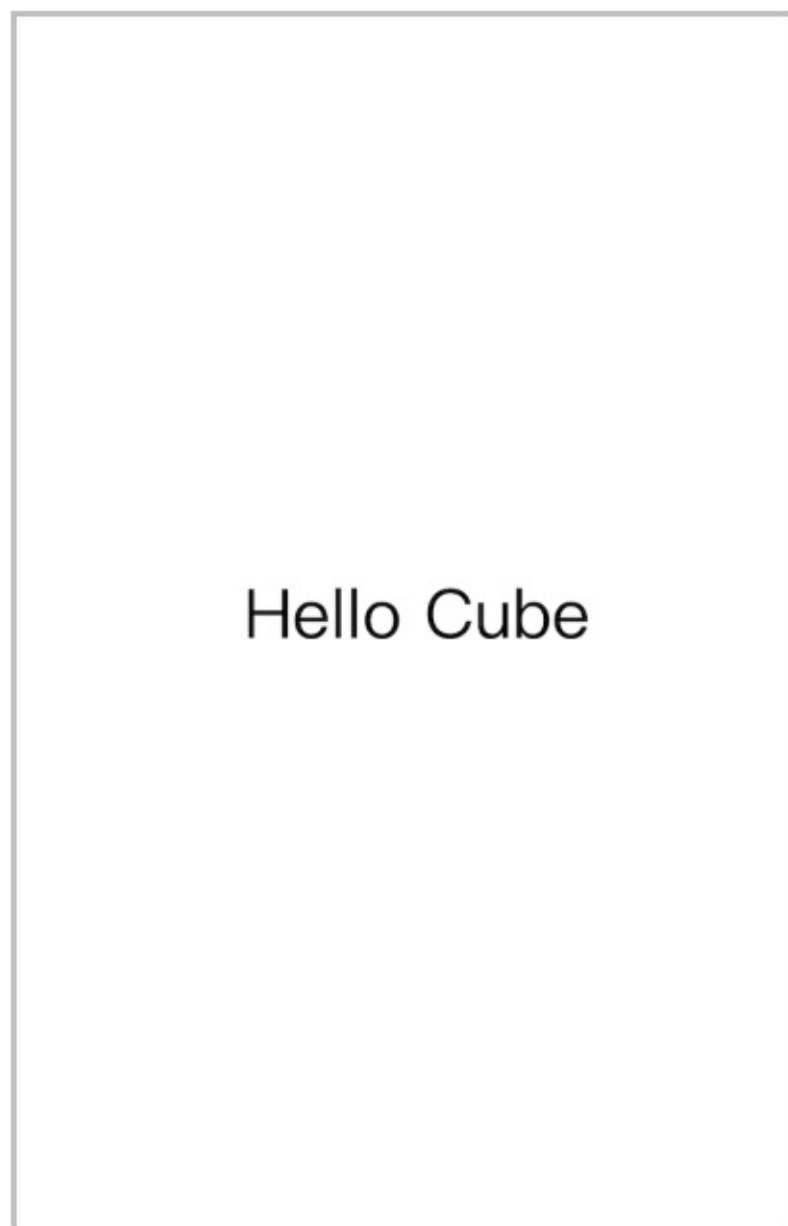
#出现上面 log 后, 构建成功, 构建产物为 miniapptools_dist/dist.tar
```

上传 dist.tar 到 ICE_EVB 的 `/resources/demo/pkg/2021001142699009` 目录下, 并在 ICE EVB 板 Linux 环境下解压 dist.tar

```
# 在 PC 上执行 scp 命令, 上传 dist.tar
scp miniapptools_dist/dist.tar root@192.168.1.100:/resources/demo/pkg/2021001142699009/.

# 在 ICE_EVB 上解压 dist.tar
cd /resources/demo/pkg/2021001142699009/
tar xf dist.tar

# 启动更新后的小程序
cube-test 2021001142699009 pages/index/index
```

1. 可以使用支付宝的小程序开发者工具进行小程序模拟、调试。小程序开发者工具是针对 Web 小程序，Cube 小程序为其子集，不支持有些样式，造成构建时语法错误和显示效果不同。

Xfce 桌面环境

Xfce 是一个轻量级的类 Unix 操作系统桌面环境，其设计目标是快速和轻便，并且同样悦目和实用。

Xfce 基于 GTK+ 2，类似 [Gnome 2](#)。Xfce 的一个首要目标之一是符合标准，特别是 freedesktop.org 制定的标准。这使得 Xfce 可以与为别的桌面环境写的程序更好兼容，只要该程序也遵循指定的标准。

Xfce 包括一系列组件，它们组合在一起，成为一个功能完整的桌面环境。这些组件各自打包，你可以根据个人喜好挑选适合自己的软件包。

如何安装 Xfce

安装 Xfce，运行以下命令：

```
apt install xfce4 dbus-x11
```

XFCE 带有很多实用的工具，可以用以下命令安装：

```
apt install xfce4-goodies
```

同时建议安装轻型的登录管理器 slim：

```
apt install slim
```

运行 Xfce

对于 **startx** 在 `~/.xinitrc` 里添加一行：

```
exec ck-launch-session startxfce4
```

安装其他桌面应用

```
# 安装浏览器
apt install -y epiphany-browser

# 安装软键盘
apt install -y florence
```

概述

开发者通过上述第一章完成了开发板的镜像烧写并体验到 Linux 的基础操作，在第二章中着重介绍了Linux 开发板上运行的一些场景化的应用。在本章介绍如何通过源代码，进行 ICE 芯片的定制化开发。主要介绍内容包括：

- 第一步：搭建开发环境
- 第二步：编译 u-boot、OpenSBI、Linux Kernel
- 第三步：制作根文件系统

通过本章的介绍，开发者可以掌握 Linux 系统的基础开发的能力。

搭建交叉开发环境

方法一：使用 **thead** 工具安装

通过 `thead` 命令下载并安装 RISC-V 工具链，如果未安装 `thead` 工具，可通过 `sudo pip install thead-tools` 安装，`thead` 命令安装成功后，可通过如下命令安装工具链：

```
thead toolchain -r
```

安装成功后，控制台显示信息如下：

```
Start to download toolchain: riscv64-linux
 100.00% [#####] Speed: 3.603MB/S
Start install, wait half a minute please.
Congratulations!
please run command:
  source /home/xxxxxx/.bashrc
```

`xxxxxx` 对应的是当前的用户名，安装成功后，将 `risc-v` 工具链路径加入到 `PATH`环境变量中：

```
export PATH=$HOME/.thead/riscv64-linux/bin:$PATH
```

方法二：通过 **tar** 包安装

```
wget "https://yoc.docs.t-head.cn/tools/riscv64-linux-x86_64-20201104.tar.bz2"

sudo mkdir -p /opt/riscv64-linux
sudo tar xf riscv64-linux-x86_64-20201104.tar.gz -C /opt/riscv64-linux

# 将 toolchain 加入到 PATH 环境变量中
export PATH=/opt/riscv64-linux/bin:$PATH
```


编译 u-boot

u-boot，全称 Universal Boot Loader，是遵循 GPL 条款的开放源码项目。u-boot 的作用是系统引导。u-boot 的工作模式有启动加载模式和下载模式。启动加载模式是 Bootloader 的正常工作模式，嵌入式产品发布时，Bootloader 必须工作在这种模式下，Bootloader 将嵌入式操作系统从 FLASH 中加载到 SDRAM 中运行，整个过程是自动的。下载模式就是 Bootloader 通过某些通信手段将内核映像或根文件系统映像等从 PC 机中下载到目标板的 FLASH 中。用户可以利用 Bootloader 提供的一些命令接口来完成自己想要的操作。

下载与编译

```
git clone "https://gitee.com/thead-linux/u-boot.git" -b master
make ice_evb_c910_defconfig
make -j ARCH=riscv CROSS_COMPILE=riscv64-unknown-linux-gnu-

# 配置文件保存在 configs/ 目录下，根据开发板选择相应的配置文件
```

编译过程如下：

```
.....
LD      u-boot
OBJCOPY u-boot.srec
OBJCOPY u-boot-nodtb.bin
SYM     u-boot.sym
COPY    u-boot.bin
CC      spl/lib/display_options.o
LD      spl/lib/built-in.o
LD      spl/u-boot-spl
OBJCOPY spl/u-boot-spl-nodtb.bin
COPY    spl/u-boot-spl.bin
CAT     u-boot-with-spl.bin
```

编译完成后，生成 `u-boot-with-spl.bin` 文件，接下来将 `u-boot-with-spl.bin` 烧写到开发板中。

烧写

方法一：使用 CCT 烧写，方法参见：[Linux环境下烧写镜像](#)

方法二：在开发板的 Linux 环境下直接烧写：

```
# 将 u-boot-with-spl.bin 传到开发板
scp u-boot-with-spl.bin root@192.168.1.100:~/
# 打开 boot 分区的写权限
ssh root@192.168.1.100 "echo 0 > /sys/block/mmcblk0boot0/force_ro"
# 将 u-boot-with-spl.bin 写入boot分区
ssh root@192.168.1.100 "dd if=u-boot-with-spl.bin of=/dev/mmcblk0boot0"
# 重启开发板
ssh root@192.168.1.100 "reboot"
```

u-boot命令使用

u-boot 提供众多命令，输入 help 或 ? 可以查看 u-boot 有哪些命令，本文介绍一些 u-boot 在日常工作中常用的一些命令：

```
C910 # help
?      - alias for 'help'
base   - print or set address offset
bdinfo - print Board Info structure
bmode  - sd1|sd2|qspi1|normal|usb|sata|ecspi1:0|ecspi1:1|ecspi1:2|ecspi1:3|esdhc1|esdhc2|esdhc3|esdhc4 [noreset]
bmp     - manipulate BMP image data
boot   - boot default, i.e., run 'bootcmd'
bootd  - boot default, i.e., run 'bootcmd'
```



```
bootelf - Boot from an ELF image in memory
bootm   - boot application image from memory
bootp   - boot image via network using BOOTP/TFTP protocol
bootvx  - Boot vxWorks from an ELF image
bootz   - boot Linux zImage image from memory
clocks  - display clocks
clrlogo - fill the boot logo area with black
cmp     - memory compare
...
```

信息查询相关命令

bdinfo

该命令用来查询当前板子的相关信息：

```
C910 # bdinfo
boot_params = 0x0000000000000000
DRAM bank   = 0x0000000000000000
-> start     = 0x0000000000000000
-> size      = 0x0000000010000000
relocaddr    = 0x00000000fff96000
reloc off    = 0x000000003ff96000
ethaddr      = 6a:af:e8:c6:23:ff
IP addr      = 192.168.1.100
baudrate     = 115200 bps
```

printenv

查询当前板子的一些相关环境变量：

```
C910 # print
arch=riscv
avail_addr=0x10000000
baudrate=115200
board=ice-c910
board_name=ice-c910
bootargs=console=ttyS0,115200 rdinit=/sbin/init rootwait rw earlyprintk root=PARTUUID=80a5a8e9-c744-491a-93c1-4f4194fd690b rootfstype=ext4 clk_ignore_unused loglevel=7 crashkernel=256M-:128M c910_mmu_v1 eth=6a:af:e8:c6:23:ff
bootcmd=ext4load mmc 0:2 0x0 fw_jump.bin; ext4load mmc 0:2 0x01f00000 hw.dtb; ext4load mmc 0:2 0x02000000 uImage; bootm 0x02000000 - 0x01f00000
bootdelay=2
cpu=c9xx
dtb_addr=0x01f00000
dtb_emmc_size=0x100
dtb_emmc_start=0x1000
ethact=ethernet@3fffc0000
ethaddr=6a:af:e8:c6:23:ff
fdtcontroladdr=fffe4630
ipaddr=192.168.1.100
kernel_addr=0x02000000
kernel_emmc_size=0xa000
kernel_emmc_start=0x2000
opensbi_addr=0x0
opensbi_emmc_size=0x280
opensbi_emmc_start=0x800
partitions=name=table,size=2031KB;name=boot,size=60MiB,type=bc13c2ff-59e6-4262-a352-b275fd6f7172;name=root,uuid=80a5a8e9-c744-491a-93c1-4f4194fd690b,size=-,type=linux
stderr=serial@3fff73000
stdin=serial@3fff73000
stdout=serial@3fff73000
uuid_rootfs=80a5a8e9-c744-491a-93c1-4f4194fd690b
vendor=thead

Environment size: 1072/131068 bytes
```

version

该命令用于查询uboot版本和交叉编译工具的相关信息

```
C910 # version
U-Boot 2020.01-g6cc5d59b0d (Dec 20 2020 - 08:37:37 +0000)

riscv64-unknown-linux-gnu-gcc (T-HEAD RISC-V Tools V1.10.2 B20201104) 8.4.0
GNU ld (GNU Binutils) 2.32
```

环境变量相关命令

环境变量常用的命令主要有两个，分别是setenv和saveenv，setenv命令用来设置或者修改当前环境变量的值，saveenv用来保存环境变量的值，一般环境变量是存放在外部的Flash中的，例如Nand Flash中，当uboot启动的时候，会将环境变量读取的DRAM中，当我们使用setenv修改了环境变量的值后，需要使用saveenv命令将修改后的环境变量的值保存到Flash中，否则修改无效。setenv命令的格式如下：

```
setenv env value
```

查看当前的bootdelay环境变量值为：

```
print cpu
```

当前的bootdelay为3，我们要将bootdelay环境变量的值设置为5，可以使用下面的命令：

```
setenv bootdelay 5
saveenv
```

清除环境变量：

```
# 恢复所有变更的默认值
env default -a
```

内存操作相关命令

内存操作的相关命令用于对DRAM内存进行读写操作，常用的内存操作命令有md、nm、mm、mw、cp、cmp。

md 命令

用于显示内存值，命令格式如下：

```
C910 # ? md
md - memory display

Usage:
md [.b, .w, .l, .q] address [# of objects]
```

命令用法中的[.b, .w, .l]对应着byte、word、long，分别以1个字节、2个字节、4个字节来进行内存值显示，address表示要查看的内存起始地址，[# of objects]表示要查看的数据长度，和显示的数据格式有关，并且需要注意的是uboot命令中的数字都是十六进制的。

例如，当我们想要查看从地址0x1000000开始的48个字节的内存值，可以使用下面的命令显示：

```
md.b 10000000 30
或者：
md.b 0x10000000 0x30
```

显示如下：

```
C910 # md.b 0x10000000 0x30
80000000: ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff  ....
```

```
80000010: ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff .....
80000020: ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff .....
```

nm 命令

用于修改指定地址的内存值，命令格式如下：

```
C910 # ? nm
nm - memory modify (constant address)

Usage:
nm [.b, .w, .l, .q] address
```

nm命令同样也是使用[b, .w, .l]来指定内存的操作格式，例如，想要使用修改0x10000000地址的数据为0x77，可使用下面的命令：

```
C910 # nm.b 10000000
10000000: ff ? 11
10000000: 11 ? q
```

再通过 md 0x10000000 查看数据是否更改：

```
C910 # md.b 0x10000000
10000000: 11 ff ff ff ff ff ff ff ff ff ff ff ff ff ff .....
10000010: ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff .....
10000020: ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff .....
```

mm命令

mm命令也是可以用来修改内存值，但是使用mm命令修改内存值的时候，地址值将会自增，使用nm命令时，地址值将不会自增，命令格式如下：

```
C910 # ? mm
mm - memory modify (auto-incrementing address)

Usage:
mm [.b, .w, .l, .q] address
```

例如，使用.b格式修改0x10000000开始的连续4个字节数据为0x12345678，使用命令如下：

```
C910 # mn.b 0x10000000
Unknown command 'mn.b' - try 'help'
C910 # mm.b 0x10000000
10000000: 11 ? 1
10000001: ff ? 2
10000002: ff ? 3
10000003: ff ? 4
10000004: ff ? 5
10000005: ff ? 6
10000006: ff ? q
C910 # md.b 0x10000000
10000000: 01 02 03 04 05 06 ff ff ff ff ff ff ff ff ff ff .....
10000010: ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff .....
10000020: ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff .....
```

mw命令

用来使用一个指定的数据填充一段内存，命令的使用方法如下：

```
C910 # ? mw
mw - memory write (fill)

Usage:
```



```
mw [.b, .w, .l, .q] address value [count]
```

该命令同样使用[.b, .w, .l]来指定操作格式，address表示要填充的内存起始地址，value表示要填充的数据，count是要填充的长度。例如，使用.b格式将以0x10000000为起始地址的0x30个内存块填充为0xAA，命令如下：

```
C910 # mw.b 10000000 aa 30
C910 # md.b 0x10000000
10000000: aa aa aa aa aa aa aa aa aa aa aa aa aa aa aa .....
10000010: aa aa aa aa aa aa aa aa aa aa aa aa aa aa aa .....
10000020: aa aa aa aa aa aa aa aa aa aa aa aa aa aa aa .....
```

cp命令

数据拷贝命令，用于将DRAM中的数据从一段内存中拷贝到另一段内存中，命令的使用格式如下所示：

```
C910 # ? cp
cp - memory copy

Usage:
cp [.b, .w, .l, .q] source target count
```

该命令同样使用[.b, .w, .l]来指定操作格式，source表示为内存源地址，target表示为目标地址，count为拷贝的长度。例如，使用.b格式将0x10000000开始地址处的0x30个字节拷贝到0x10000800地址处，命令如下所示：

```
cp.b 10000000 10000800 30
或者：
cp.b 0x10000000 0x10000800 30
```

cmp命令

用于比较两段内存的数据是否相等，命令的使用格式如下所示：

```
C910 # ? cmp
cmp - memory compare

Usage:
cmp [.b, .w, .l, .q] addr1 addr2 count
```

该命令同样使用[.b, .w, .l]来指定操作格式，addr1为第一段内存首地址，addr2为第二段内存首地址，count表示要比较的长度。

例如，使用.b格式来比较0x80000000和0x80000100两个地址的数据是否相等，比较的长度为0x14个字节，命令如下：

```
C910 # cmp.b 0x10000000 0x10000800 30
byte at 0x0000000010000000 (0xaa) != byte at 0x0000000010000800 (0xff)
Total of 0 byte(s) were the same
```

emmc和sd卡相关操作命令

对于uboot来说是支持emmc或者sd卡，因此也需要提供给用户emmc和sd卡的相关操作命令，uboot中常用于操作mmc设备的命令为"mmc"，mmc是一系列的命令，它的后面可以跟不同的参数，在uboot命令行中，输入下面的命令可以查看详细的用法：

```
C910 # ? mmc
mmc - MMC sub system

Usage:
mmc info - display info of the current MMC device
mmc read addr blk# cnt
mmc write addr blk# cnt
mmc erase blk# cnt
mmc rescan
mmc part - lists available partition on current mmc device
mmc dev [dev] [part] - show or set current mmc device [partition]
```



```
mmc list - lists available devices
mmc hwpartition [args...] - does hardware partitioning
arguments (sizes in 512-byte blocks):
[user [enh start cnt] [wrrel {on|off}]] - sets user data area attributes
[gp1|gp2|gp3|gp4 cnt [enh] [wrrel {on|off}]] - general purpose partition
[check|set|complete] - mode, complete set partitioning completed
WARNING: Partitioning is a write-once setting once it is set to complete.
Power cycling is required to initialize partitions after set to complete.
mmc setdsr <value> - set DSR register value
```

mmc后跟不同的参数可以实现不同的功能，如下：

命令	功能
mmc info	输出mmc设备的信息
mmc read	读取mmc设备中的数据
mmc write	向mmc设备中写入数据
mmc rescan	扫描当前存在的mmc设备
mmc part	列出mmc设备的分区
mmc dev	切换mmc设备
mmc list	列出当前的mmc设备
mmc hwpartition	设置mmc设备的分区
mmc bootbus	设置指定mmc设备的BOOT_BUS_WIDTH的值
mmc bootpart	设置指定mmc设备的boot和RPMB分区大小
mmc partconf	设置指定mmc设备的PARTITION_CONFIG的值
mmc rst	mmc设备复位
mmc setdsr	设置mmc设备DSR寄存器的值

mmc info

mmc info命令用来显示当前选中的mmc设备的信息：

```
C910 # mmc info
Device: mmc0@3ffffb0000
Manufacturer ID: 11
OEM: 100
Name: 004GA
Bus Speed: 25000000
Mode: MMC High Speed (52MHz)
Rd Block Len: 512
MMC version 5.0
High Capacity: Yes
Capacity: 3.7 GiB
Bus Width: 8-bit
Erase Group Size: 512 KiB
HC WP Group Size: 4 MiB
User Capacity: 3.7 GiB WRREL
Boot Capacity: 2 MiB ENH
RPMB Capacity: 512 KiB ENH
```

mmc rescan

mmc rescan命令用于扫描当前目标板上所有的mmc设备，包括eMMC和SD卡，命令输入如下：

```
C910 # mmc rescan
```

mmc list

mmc list命令可用于查看当前目标板共有多少个mmc设备，输入命令如下：

```
C910 # mmc list
mmc0@3ffffb0000: 0 (eMMC)
```

mmc dev

mmc dev命令可以用来切换当前的mmc设备，命令格式如下：

```
mmc dev [dev] [part]
```

其中[dev]表示要切换的mmc设备号，[part]是mmc设备的分区号，如果不写分区号，则默认为分区0。

例如，使用命令切换到eMMC设备：

```
#当0为SD卡，1为eMMC时
=> mmc dev 1
```

mmc part

mmc part命令可以用来查看当前mmc设备的分区，例如查看当前sd卡中的分区：

```
C910 # mmc part

Partition Map for MMC device 0  --  Partition Type: EFI

Part      Start LBA      End LBA      Name
Attributes
Type GUID
Partition GUID
1  0x00000022      0x00000fff      "table"
attrs:  0x0000000000000000
type:    ebd0a0a2-b9e5-4433-87c0-68b6b72699c7
type:    data
guid:    72fd13fc-2c69-42f4-9f02-d19dd22ecd0e
2  0x00001000      0x0001efff      "boot"
attrs:  0x0000000000000000
type:    bc13c2ff-59e6-4262-a352-b275fd6f7172
guid:    02382dbd-94d7-435f-ace0-598b4a5acb5e
3  0x0001f000      0x0075ffde      "root"
attrs:  0x0000000000000000
type:    0fc63daf-8483-4772-8e79-3d69d8477de4
type:    linux
guid:    80a5a8e9-c744-491a-93c1-4f4194fd690b
```

mmc read

mmc read命令可用于读取mmc设备的数据，它的使用格式如下所示：

```
mmc read addr blk# cnt
```

其中addr是将数据读取到DRAM中的地址，blk是要读取的块起始地址，一块为512字节，cnt则是要读取的块的数量。

例如，从当前的SD卡设备的第2048块开始，读取20个块数据到DRAM的0x10000100地址处，该命令如下：

```
C910 # mmc list
C910 # mmc dev 0
C910 # mmc read 10000100 800 14
```

读取成功后，如下所示：

```
MMC read: dev # 0, block # 2048, count 20 ... 20 blocks read: OK
```

mmc write

mmc write命令可以将DRAM中的数据写入到mmc设备里面，其命令的格式如下所示：

```
mmc write addr blk# cnt
```

其中addr是要写入到mmc设备中的数据在DRAM中的起始地址，blk是要写入mmc的块起始地址，cnt是要写入的块数量，一个块的大小为512字节。

例如，将DRAM地址0x10000100开始的数据，从mmc设备的2048个块开始烧写，烧写20个块，命令如下：

```
C910 # mmc read 10000100 800 14
```

写入成功后，如下所示：

```
MMC write: dev # 0, block # 8264, count 20 ... 20 blocks written: OK
```

mmc erase

mmc erase可以用来擦除mmc设备中指定的块，其使用命令格式如下：

```
mmc erase blk# cnt
```

其中blk是要擦除的起始块，cnt是要擦除的块数量。

例如，可以使用下面的命令擦除mmc设备从2048个块开始的20个块：

```
C910 # mmc list
C910 # mmc part
C910 # mmc erase 800 14
```

文件系统相关操作

当我们需要在uboot中对SD卡或者eMMC设备中存储的文件进行操作时，这个时候就需要用到uboot中文件系统的操作命令，u-boot 支持文件系统，包括 fat、ext2/ext3/ext4 等。对于fat 文件系统，u-boot 提供： fatinfo、fatls、fstype、fatload、fatwrite 等命令，对于 ext2/ext3/ext4 文件，u-boot 出提供相对的操作命令： ext2load ext2ls ext4load ext4ls ext4size 等。下文以 ext4 为参考，介绍 文件系统操作。

fstype 命令

fstype命令可以用于查看mmc设备中某个分区的文件系统格式，该命令的用法如下所示：

```
C910 # ? fstype
fstype - Look up a filesystem type

Usage:
fstype <interface> <dev>:<part>
- print filesystem type
fstype <interface> <dev>:<part> <varname>
- set environment variable to filesystem type
```

从上面的输出可以看到，fstyp命令具有两个用法，第一个用来查看mmc设备分区中的文件系统类型，第二个则是用来设置文件系统类型的环境变量，对于第一个命令用法，表示接口，例如mmc，:中dev则表示要查询的设备号，part则是设备的分区。

例如，查看我当前目标板中sd设备的第一个分区的文件系统类型，可以使用下面命令：

```
=> fstype mmc 0:2
```

命令输入后回车，输出如下所示：

```
C910 # fstype mmc 0:2
ext4
```

ext4ls 命令

ext4ls 命令可以用于查询FAT格式文件系统的目录和文件信息，该命令的用法如下：


```
C910 # ? ext4ls
ext4ls - list files in a directory (default /)

Usage:
ext4ls <interface> <dev[:part]> [directory]
    - list files from 'dev' on 'interface' in a 'directory'
```

在该命令的用法中，表示要查询的接口，[dev[:part]] 中dev表示要查询的设备号，part表示要查询分区，[directory]表示要查询的目录，如果该参数不输入的话，默认为根目录。

例如，查询我当前sd卡中分区2中的目录和文件，可以输入下面命令：

```
=> ext4ls mmc 0:2
```

命令输入回车后，将会列举根目录下所有的目录和文件，输出如下：

```
C910 # ext4ls mmc 0:3
<DIR>      1024 .
<DIR>      1024 ..
<DIR>     12288 lost+found
<DIR>      4096 bin
<DIR>      1024 boot
<DIR>      1024 dev
<DIR>      5120 etc
<DIR>      1024 home
<DIR>      1024 lib
<DIR>      1024 media
<DIR>      1024 mnt
<DIR>      1024 opt
<DIR>      1024 proc
<DIR>      1024 root
<DIR>      1024 run
<DIR>      4096/sbin
<DIR>      1024 srv
<DIR>      1024 sys
<DIR>      1024 tmp
<DIR>      1024 usr
<DIR>      1024 var
<DIR>      1024 resources
```

当我们再想查看 etc 目录下的目录或者文件时，可以使用下面的命令：

```
C910 # ext4ls mmc 0:3 /etc
```

ext4load 命令

ext4load 命令用来将指定的文件读取到DRAM内存中，该命令的使用格式如下：

```
C910 # ? ext4load
ext4load - load binary file from a Ext4 filesystem

Usage:
ext4load <interface> [<dev[:part]> [addr [filename [bytes [pos]]]]]
    - load binary file 'filename' from 'dev' on 'interface'
      to address 'addr' from ext4 filesystem
```

该命令的用法中，表示设备接口，例如mmc，[dev:[part]] 中的dev表示设备号，part表示mmc设备的分区，则是文件读取到DRAM中的起始地址，则是要读取的文件的名字，bytes表示要读取多少字节的数据，如果该值为0或者未使用，则表示将要读取整个文件，pos表示要读的文件相对于文件首地址的偏移，如果为0或者未使用，则表示从文件首地址开始读取。

例如，在我当前的目标帮中，将mmc卡中第二个分区中的 uImage 文件读取到DRAM中0x10000000起始地址中，可以使用下面的命令：

```
C910 # ext4ls mmc 0:2
C910 # ext4load mmc 0:2 0x10000000 uImage
```

命令输入后回车，输出如下：

```
14058560 bytes read in 610 ms (22 MiB/s)
```

BOOT启动相关操作命令

u-boot最主要的工作就是引导启动Linux系统，因此uboot中肯定是有相关的boot启动命令的，和boot启动常用相关的命令有boot、bootm和bootz，接下来，我们了解一下这些命令的使用。

boot 命令

boot命令是用来启动Linux系统的，该命令的用法如下：

```
C910 # ? boot
boot - boot default, i.e., run 'bootcmd'

Usage:
boot
```

可以看到到，该命令将会运行bootcmd，也就是boot命令将会读取bootcmd这个环境变量，并运行这个环境变量中的命令，来看一下当前我目标板中的bootcmd环境变量内容，使用下面命令查看：

```
C910 # print bootcmd
```

显示如下：

```
bootcmd=ext4load mmc 0:2 0x0 fw_jump.bin; ext4load mmc 0:2 0x01f00000 hw.dtb; ext4load mmc 0:2 0x02000000 uImage; bootm 0x02000000 - 0x01f00000
```

可以看到，该环境变量就是定义了一些启动引导的命令集合，先使用ext4load命令从eMMC 第个分区读取fw_jump.bin、hw.dtb、uImage 到DRAM 内存地址中，然后使用bootm命令启动Linux系统。

bootm 命令

bootm命令用于从内存中启动uImage镜像，该命令的用法如下：

```
C910 # ? bootm
bootm - boot application image from memory

Usage:
bootm [addr [arg ...]]
    - boot application image stored in memory
      passing arguments 'arg ...'; when booting a Linux kernel,
      'arg' can be the address of an initrd image
    When booting a Linux kernel which requires a flat device-tree
    a third argument is required which is the address of the
    device-tree blob. To boot that kernel without an initrd image,
    use a '-' for the second argument. If you do not pass a third
    a bd_info struct will be passed instead

For the new multi component uImage format (FIT) addresses
must be extended to include component or configuration unit name:
addr:<subimg_uname> - direct component image specification
addr#<conf_uname>  - configuration specification
Use iminfo command to get the list of existing component
images and configurations.

Sub-commands to do part of the bootm sequence.  The sub-commands must be
```



```
issued in the order below (it's ok to not issue all sub-commands):
start [addr [arg ...]]
loados - load OS image
ramdisk - relocate initrd, set env initrd_start/initrd_end
fdt      - relocate flat device tree
cmdline - OS specific command line processing/setup
bdt      - OS specific bd_t processing
prep     - OS specific prep before relocation or go
go       - start OS
```

主要环境变量介绍

环境变量	描述
bootdelay	执行自动启动的等候秒数
baudrate	串口控制台的波特率
netmask	以太网接口的掩码
ethaddr	以太网卡的网卡物理地址
bootfile	缺省的下载文件
bootargs	传递给内核的启动参数
bootcmd	自动启动时执行的命令
serverip	服务器端的ip地址
ipaddr	本地ip 地址

U-boot的环境变量值得注意的有两个： bootcmd 和bootargs。

- bootcmd：前面有说过bootcmd是自动启动时默认执行的一些命令，因此你可以在当前环境中定义各种不同配置，不同环境的参数设置，然后设置bootcmd为你经常使用的那种参数。

```
# 配置 bootcmd
setenv bootcmd "ext4load mmc 0:2 $opensbi_addr fw_jump.bin"
setenv bootcmd "$bootcmd;ext4load mmc 0:2 $opensbi_addr fw_jump.bin"
setenv bootcmd "$bootcmd;ext4load mmc 0:2 $dtb_addr hw.dtb"
setenv bootcmd "$bootcmd;ext4load mmc 0:2 $kernel_addr uImage"
setenv bootcmd "$bootcmd;bootm $kernel_addr - $dtb_addr"
```

- bootargs：bootargs是环境变量中的重中之重，甚至可以说整个环境变量都是围绕着bootargs来设置的。

```
# 配置内核启动参数 bootargs
setenv bootargs "console=ttyS0,115200"
setenv bootargs "$bootargs root=PARTUUID=$uuid_rootfs rootfstype=ext4"
setenv bootargs "$bootargs rdinit=/sbin/init rootwait rw earlyprintk"
setenv bootargs "$bootargs clk_ignore_unused loglevel=7 c910_mmu_v1"
setenv bootargs "$bootargs eth=$ethaddr"
```

bootargs的种类非常的多，主要参考网址：

- https://linux-sunxi.org/Kernel_arguments
- <https://linux-sunxi.org/U-Boot>

编译 OpenSBI

ISC-V Supervisor二进制接口（SBI）是以下之间的推荐接口：

- 在M模式下运行的特定于平台的固件，以S模式或HS模式执行的引导加载程序，管理程序或通用OS。
- 在HS模式下运行的管理程序，以及在VS模式下执行的引导加载程序或通用OS。

OpenSBI 项目的目标是为在M模式下执行的平台特定固件（上面提到的情况1）提供RISC-V SBI规范的开源参考实现。RISC-V 平台和片上系统供应商可以轻松扩展 OpenSBI 实现，以适应特定的硬件配置。

OpenSBI的主要组件以独立于平台的静态库libsbi.a的形式提供，实现了SBI接口。固件或引导加载程序实现可以链接到此库，以确保符合SBI接口规范。libsbi.a还定义了用于与平台固件实现提供的平台特定操作集成的接口（例如，控制台访问功能，处理器间中断控制等）。

下载与编译

```
git clone "https://gitee.com/thead-linux/opensbi.git" -b release
make -j PLATFORM=generic FW_PIC=y CROSS_COMPILE=riscv64-unknown-linux-gnu-
```

烧写

```
scp opensbi/build/platform/generic/firmware/fw_dynamic.bin \
    root@192.168.1.100:/boot/
ssh root@192.168.1.100 reboot
```

将 192.168.1.100 地址换成 EVB 开发板的网络IP 地址。

编译 Linux 内核

下载与编译

```
git clone "https://gitee.com/thead-linux/linux-5.10.4.git"
make ARCH=riscv CROSS_COMPILE=riscv64-unknown-linux-gnu- defconfig
make -j ARCH=riscv CROSS_COMPILE=riscv64-unknown-linux-gnu- LOCALVERSION=

mkimage -A riscv -O linux -T kernel -C none \
  -a 0x00200000 -e 0x00200000 -n Linux \
  -d arch/riscv/boot/Image uImage
```

烧写

```
scp uImage root@192.168.1.100:/boot/uImage
scp arch/riscv/boot/dts/thead/*.dtb root@192.168.1.100:/boot/
ssh root@192.168.1.100 reboot
```

将 192.168.1.100 地址换成 EVB 开发板的网络 IP 地址。

Buildroot 构建

Buildroot 用来构建根文件系统。

下载与编译

```
git clone https://gitee.com/thead-linux/buildroot
cd buildroot
make thead_ice_evb_c910_br_defconfig
git clone https://gitee.com/thead-linux/dl.git
make source
make
```

编译过程较长，请耐心等待。

查看编译结果

```
ls output/images
```

显示如下：

```
root@linux:~/works/ice/buildroot > ls output/images -l
total 26804
-rw-r--r--. 1 thead thead 62914560 Dec 24 12:41 rootfs.ext2
```

烧写

EVB 开发板进入用fastboot 模式(fastboot udp)，使用 fastboot 工具烧写， 详见：[Linux环境下烧写镜像](#)

```
# 烧写 root 分区
fastboot -s udp:192.168.1.100 flash root rootfs.ext2
```

启动镜像构建

制作 boot.ext4

方法1：使用 `make_ext4fs` 命令

如果未找到 `make_ext4fs`，请使用 `sudo pip install thead-tools` 安装，或者通过源码编译得到 `make_ext4fs`：

```
git clone https://github.com/superr/make_ext4fs.git
cd make_ext4fs
make
sudo cp make_ext4fs /usr/bin/
```

制作 boot.ext4 (需要先编译opensbi和linux kernel)：

```
mkdir boot
cp opensbi/build/platform/thead/c910/firmware/fw_jump.bin boot/.
cp kernel/uImage boot/.
cp kernel/arch/riscv/boot/dts/thead/ice-evb.dtb boot/hw.dtb

make_ext4fs -l 30M boot.ext4 boot
```

方法2：使用 `mkfs.ext4` 与 `mount`

该方法需要 `sudo` 权限

```
truncate -s 30M boot.ext4
mkfs.ext4 boot.ext4
mkdir boot
sudo mount boot.ext4 boot

cp opensbi/build/platform/thead/c910/firmware/fw_jump.bin boot/fw_jump.bin boot/.
cp kernel/uImage boot/hw.dtb boot/.
cp kernel/arch/riscv/boot/dts/thead/ice-evb.dtb boot/uImage boot/.

sudo umount boot.ext4
```


构建 Debian 根镜象

构建新的 debian 镜象，最好在已经安装好的 EVB 开发板的 Linux 环境下进行。

准备环境

```
apt update
apt install -y mmdebstrap
```

生成 rootfs目录

通过 mmdebstrap 命令制作 Debian 根文件系统：

```
# 生成 deiban rootfs
mkdir debian_rootfs

mmdebstrap --architectures=riscv64 sid debian_rootfs \
--keyring=/etc/apt/trusted.gpg.d \
--include="debian-ports-archive-keyring,net-tools,wget,openssh-server,haveged" \
--dpkgopt='path-exclude=/usr/share/man/*' \
--dpkgopt='path-include=/usr/share/man/man[1-9]/*' \
--dpkgopt='path-exclude=/usr/share/locale/*' \
--dpkgopt='path-include=/usr/share/locale/locale.alias' \
--dpkgopt='path-exclude=/usr/share/doc/*' \
--dpkgopt='path-include=/usr/share/doc/*/copyright' \
--dpkgopt='path-include=/usr/share/doc/*/changelog.Debian.*' \
--dpkgopt='path-exclude=/usr/share/{doc,info,man,omf,help,gnome/help}/*' \
"deb http://mirrors.aliyun.com/debian-ports sid main" \
"deb http://mirrors.aliyun.com/debian-ports unreleased main" \
"deb http://mirrors.aliyun.com/thead/debian-riscv64 sid main"

# 添加 thead riscv 仓库
wget "http://mirrors.aliyun.com/thead/thead-keys/thead-repo.gpg" \
-O debian_rootfs/etc/apt/trusted.gpg.d/thead-repo.gpg

echo "" >> debian_rootfs/etc/apt/sources.list
echo "deb http://mirrors.aliyun.com/thead/debian-riscv64 sid main" \
>> debian_rootfs/etc/apt/sources.list

# 使能 root ssh 登录
echo "PermitRootLogin yes" >> debian_rootfs/etc/ssh/sshd_config

# 删除 root 登录密码
sed -i 's/^root:x:/root::/g' debian_rootfs/etc/passwd

# 配置基础环境
echo "export LANG='en_US.UTF-8'" > debian_rootfs/etc/profile.d/base-set-env.sh
echo "export TERM=linux" >> debian_rootfs/etc/profile.d/base-set-env.sh
echo "alias ls='ls --color=auto'" >> debian_rootfs/etc/profile.d/base-set-env.sh
echo "alias ll='ls -l'" >> debian_rootfs/etc/profile.d/base-set-env.sh

# 查看文件夹大小
du -sm debian_rootfs

# 创建镜像文件
truncate -s 160M debian-rootfs.ext4
mkfs.ext4 debian-rootfs.ext4
mkdir tmp_rootfs
mount debian-rootfs.ext4 tmp_rootfs

mv debian_rootfs/* tmp_rootfs/
```

```
# umount
umount tmp_rootfs
```

烧写

方法一：使用 CCT 烧写，方法参见：[Linux环境下烧写镜像](#)

方法二：将 debian-rootfs.ext4 传输到 evb 开发板上，`dd if=debian-rootfs.ext4 of=/dev/mmcblk0p3`

概述

本章开始介绍在 ICE-EVB 开发板上进行软件开发，开发 Linux 应用程序有两种方式，第一次是直接在 ICE-EVB 开发板上建立开发环境，直接在 ICE-EVB 的Linux 环境下进行软件开发。另一种是交叉开发环境，即在PC电脑上搭建 RISC-V 的应用开发环境，在PC机上完成应用开发，再传开发出来的程序上传到ICE-EVB 的Linux 下。

- [安装开发环境](#)
- [搭建NFS环境](#)
- [GPIO 编程](#)
- [I2C 编程](#)
- [使用vector核](#)

安装开发环境

开发板 Native 开发环境

在 ICE-EVB 的开发板上安装开发环境：

```
apt update
apt install -y build-essential autoconf automake gdb gdbserver
```

示例：编写 helloworld 程序

- 1. 编程示例: helloworld.c `vi helloworld.c`

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char **argv)
{
    printf("Hello world!\n");
    return EXIT_SUCCESS;
}
```

- 2. 编译、执行：

```
# 编译
gcc helloworld.c -o helloworld

# 执行
./helloworld
Hello world!
```

交叉开发环境

方法一：使用 thead 工具安装

通过 thead 命令下载并安装 RISC-V 工具链，如果未安装 thead 工具，可通过 `sudo pip install thead-tools` 安装，thead 命令安装成功后，可通过如下命令安装工具链：

```
thead toolchain -r
```

安装成功后，控制台显示信息如下：

```
Start to download toolchain: riscv64-linux
 100.00% [#####] Speed: 3.603MB/S
Start install, wait half a minute please.
Congratulations!
please run command:
  source /home/xxxxxx/.bashrc
```

xxxxxx 对应的是当前的用户名，安装成功后，将 risc-v 工具链路径加入到 PATH环境变量中：

```
export PATH=$HOME/.thead/riscv64-linux/bin:$PATH
```

方法二：通过 tar 包安装

```
wget "https://yoc.docs.t-head.cn/tools/riscv64-linux-x86_64-20201104.tar.bz2"
```

```
sudo mkdir -p /opt/riscv64-linux
sudo tar xf riscv64-linux-x86_64-20201104.tar.gz -C /opt/riscv64-linux

# 将 toolchain 加入到 PATH 环境变量中
export PATH=/opt/riscv64-linux/bin:$PATH
```

交叉开发示例：编写 helloworld 程序

1. 在 PC 机编写代码: `vim helloworld.c`

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char **argv)
{
    printf("Hello world!\n");
    return EXIT_SUCCESS;
}
```

2. 编译与安装

```
# 在PC 机上完成编译
riscv64-linux-gnu-gcc helloworld.c -o helloworld
# 将程序安装到 ICE-DVB 开发板
scp helloworld root@192.168.1.100:~/
```

3. 在 ICE-EVB 上运行 helloworld 程序:

```
ssh root@192.168.1.100
./helloworld
Hello world!
```


搭建 NFS 环境

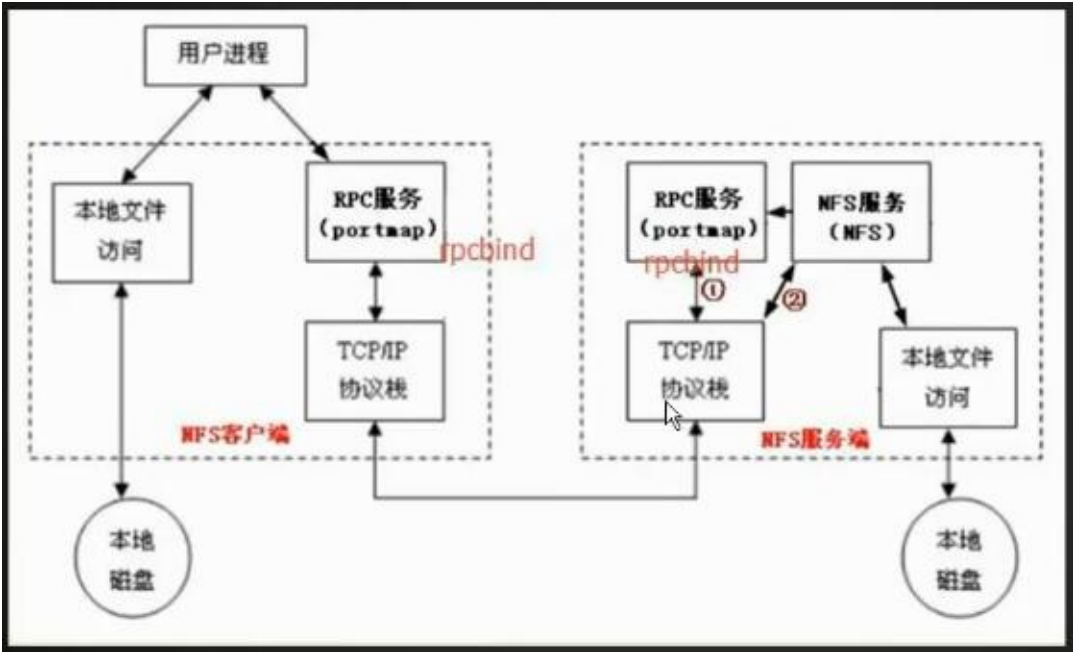
NFS 介绍

NFS(Network File System) 即网络文件系统，是FreeBSD支持的文件系统中的一种，它允许网络中的计算机之间通过TCP/IP网络共享资源。在NFS的应用中，本地NFS的客户端应用可以透明地读写位于远端NFS服务器上的文件，就像访问本地文件一样。NFS的数据传输基于RPC（remote procedure call）协议。

NFS 应用场景是：A，B，C 三台机器上需要保证被访问到的文件是一样的。A共享数据出来，B和C分别去挂载A共享的数据目录，从而B和C访问到的数据和A上的一致。

例子：跑了一个网站，上面传输了很多图片，用户访问一个图片时，需要从A机器上去请求，但A机器负载高，为了分担负载，就多弄了两台机器，B机器C机器同时提供服务；正常的话，需要到A机器上才能拿到数据，但是B机器和C机器做了负载均衡，分担了相同的服务器，那么用户也有可能到B机器或者C机器上；那么用户请求到B机器上的时候，如何才能获取到A机器上的数据呢；要么把A机器的数据传输到B机器上，同时传输到C机器上，但是这个不能时时更新，（用户上传的数据是存放在A机器上，但用户请求的时候数据是请求到B机器上）这样A上的数据还没到B上面去，就会导致用户请求获取的数据访问不到，访问为空，为404；那么NFS服务就可以解决这个问题，将A机器的数据共享到B机器、C机器，通过NFS来实现。有NFS服务以后，上传到A机器上的数据，B机器或C机器上就能马上看到和调用。NFS可以实时同步数据。

NFS原理图：



NFS 服务端安装配置

服务端配置

服务端IP：192.168.1.101

- 安装NFS工具：

```
sudo apt install -y nfs-kernel-server
```

- 配置NFS：

安装NFS服务后，会新增一个 `/etc/exports` 文件，NFS服务根据它的配置来运行，其默认内容可通过命令 `cat /etc/exports` 查看，它默认包含了一些配置的范例，内容如所示。

```
# /etc/exports: the access control list for filesystems which may be exported
# to NFS clients.
# See exports(5).
#
# Example for NFSv2 and NFSv3:
# /srv/homes hostname1(rw,sync,no_subtree_check) hostname2(ro,sync,no_subtree_check)
#
# Example for NFSv4:
# /srv/nfs4 gss/krb5i(rw,sync,fsid=0,crossmnt,no_subtree_check)
# /srv/nfs4/homes gss/krb5i(rw,sync,no_subtree_check)
```

编辑 `/etc/exports` 文件，加入下面内容：

```
[root@localhost ~]# sudo vim /etc/exports
/home/nfstestdir 192.168.1.0/24(rw,sync,all_squash,anonuid=1000,anongid=1000,no_subtree_check)
```

- **/home/nfstestdir**：要共享的开发主机目录，注意使用空格与后面的配置隔开。
- **192.168.1.0/24** 配置谁可以访问，其中的/24是掩码，即掩码是255.255.255.0。
- **rw**: 表示客户机的权限，rw表示可读写，具体的权限还受文件系统的rwx及用户身份影响。
- **sync**：资料同步写入到内存与硬盘中。
- **anonuid=1000**：将客户机上的用户映射成指定的本地用户ID的用户，请根据具体的主机用户组gid进行配置。
- **anongid=1000**：将客户机上的用户映射成属于指定的本地用户 组ID，请根据具体的主机用户组gid进行配置。
- **no_subtree_check**：不检查子目录权限，默认配置。

• 更新**exports**配置:

修改完/etc/exports文件并保存后，可使用exportfs命令更新配置：

```
sudo exportfs -arv
```

• 启动NFS服务：

安装完成后，系统会自动启动rpcbind服务（在服务端进程名为systemd），默认监听的端口为111端口：

```
[root@localhost ~]# ps aux | grep rpc
rpc 2390 0.0 0.0 64964 1044 ? Ss 21:19 0:00 /sbin/rpcbind -w
root 3826 0.0 0.0 112680 972 pts/0 R+ 21:31 0:00 grep --color=auto rpcbind
[root@localhost ~]# netstat -lnpt
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address Foreign Address State PID/Program name
tcp 0 0 0.0.0.0:111 0.0.0.0: *LISTEN 1/systemd
.....
tcp6 0 0 :::111 :::* LISTEN 1/systemd
.....
```

启动NFS服务命令：

```
sudo systemctl start nfs
```

将NFS服务加入开机启动项命令：

```
sudo systemctl enable nfs
```

客户端配置

在 ICE-EVB 开发板上安装 NFS 客户端，安装命令如下:

```
apt install nfs-common -y
```

• 客户端挂载:

检查客户端是否有权限访问服务端文件：

```
showmount -e 192.168.1.101
```

显示信息如下：

```
[root@localhost ~]# showmount -e 192.168.1.101
Export list for 192.168.1.101:
/home/nfstestdir 192.168.1.0/24
```

客户端已有限访问服务端

开始挂载：

```
mount -t nfs 192.168.1.101:/home/nfstestdir /mnt/
```

```
df -h

.....
192.168.1.101:/home/nfstestdir 16G 5.2G 11G 33% /mnt
```

- 测试:

在客户端/mnt目录下创建test文件：

```
[root@localhost ~]# touch /mnt/test
[root@localhost ~]# ls -l /mnt/
总用量 0
-rw-r--r--. 1 mysql mysql 0 1月 16 22:05 test
```

查看服务端/home/nfstestdir/目录：

```
[root@localhost ~]# ls -l /home/nfstestdir/
总用量 0
-rw-r--r--. 1 mysql mysql 0 1月 16 22:05 test
// 存在个客户端上一样的文件，实现了同步共享。
```

- 取消挂载：

```
sudo umount /mnt
```


Linux 下的 GPIO 编程

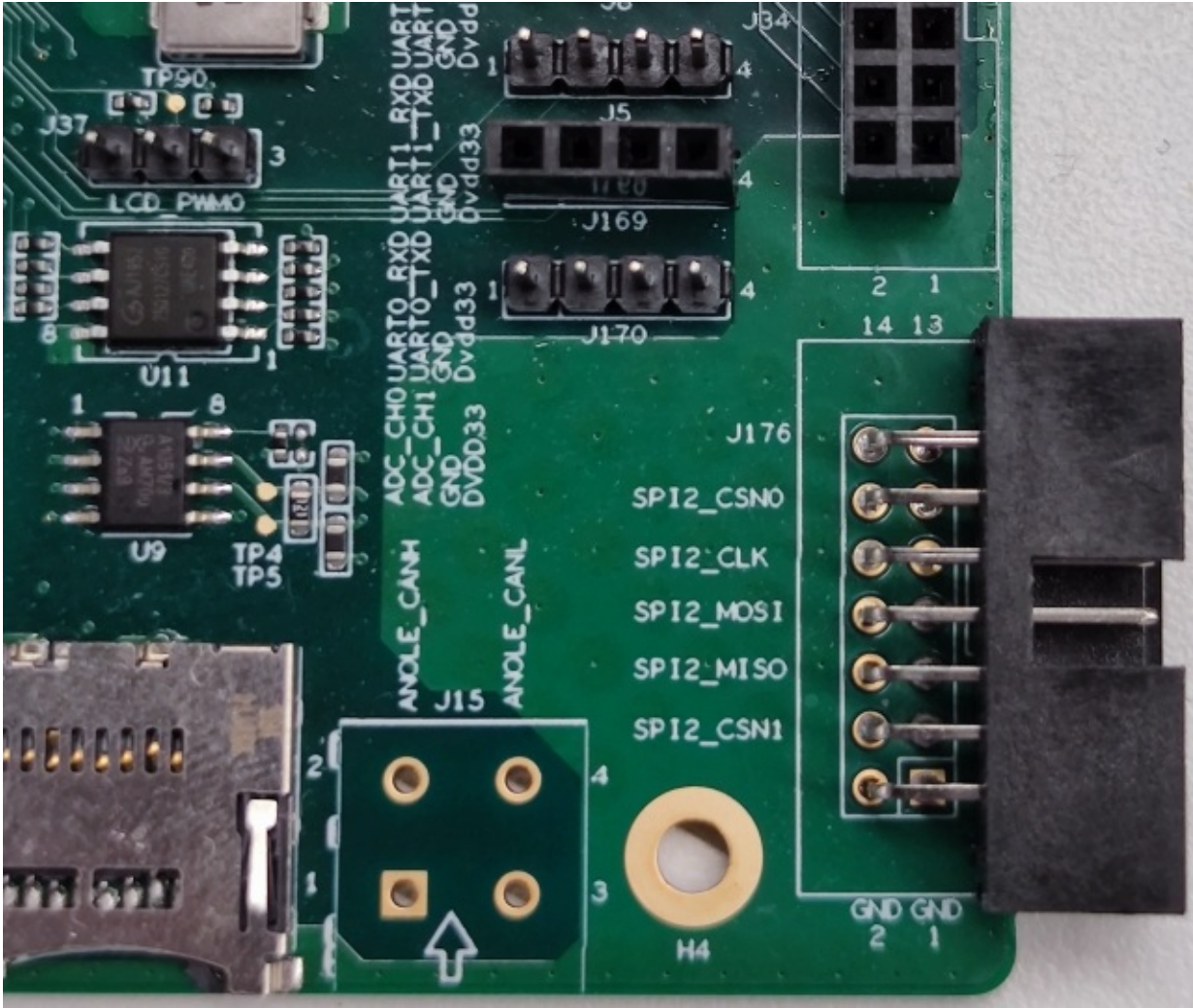
GPIO(General-purpose input/output) 通用型之输入输出的简称。在嵌入式系统中，经常需要控制许多结构简单的外部设备或者电路，这些设备有的需要通过 CPU 控制，有的需要 CPU 提供输入信号。并且，许多设备或电路只要求有开/关两种状态就够了，比如 LED 的亮与灭。对这些设备的控制，使用传统的串口或者并口就显得比较复杂，所以，在嵌入式微处理器上通常提供了一种“通用可编程 I/O 端口”，也就是 GPIO。

Linux 下的 GPIO 设备

配置 Pin MUX

通过配置寄存器可以定义 I/O 管脚功能，在使用 GPIO 功能时，首先要将相应的引脚配置为 GPIO，管脚寄存器和配置功能参考《PAD config registers》手册。

- 下面以 J176 上的 SPI2_CSN0，SPI2_CLK，SPI2_MOSI，SPI2_MISO，SPI2_CSN1 做简单说明。



- 引脚功能定义：MUX_CFG_002 (address：0x3fe830334)

[21:20]	SPI2_CLK_MUX_CFG	R/W	0x0	MUX config for pad: SPI2_CLK 00: SPI2_CLK 01: GPIO0[34] 10: SPI_SLV_SCLK
[23:22]	SPI2_CSN0_MUX_CFG	R/W	0x0	MUX config for pad: SPI2_CSN0 00: SPI2_CSN0 01: GPIO0[35] 10: SPI_SLV_CSN
[25:24]	SPI2_CSN1_MUX_CFG	R/W	0x0	MUX config for pad: SPI2_CSN1 00: SPI2_CSN1 01: GPIO0[36]
[27:26]	SPI2_MOSI_MUX_CFG	R/W	0x0	MUX config for pad: SPI2_MOSI 00: SPI2_MOSI 01: GPIO0[37] 10: SPI_SLV_MOSI
[29:28]	SPI2_MISO_MUX_CFG	R/W	0x0	MUX config for pad: SPI2_MISO 00: SPI2_MISO 01: GPIO0[38] 10: SPI_SLV_MISO
[31:30]	I2S_MCLK_MUX_CFG	R/W	0x0	MUX config for pad: I2S_MCLK 00: I2S_MCLK 01: GPIO0[39]

- 设置 SPI2_CLK 为GPIO0_34

```
# 读取 0x3fe830334 数值
root@thead-910:~# memtool md -l 0x3fe830334
3fe830334: 00005014 00000000 00400000 00000000 .P.....@.....
3fe830344: 00000000 00000000 00000000 00000000 .....

# 将 bit21:20 置为 '01', 写回寄存器
root@thead-910:~# memtool mw -l 0x3fe830334 0x105014
```

注：如果提示没有 memtool 命令，使用 `apt install -y memtool` 安装。

GPIO 设备

通过 sysfs 方式控制 GPIO，先访问 `/sys/class/gpio` 目录，向 export 文件写入 GPIO 编号，使得该 GPIO 的操作接口从内核空间暴露到用户空间，GPIO 的操作接口包括 direction 和 value 等，direction 控制 GPIO 方向，而 value 可控制 GPIO 输出或获得 GPIO 输入。文件 IO 方式操作 GPIO，使用到了4个函数 open、close、read、write。通过：`ls -l /sys/class/gpio` 可能以查看 GPIO的信息：

```
root@thead-910:~# ls -l /sys/class/gpio
total 0
--w----- 1 root root 4096 Jan  6 13:33 export
lrwxrwxrwx 1 root root  0 Dec 26 20:33 gpiochip352 -> ../../devices/platform/soc/3fff72000.gpio/gpio/gpiochip352
lrwxrwxrwx 1 root root  0 Dec 26 20:33 gpiochip384 -> ../../devices/platform/soc/3fff72000.gpio/gpio/gpiochip384
lrwxrwxrwx 1 root root  0 Dec 26 20:33 gpiochip416 -> ../../devices/platform/soc/3fff72000.gpio/gpio/gpiochip416
lrwxrwxrwx 1 root root  0 Dec 26 20:33 gpiochip448 -> ../../devices/platform/soc/3fff71000.gpio/gpio/gpiochip448
lrwxrwxrwx 1 root root  0 Dec 26 20:33 gpiochip480 -> ../../devices/platform/soc/3fff71000.gpio/gpio/gpiochip480
--w----- 1 root root 4096 Dec 26 20:33 unexport
```

文件说明：

- 1. gpio_operation 通过 /sys/ 文件接口操作 IO 端口 GPIO 到文件系统的映射
- 2. 控制 GPIO 的目录位于 /sys/class/gpio
- 3. /sys/class/gpio/export 文件用于通知系统需要导出控制的 GPIO 引脚编号
- 4. /sys/class/gpio/unexport 用于通知系统取消导出
- 5. /sys/class/gpio/gpiochipXXX 目录保存系统中 GPIO 寄存器的信息，包括每个寄存器控制引脚的起始编号 base，寄存器名称，引脚总数

dts 中的定义分别为：

```
gpio0: gpio@3fff71000 {
    compatible = "snps,dw-apb-gpio";
    reg = <0x3 0xfff71000 0x0 0x1000>;
    #address-cells = <1>;
    #size-cells = <0>;

    /* GPIO0[0-31] */
    gpio0_porta: gpio0-controller@0 {
        compatible = "snps,dw-apb-gpio-port";
        gpio-controller;
        #gpio-cells = <2>;
        snps,nr-gpios = <32>;
        reg = <0>;

        interrupt-controller;
        #interrupt-cells = <2>;
        interrupt-parent = <&intc>;
        interrupts = <27>;
    };

    /* GPIO0[32-63] */
    gpio0_portb: gpio0-controller@1 {
        compatible = "snps,dw-apb-gpio-port";
        gpio-controller;
        #gpio-cells = <2>;
        snps,nr-gpios = <32>;
        reg = <1>;
    };
};

gpio1: gpio@3fff72000 {
    compatible = "snps,dw-apb-gpio";
    reg = <0x3 0xfff72000 0x0 0x1000>;
    #address-cells = <1>;
    #size-cells = <0>;

    /* GPIO1[0-31] */
```



```
gpio1_porta: gpio1-controller@0 {
    compatible = "snps,dw-apb-gpio-port";
    gpio-controller;
    #gpio-cells = <2>;
    snps,nr-gpios = <32>;
    reg = <0>;
};

/* GPIO1[32-63] */
gpio1_portb: gpio1-controller@1 {
    compatible = "snps,dw-apb-gpio-port";
    gpio-controller;
    #gpio-cells = <2>;
    snps,nr-gpios = <32>;
    reg = <1>;
};

/* GPIO1[64-95] */
gpio1_portc: gpio1-controller@2 {
    compatible = "snps,dw-apb-gpio-port";
    gpio-controller;
    #gpio-cells = <2>;
    snps,nr-gpios = <32>;
    reg = <2>;
};
};
```

gpiochipxxx 与 gpio 的对应关系：

```
gpiochip352 : gpio1_portc , GPIO1[64-95]  分别对应为  352 ~ 383 号
gpiochip384 : gpio1_portb , GPIO1[32-63]  分别对应为  384 ~ 415 号
gpiochip416 : gpio1_porta , GPIO1[ 0-31]  分别对应为  416 ~ 447 号
gpiochip448 : gpio0_portb , GPIO0[32-63]  分别对应为  448 ~ 479 号
gpiochip480 : gpio0_porta , GPIO0[ 0-31]  分别对应为  480 ~ 511 号
```

GPIO 导出

需要对一个GPIO 控制，首先需要对 GPIO 进行导出，操作步骤如下：

第一步：确定 GPIO 引脚编号

```
引脚编号 = 控制引脚的寄存器基数 + 控制引脚寄存器位数
```

例如：（具体 GPIO 需要参考数据手册），如果使想用 SPI2_CLK(GPIO0_34)，那么引脚编号就可能等于 448 + (34 - 32) = 450

第二步：导出引脚

向 /sys/class/gpio/export 写入此编号，比如 450 号引脚，在 shell 中可以通过以下命令实现：

```
# 导出 450号 GPIO
echo 450 > /sys/class/gpio/export

# 查看导出是否成功
ls /sys/class/gpio/gpio450
```

命令成功后生成 /sys/class/gpio/gpio450 目录，如果没有出现相应的目录，说明此引脚不可导出。

第三步：设置引导方向

通过 GPIO 的 direction文件，修改 GPIO 引脚的方向：

```
echo out > /sys/class/gpio/gpio450/direction
```

direction 接受的参数可以是：in、out、high、low。其中参数 high / low 在设置方向为输出的同时，将 value 设置为相应的 1 / 0。

第四步：设置或者查看 GPIO电平

GPIO 的 value 文件是端口的数值，为1或0：

```
# 设置 GPIO 电平
echo 1 > /sys/class/gpio/gpio450/value

# 查看 GPIO 电平
cat /sys/class/gpio/gpio450/value
```

第五步：取消GPIO 导出

```
echo 450 > /sys/class/gpio/unexport
```

编程示例

```
#include <stdlib.h>
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>    //define O_WRONLY and O_RDONLY

static int write_file(const char *filename, const char *value)
{
    int fd = open(filename, O_WRONLY);
    if(fd != 0) {
        write(fd, value, strlen(value));
        close(fd);
        return 0;
    }

    return EXIT_FAILURE;
}

// 打开 GPIO
static int gpio_open(char *id)
{
    return write_file("/sys/class/gpio/export", id);
}

// 设置 GPIO 方向
static int gpio_direction(char *id, char *dir)
{
    char filename[128];
    sprintf(filename, "/sys/class/gpio/gpio%s/direction", id);

    return write_file(filename, dir);
}

// 设置 GPIO 电平
static int gpio_set(char *id, char *dir)
{
    char filename[128];
    sprintf(filename, "/sys/class/gpio/gpio%s/value", id);

    return write_file(filename, dir);
}

#define GPIO_PIN_450    "450"
int main(int argc, char **argv)
{
    if (gpio_open(GPIO_PIN_450) == 0) {
```

```
    gpio_direction(GPIO_PIN_450, "OUT");

    while(1) {
        gpio_set(GPIO_PIN_450, "1");
        usleep(1000000);
        gpio_set(GPIO_PIN_450, "0");
        usleep(1000000);
    }
}

return 0;
}
```

可以使用SHELL 脚本对GPIO的控制：

```
#!/bin/bash

echo 450 > /sys/class/gpio/export
echo out > /sys/class/gpio/gpio450/direction
while :
do
    echo 1 > /sys/class/gpio/gpio450/value
    sleep 1
    echo 0 > /sys/class/gpio/gpio450/value
    sleep 1
done
```

注：如果没有 `sleep` 命令，使用 `apt install -y coreutils` 安装。

Linux 下的 I2C 编程

I2C 简介

I2C (Inter Integrated Circuit) 总线是 PHILIPS 公司开发的一种半双工、双向二线制同步串行总线。I2C 总线包含两根信号线，双向数据线 SDA，时钟线 SCL)。

它的物理层有如下特点：

- 它是一个支持多设备的总线。“总线”指多个设备共用的信号线。 在一个 I2C 通讯总线中，可连接多个 I2C 通讯设备，支持多个通讯主机及多个通讯从机。
- 一个 I2C 总线只使用两条总线线路，一条双向串行数据线 (SDA)，一条串行时钟线 (SCL)。 数据线即用来表示数据，时钟线用于数据收发同步。
- 每个连接到总线的设备都有一个独立的设备地址，主机可以利用这个地址进行不同设备之间的访问。 其中地址是一个七位或十位的数字。
- 总线通过上拉电阻接到电源。当 I2C 设备空闲时，会输出高阻态，而当所有设备都空闲， 都输出高阻态时，由上拉电阻把总线拉成高电平。
- 多个主机同时使用总线时，为了防止数据冲突，会利用仲裁方式决定由哪个设备占用总线。
- 具有三种传输模式：标准模式传输速率为 100kbit/s，快速模式为 400kbit/s，高速模式下可达 3.4Mbit/s，但目前大多 I2C 设备尚不支持高速模式。

i2c-tools

Linux 系统提供 i2c-tools 工具来对系统中的 I2C 总线进行调试，下面重要介绍 i2c-tools 的使用方法。

安装 i2c-tools

在开发板的 Linux 命令环境中通过下面命令安装：

```
apt install i2c-tools
```

i2c-tools 提供 i2cdetect、i2cdump、i2ctransfer、i2cset、i2cget 命令，来操作 I2C。

i2cdetect

- 显示所有可用的I2C总线： i2cdetect -l

i2c-0	i2c	Synopsys DesignWare I2C adapter	I2C adapter
-------	-----	---------------------------------	-------------

- I2C设备查询: i2cdetect -y 0

```
0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00:                -- -- -- -- -- -- -- -- -- --
10: -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
20: -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
30: -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
40: -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
50: 50 -- -- -- -- -- -- -- 57 -- -- -- -- -- --
60: -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
70: -- -- -- -- -- -- -- -- --
```

i2ctransfer

- 写： i2ctransfer -f -y 0 w3@0x36 0x50 0x81 0x01
- 读： i2ctransfer -f -y 0 w2@0x36 0x30 0x0A r3

i2cdump

通过 i2cdump 指令可导出 I2C 设备中的所有寄存器内容，例如输入 i2cdump -y 0 0x50，可获得以下内容:

```
No size specified (using byte-data access)
0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f    0123456789abcdef
00: 01 00 0d 00 84 08 e8 03 04 4a 00 00 00 00 00 00    ?.?.?????J.....
10: 00 00 35 02 36 39 39 2d 38 32 31 38 30 2d 31 30    ..5?699-82180-10
20: 30 30 2d 34 31 30 20 4a 2e 30 ff ff ff ff ff ff    00-410 J.0.....
30: ff ff 35 2d 66 4b 04 00 36 2d 66 4b 04 00 00 00    ..5-fK?.6-fK?...
40: 00 00 00 00 37 2d 66 4b 04 00 30 33 32 33 32 31    ....7-fK?.032321
50: 36 31 33 30 35 36 35 ff ff ff ff ff ff ff ff ff    6130565.....
60: ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff    .....
70: ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff    .....
80: ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff    .....
90: ff ff ff ff ff ff 4e 56 43 42 1c 00 4d 31 00 00    .....NVCB?.M1..
a0: 35 2d 66 4b 04 00 36 2d 66 4b 04 00 37 2d 66 4b    5-fK?.6-fK?.7-fK
b0: 04 00 ff ff ff ff ff ff ff ff ff ff ff ff ff ff    ?......
c0: ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff    .....
d0: ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff    .....
e0: ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff    .....
f0: ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff 96    .....?
```

i2cdump -y 0 0x50指令中，

- y 代表取消用户交互过程，直接执行指令；
- 0 代表I2C总线编号；
- 0x50 代表I2C设备从机地址，此处选择配置芯片的高256字节内容。

使用玄铁 910-VECTOR 核

ICE 芯片内部有三个玄铁910处理器，其中 CPU0与 CPU1为同构处理器，不支持 VECTOR 指令，CPU2支持VECTOR 扩展指令集。在 ICE 芯片中使用 VECTOR 核时，可以通过 U-Boot 配置来实现两个环境的切换，具体操作如下：

切换到 **Vector** 核：

```
setenv boot_vector 1
saveenv
boot
```

切换到非 **VECTOR** 核：

```
setenv boot_vector 0
saveenv
boot
```

启动后, 查看 **cpuinfo**：isa已带v

```
root@thead-910:~# cat /proc/cpuinfo
processor       : 0
hart          : 2
isa           : rv64imafdcvsu
mmu           : sv39
model name    : T-HEAD C910
freq          : 1.2GHz
icache        : 64kB
dcache        : 64kB
l2cache       : 2MB
tlb           : 1024 4-ways
cache line    : 64Bytes
address sizes  : 40 bits physical, 39 bits virtual
vector version : 0.7.1
```

编译 **Vector** 程序

testvector.c:

```
#include<stdio.h>
#include<stdlib.h>
#include<math.h>
#include<riscv-vector.h>

#define vlen 128
#define sew 16
#define vmul 1
#define ISUNSIGN 0
#define VCTYPE "int16"

int main(int argc, char **argv)
{
    int vlmax = vmul*vlen/sew;
    int N=5;
    int avl[6] = {vlmax, vlmax-1, 2*vlmax-1, 2*vlmax+1000, 2*vlmax, 0};
    short int expect;
    int16xm1_t bb;
    int rte = 0;
    unsigned int gvl=0;
    long long max = 0;
    long long min = 0;
    char *name = argv[0];
    for(int t=0; t<N; t++) {
```

```
    gvl = vsetvli(vlmax, RVV_E16, RVV_M1);
    bb=vmvvi_int16xm1(15, gvl);
    for(int i=0; i<vlmax; i++) {
        if ( i < avl[t] ) {
            expect = 15;
        } else {
            expect = 0;
        }
        if ( bb[i]- expect != 0 ) {
            rte = -1;
        }
        printf("Expect = %d, Actual = %d, avl = %d\n",expect, bb[i], avl[t]);
    }
}
if ( rte != 0 )
    printf("FAIL: %s\n",name);
else
    printf("PASS: %s\n",name);
return rte;
}
```

编译命令:

```
riscv64-unknown-linux-gnu-gcc -O2 -march=rv64gcvxthead -mabi=lp64v -g -static -lm testvector.c -o testvector
```

概述

本章介绍在Linux 环境进行软件开发、调试的一些基本知识。掌握Linux 下如果使用 GDB/GDBSERVER 进行基本的调试，还进一步介绍如何通过 coredump 的手段，对出错的应用进行分析。在 [Linux系统性能分析工具集](#) 章节中，还介绍了众多的分析工具，进一步帮助开发者掌握更高阶的系统开发与分析的能力。

GDB调试入门

GDB是GNU开源组织发布的一个强大的UNIX下的程序调试工具。或许，各位比较喜欢那种图形界面方式的，像VC、BCB等IDE的调试，但如果你是在 UNIX平台下做软件，你会发现**GDB**这个调试工具有比VC、BCB的图形化调试器更强大的功能。所谓“寸有所长，尺有所短”就是这个道理。一般来说，**GDB**主要帮忙你完成下面四个方面的功能：

- 启动你的程序
- 可让被调试的程序在你所指定的调置的断点处停下来
- 当程序被停下来看，可以检查此时你的程序中所发生的事
- 动态的改变你程序的执行环境

GDB是一个非常强大的调试工具，即使大家使用习惯了图形化调试工具，但这些图形化的调试工具的后端大多数都是使用 GDB 的，命令行的调试工具却有着图形化工具所不能完成的功能。以下以 `helloworld.c` 为例介绍 GDB 的调试入门：

编写代码

开始工作时，先确认在你的编程环境中是否安装了开发环境，如果未安装，参考 [安装开发环境](#) 章节完成开发环境的安装：

```
#include <stdio.h>

int main(int argc, char **argv)
{
    int i;
    int result = 0;

    printf("hello world.\n");

    for(i = 1; i <= 100; i++) {
        result += i;
    }

    printf("result = %d\n", result );

    return 0;
}
```

编译代码，使用 GDB 调试程序时，在 `gcc` 编译时要带上 `-g` 参数：

```
gcc helloworld.c -o hellowrld -g
```

运行一下程序，看看效果：

```
thead@yoc:~/works > ./hellowrld China
Hello World China!
result = 5050
thead@yoc:~/works > ./hellowrld
helloworld.
result = 5050
```

启动调试

```
$ gdb helloworld
GNU gdb (GDB) Red Hat Enterprise Linux 8.2-12.el8
Copyright (C) 2018 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-redhat-linux-gnu".
Type "show configuration" for configuration details.
```

```
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
  <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from helloworld...done.
(gdb) run                                <----- 不带参数运行
Starting program: /home/zhuzhg/helloworld
Missing separate debuginfos, use: yum debuginfo-install glibc-2.28-101.el8.x86_64
helloworld.
result = 5050
[Inferior 1 (process 1069013) exited normally]
(gdb) run China                          <----- 带参数运行
Starting program: /home/zhuzhg/helloworld China
Hello World China!
result = 5050
[Inferior 1 (process 1071086) exited normally]
(gdb)
```

断点

在介绍之前，我们首先需要了解，为什么需要设置断点。我们在指定位置设置断点之后，程序运行到该位置将会“暂停”，这个时候我们就可以对程序进行更多的操作，比如查看变量内容，堆栈情况等等，以帮助我们调试程序。

设置断点

- 文件行号断点：`break hellowrld.c:9`
- 函数断点：`break main`
- 条件断点：`break helloworld.c:17 if c == 10`
- 临时断点, 假设某处的断点只想生效一次，那么可以设置临时断点，这样断点后面就不复存在了：`tbreak helleworld.c:9`
- 禁用或启动断点：

```
disable                # 禁用所有断点
disable bnum           # 禁用标号为bnum的断点
enable                 # 启用所有断点
enable bnum            # 启用标号为bnum的断点
enable delete bnum     # 启动标号为bnum的断点，并且在此之后删除该断点
```

- 断点清除：

```
clear                  # 删除当前行所有breakpoints
clear function         # 删除函数名为function处的断点
clear filename:function # 删除文件filename中函数function处的断点
clear lineNum          # 删除行号为lineNum处的断点
clear f:lename:lineNum # 删除文件filename中行号为lineNum处的断点
delete                 # 删除所有breakpoints,watchpoints和catchpoints
delete bnum            # 删除断点号为bnum的断点
```

变量查看

在启动调试以及设置断点之后，就到了我们非常关键的一步-查看变量。GDB调试最大的目的之一就是走查代码，查看运行结果是否符合预期。既然如此，我们就不得不了解一些查看各种类型变量的方法，以帮助我们进一步定位问题。

- 变量查看: 最常见的使用便是使用print（可简写为p）打印变量内容。

以上述程序为例：

```
gdb helloworld
```



```
break helloworld.c:17 if i == 0
(gdb) run
Starting program: /home/zhuzhg/helloworld
helloworld.

Breakpoint 2, main (argc=1, argv=0x7fffffffedca8) at helloworld.c:17
17         result += i;
(gdb) print i           <----- 查看变量 i 当前的值
$1 = 10
(gdb) print result      <----- 查看变量 result 当前的值
$2 = 45
(gdb) print argc        <----- 查看变量 argc 当前的值
$3 = 1
(gdb) print str
$4 = 0x4006c8 "Helleo World" <----- 查看变量 str 当前的值
```

- 查看内存：examine(简写为x)可以用来查看内存地址中的值。语法如下：

```
x/[n][f][u] addr
```

其中：

- n 表示要显示的内存单元数，默认值为1
- f 表示要打印的格式，前面已经提到了格式控制字符
- u 要打印的单元长度
- addr 内存地址

单元类型常见有如下：

- b 字节
- h 半字，即双字节
- w 字，即四字节
- g 八字节

示例：

```
(gdb) x/4b str
0x4006c8:  01001000  01100101  01101100  01101100
```

可以看到，变量 str 的四个字节都以二进制的方式打印出来了。

- 查看寄存器内容：

```
ra      0x3ff7ef2282  0x3ff7ef2282 <__libc_start_main+160>
sp      0x3fffffff00  0x3fffffff00
gp      0x2aaaaac800  0x2aaaaac800
tp      0x3ff7fdd250  0x3ff7fdd250
t0      0x3ff7ed60b0  274742468784
t1      0x3ff7ef21e2  274742583778
t2      0x2aaaaac4f0  183251944688
fp      0x3fffffffab0  0x3fffffffab0
s1      0x0          0
a0      0x1          1
a1      0x3fffffff00  274877905960
a2      0x3fffffff00  274877905976
a3      0x0          0
a4      0x3fffffffad8  274877905624
a5      0x0          0
a6      0x3ff7fd88a8  274743527592
(内容过多未显示完全)
```

单步调试

在启动调试设置断点观察之后，没有我们想要的信息怎么办呢？这个时候，就需要单步执行或者跳过当前断点继续执行等等。而本文所说的单步调试并非仅仅指单步执行，而是指在你的控制之下，按要求执行语句。

• 单步执行-**next**：

next命令（可简写为n）用于在程序断住后，继续执行下一条语句，假设已经启动调试，并在第12行停住，如果要继续执行，则使用n执行下一条语句，如果后面跟上数字num，则表示执行该命令num次，就达到继续执行n行的效果了：

```
gdb helloworld                                <----- 加载程序
(gdb) break helloworld.c:18                   <----- 设置断点
(gdb) run                                     <----- 启动调试
The program being debugged has been started already.
Start it from the beginning? (y or n) y
Starting program: /home/zhuzhg/helloworld
Helleo World.

Breakpoint 2, main (argc=1, argv=0x7fffffff dca8) at helloworld.c:18      <----- 程序在 18 行暂停
18      result += i;
Breakpoint 2, main (argc=1, argv=0x7fffffff dca8) at helloworld.c:18
18      result += i;
(gdb) next                                    <----- 单步执行

17      for(i = 1; i <= 100; i++) {

Breakpoint 2, main (argc=1, argv=0x7fffffff dca8) at helloworld.c:18
18      result += i;
(gdb) next 2                                  <----- 执行两次

Breakpoint 2, main (argc=1, argv=0x7fffffff dca8) at helloworld.c:18
18      result += i;
```

• 单步进入-**step**：

如果我们想跟踪add函数内部的情况，可以使用step命令（可简写为s），它可以单步跟踪到函数内部，但前提是该函数有调试信息并且有源码信息。

• 断点继续-**continue**：

我们可能打了多处断点，或者断点打在循环内，这个时候，想跳过这个断点，甚至跳过多次断点继续执行该怎么做呢？可以使用continue命令（可简写为c）或者fg，它会继续执行程序，直到再次遇到断点处。

使用 gdbserver 远程调试

使用 gdbserver 可以对目标设备上的程序进行远程调试。

安装 GDBSERVER

在 ICE-EVB 开发板上，通过下面命令来安装 GDBSEVER。

```
apt install gdbserver
```

gdbserver 用法

Usage: gdbserver [OPTIONS] COMM PROG [ARGS ...]
 gdbserver [OPTIONS] --attach COMM PID
 gdbserver [OPTIONS] --multi COMM

COMM may either be a tty device (for serial debugging),
HOST:PORT to listen for a TCP connection, or '-' or 'stdio' to use
stdin/stdout of gdbserver.
PROG is the executable program. ARGS are arguments passed to inferior.
PID is the process ID to attach to, when --attach is specified.

Operating modes:

--attach	Attach to running process PID.
--multi	Start server without a specific program, and only quit when explicitly commanded.
--once	Exit after the first connection has closed.
--help	Print this message and then exit.
--version	Display version information and exit.

第一步：在目标机上启动 **gdbserver** 服务

要进行**gdb**调试，首先要在目标系统上启动gdbserver服务。在gdbserver所在目录下输入命令：

```
gdbserver :12345 helloworld
```

此时gdbserver监听端口号1234，并等待客户端连接。

第二步：在宿主机上运行 **GDB**：

```
riscv64-linux-gnu-gdb helloworld  
  
(gdb) target remote 192.168.1.4:12345
```

GDB 的调用的使用方法，可以参考：[GDB调试入门](#)

核心转储

在 Linux 系统中，常将“主内存”称为核心(core)，而核心映像(core image) 就是“进程”(process)执行当时的内存内容。当进程发生错误或收到“信号”(signal) 而终止执行时，系统会将核心映像写入一个文件，以作为调试之用，这就是所谓的核心转储(core dump)。

通常在系统收到特定的信号时由操作系统生成。信号可以由程序执行过程中的异常触发，也可以由外部程序发送。动作的结果一般是生成一个某个进程的内存转储的文件，文件包含了此进程当前的运行堆栈信息。有时程序并未经过彻底测试，这使得它在执行的时候一不小心就会找到破坏。这可能会导致核心转储（core dump）。

当在一个程序崩溃时，系统会在指定目录下生成一个core文件，我们就可以通过 core文件来对造成程序崩贵的原因进行调试定位。

开启核心转储

默认情况下，Linux 没有打开core文件生成功能，通过以下命令打开core文件的生成：

```
# 不限制产生 core 的大小
ulimit -c unlimited
```

unlimited 意思是系统不限制core文件的大小，只要有足够的磁盘空间，会转存程序所占用的全部内存，如果需要限制系统产生 core 的大小，可以使用以下命令：

```
# core 最大限制大小为 409600 字节
ulimit -c 409600
```

如果需要关装核心转储功能，只需要将限制大小设为 0 即可：

```
ulimit -c 0
```

上述操作在一个终端中有效，退出或者新打开终端时无效，可以在将上述配置加入到 /etc/profile 中：

```
# 编辑 profile 文件
vi /etc/profile

# 将下行加入到profile 文件中
ulimit -c unlimited
```

使用GDB调试 core 文件

首先我们来编写一个产生错误的程序， vi test.c

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char **argv)
{
    int *p = NULL;

    // 给一个NULL指令赋值，会产生 Segmentation fault 错误
    *p = 100;

    return 0;
}
```

以上程序给一个空指令赋值，这是C语言编程中遇到的段错误，程序运行后，如果开启了核心转储后，就会产生一个 core 文件。

```
# 编译 test.c 生成 test 程序
gcc test.c -o test -g -MD

# 运行该程序
```

```
./test
```

运行后，我们可以看到 `Segmentation fault (core dumped)` 提示信息，表示已经在当前目录下产生了一 `core` 文件，通过 `ls -l core` 命令可以查看：

```
root@thead-910:~# ls -l core
-rw----- 1 root root 225280 Jan  6 10:51 core
```

下面我样就可以通过 `core` 来进行调试：

```
root@thead-910:~# gdb test core
GNU gdb (Debian 10.1-1.7) 10.1.90.20210103-git
Copyright (C) 2021 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "riscv64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from test...
[New LWP 1134]
Core was generated by `./test'.
Program terminated with signal SIGSEGV, Segmentation fault.
#0  0x0000002ae6775600 in main (argc=1, argv=0x3fffb22d18) at test.c:9
9          *p = 100;
```

通过 GDB 可以看到程序的第9行出错。

如果我们想一步步调试，那么可以在gdb中打断点(b)，然后运行(r)，或者一步步(s)调试，

```
(gdb) b test.c:9
Breakpoint 1 at 0x2ae67755f8: file test.c, line 9.
(gdb) r
Starting program: /root/test

Breakpoint 1, main (argc=1, argv=0x3fffffffce8) at test.c:9
9          *p = 100;
(gdb) p p
$1 = (int *) 0x0
(gdb) n

Program received signal SIGSEGV, Segmentation fault.
0x0000002aaaaaa600 in main (argc=1, argv=0x3fffffffce8) at test.c:9
9          *p = 100;
(gdb) quit
A debugging session is active.

        Inferior 1 [process 1198] will be killed.

Quit anyway? (y or n) y
```

使用 GDB 调式 `core` 的命令格式如下：

```
gdb <程序> <core文件>
```


Perf 使用手册

安装

安装 perf

```
apt install -y perf
```

安装 flamegraph

火焰图是脚本，只需要下载:

```
git clone https://github.com/brendangregg/FlameGraph.git
```

认识 Perf

下面将由一个最简单的perf命令，初步了解perf的功能：

```
perf stat -e cycles ls
```

```
Performance counter stats for 'ls':

          1040224          cycles

          0.001632667 seconds time elapsed

          0.001925000 seconds user
          0.000000000 seconds sys
```

上述命令将展示，执行ls这一条命令，cpu运行了多少条指令。

而perf中还能提供更多的此种信息，例如ls命令运行时执行了多少条分支跳转指令，分支跳转预测的成功概率又是多少，在这期间发生了多少次上下文切换，多少次page fault，cache的命中率又如何，等等。

```
perf list
```

```
branch-instructions OR branches      [Hardware event]
branch-misses                        [Hardware event]
cache-misses                         [Hardware event]
... ..

context-switches OR cs               [Software event]
page-faults OR faults                [Software event]
... ..

L1-dcache-load-misses                [Hardware cache event]
L1-dcache-loads                      [Hardware cache event]
... ..

block:block_bio_backmerge            [Tracepoint event]
clk:clk_enable                       [Tracepoint event]
dma_fence:dma_fence_destroy          [Tracepoint event]
... ..
```

其中

- hardware event 是硬件相关的观测点

- software event 是软件相关的观测点
- hardware cache event 是cache相关的观测点
- tracepoint event 是内核设置的观测点

Perf 功能概览

perf有很多子命令，之前的例子中用到的stat只是其中之一

perf	
annotate	Read perf.data (created by perf record) and display annotated code
archive	Create archive with object files with build-ids found in perf.data file
bench	General framework for benchmark suites
buildid-cache	Manage build-id cache.
buildid-list	List the buildids in a perf.data file
c2c	Shared Data C2C/HITM Analyzer.
config	Get and set variables in a configuration file.
data	Data file related processing
diff	Read perf.data files and display the differential profile
evlist	List the event names in a perf.data file
ftrace	simple wrapper for kernel's ftrace functionality
inject	Filter to augment the events stream with additional information
kallsyms	Searches running kernel for symbols
kmem	Tool to trace/measure kernel memory properties
kvm	Tool to trace/measure kvm guest os
list	List all symbolic event types
lock	Analyze lock events
mem	Profile memory accesses
record	Run a command and record its profile into perf.data
report	Read perf.data (created by perf record) and display the profile
sched	Tool to trace/measure scheduler properties (latencies)
script	Read perf.data (created by perf record) and display trace output
stat	Run a command and gather performance counter statistics
test	Runs sanity tests.
timechart	Tool to visualize total system behavior during a workload
top	System profiling tool.
version	display the version of perf binary
probe	Define new dynamic tracepoints

perf的子命令大致可分为2类

- 基础子命令：用于做一些性能数据的列举，统计和展示
- 功能型子命令：针对特定的功能进行数据的采样和分析

基础子命令

子命令名	作用
list	列出当前系统支持的所有事件，分为4种类型：hardware event，software event，hardware cache event，tracepoint event
stat	执行某个命令，收集特定进程的性能概况
record	收集采样信息，并将其记录在数据文件中，随后可通过其它工具对数据文件进行分析
report	读取perf record创建的数据文件，并给出分析结果
probe	动态增加tracepoint
script	执行perl或python写的功能扩展脚本、生成脚本框架、读取数据文件中的数据信息等
timechart	分析CPU在一段时间内的使用情况

功能型子命令

子命令名	作用
lock	分析内核中锁的使用情况
kmem	分析内核slab的使用情况
sched	分析内核调度事件的情况
top	top的升级版本，能够看到各个进程实时运行的函数
bench	perf内部的一个benchmark木桶

bench	perf 目的 + benchmark名称
evlist	查看perf.data采样了的事件
kallsyms	判别一个符号是否在当前运行的kernel中导出

Perf 基础子命令

perf list

```
perf list
```

branch-instructions OR branches	[Hardware event]
branch-misses	[Hardware event]
bus-cycles	[Hardware event]
cache-misses	[Hardware event]
cache-references	[Hardware event]
cpu-cycles OR cycles	[Hardware event]
instructions	[Hardware event]
ref-cycles	[Hardware event]
stalled-cycles-backend OR idle-cycles-backend	[Hardware event]
stalled-cycles-frontend OR idle-cycles-frontend	[Hardware event]
... ..	
alignment-faults	[Software event]
bpf-output	[Software event]
context-switches OR cs	[Software event]
cpu-clock	[Software event]
cpu-migrations OR migrations	[Software event]
dummy	[Software event]
emulation-faults	[Software event]
major-faults	[Software event]
minor-faults	[Software event]
... ..	
L1-dcache-load-misses	[Hardware cache event]
L1-dcache-loads	[Hardware cache event]
L1-dcache-store-misses	[Hardware cache event]
L1-dcache-stores	[Hardware cache event]
LLC-load-misses	[Hardware cache event]
LLC-loads	[Hardware cache event]
LLC-store-misses	[Hardware cache event]
LLC-stores	[Hardware cache event]
... ..	
block:block_bio_backmerge	[Tracepoint event]
clk:clk_enable	[Tracepoint event]
dma_fence:dma_fence_destroy	[Tracepoint event]
ext4:ext4_alloc_da_blocks	[Tracepoint event]
gadget:usb_ep_alloc_request	[Tracepoint event]
irq:irq_handler_exit	[Tracepoint event]
irq:softirq_entry	[Tracepoint event]
irq:softirq_exit	[Tracepoint event]
irq:softirq_raise	[Tracepoint event]
jbd2:jbd2_checkpoint	[Tracepoint event]
kmem:kmalloc	[Tracepoint event]
mmc:mmc_request_done	[Tracepoint event]
neigh:neigh_update_done	[Tracepoint event]
net:napi_gro_frags_entry	[Tracepoint event]
nfs4:ff_layout_commit_error	[Tracepoint event]
oom:compact_retry	[Tracepoint event]
page_pool:page_pool_release	[Tracepoint event]
pagemap:mm_lru_activate	[Tracepoint event]
percpu:percpu_alloc_percpu	[Tracepoint event]
power:clock_disable	[Tracepoint event]

rcu:rcu_utilization	[Tracepoint event]
regmap:regcache_drop_region	[Tracepoint event]
regmap:regmap_reg_write	[Tracepoint event]
rpcgss:rpcgss_bad_seqno	[Tracepoint event]
sched:sched_kthread_stop	[Tracepoint event]
signal:signal_deliver	[Tracepoint event]
skb:consume_skb	[Tracepoint event]
smbus:smbus_read	[Tracepoint event]
sock:inet_sock_set_state	[Tracepoint event]
spi:spi_controller_busy	[Tracepoint event]
swiotlb:swiotlb_bounced	[Tracepoint event]
task:task_newtask	[Tracepoint event]
tcp:tcp_destroy_sock	[Tracepoint event]
timer:hrtimer_cancel	[Tracepoint event]
vmscan:mm_shrink_slab_end	[Tracepoint event]
workqueue:workqueue_activate_work	[Tracepoint event]
writeback:balance_dirty_pages	[Tracepoint event]

其中主要分为4种类型的采样事件

- hardware event 是硬件相关的观测点
- software event 是软件相关的观测点
- hardware cache event 是cache相关的观测点
- tracepoint event 是内核设置的观测点

perf stat

perf stat子命令可以采样perf list种展示的4种类型的数据，并展示。

同时，perf stat子命令也具有相当多的参数，其中，-e参数用来指定具体的event，是必选的。

```
perf stat -h
```

-a, --all-cpus	system-wide collection from all CPUs
-A, --no-aggr	disable CPU count aggregation
-B, --big-num	print large numbers with thousands' separators
-C, --cpu <cpu>	list of cpus to monitor in system-wide
-D, --delay <n>	ms to wait before starting measurement after program start (-1: start with events disabled)
-d, --detailed	detailed run - start a lot of events
-e, --event <event>	event selector. use 'perf list' to list available events
-G, --cgroup <name>	monitor event in cgroup name only
-g, --group	put the counters into a counter group
... ..	

实例:

hardware event：

```
perf stat -e instructions ls
```

Performance counter stats for 'ls':	
267995	instructions
... ..	

可以看到，执行ls，cpu运行了267995条指令

software event：

```
perf stat -e context-switches,page-faults dmesg
```

```
Performance counter stats for 'dmesg':

              6      context-switches
             35      page-faults
... ..
```

可以看到，执行dmesg，cpu发生了6次上下文切换，35次page fault

hardware cache event：

```
perf stat -e L1-dcache-store-misses ls

Performance counter stats for 'ls':

             365      L1-dcache-store-misses
... ..
```

可以看到，执行ls，cpu发生了365次L1 cache的store miss（写不命中）

tracepoint event：

tracepoint event本身就分为很多种类型，例如：block:block_bio_backmerge，就表示block这个类型的block_bio_backmerge这个采样点。

tracepoint event支持采样类型和采样点非常多，有clk，mmc，sched等类型，而其中每个采样类型又分为更多的采样点，因此，可以通过perf list后面跟上采样类型的名字，来查看某个特性类型的采样点。

```
perf list sched

sched:sched_kthread_stop      [Tracepoint event]
sched:sched_kthread_stop_ret  [Tracepoint event]
sched:sched_migrate_task      [Tracepoint event]
sched:sched_move_numa         [Tracepoint event]
sched:sched_pi_setprio        [Tracepoint event]
sched:sched_process_exec      [Tracepoint event]
sched:sched_process_exit      [Tracepoint event]
sched:sched_process_fork      [Tracepoint event]
sched:sched_process_free      [Tracepoint event]
sched:sched_process_hang      [Tracepoint event]
sched:sched_process_wait      [Tracepoint event]
sched:sched_stat_runtime      [Tracepoint event]
sched:sched_stick_numa        [Tracepoint event]
sched:sched_swap_numa         [Tracepoint event]
sched:sched_switch            [Tracepoint event]
sched:sched_wait_task         [Tracepoint event]
sched:sched_wake_idle_without_ipi [Tracepoint event]
sched:sched_wakeup            [Tracepoint event]
sched:sched_wakeup_new        [Tracepoint event]
sched:sched_waking            [Tracepoint event]
```

可以看到，一个sched类型，对应很多采样点。

这些采样点由内核代码中已经写好的代码构成，例如：

```
2458 static void ttwu_do_wakeup(struct rq *rq, struct task_struct *p, int wake_flags,
2459                          struct rq_flags *rf)
2460 {
2461     check_preempt_curr(rq, p, wake_flags);
2462     p->state = TASK_RUNNING;
2463     trace_sched_wakeup(p);
... ..
```

第2463行的trace_sched_wakeup就是一个tracepoint，对应着sched:sched_wakeup这个采样点。


```
4410 static void __sched notrace __schedule(bool preempt)
4411 {
    ...
4525     trace_sched_switch(preempt, prev, next);
    ...
```

第4525行的trace_sched_switch就是一个tracepoint，对应着sched:sched_switch这个采样点。

通过perf stat子命令可以得到观测结果

```
perf stat -e sched:sched_switch -a sleep 1
```

```
Performance counter stats for 'system wide':

          29          sched:sched_switch
```

可以看到，执行ls，cpu运行了29次sched_switch，也就是对应的__schedule函数。

但看这个效果，貌似tracepoint event功能并不强大，但其实tracepoint event最强大的地方在于时间的统计，内核中每个采样点都代表了一些重要的时刻，比如，进程切换了，调度器把进程放入runqueue，或者开始真正的进程运行，都会加上一个tracepoint，来记录时间，从而为分析和调试提供支持。

perf record & perf report

perf stat子命令展示的是即时的数据，若想要获取更多信息，则需要用perf record子命令将信息组成一个perf.data文件，并利用perf report将其解析并展示出来。

同样的，perf record和perf report子命令也具有相当多的参数，其中，-e参数用来指定具体的event，是必选的

```
perf record -h
```

```
-a, --all-cpus      system-wide collection from all CPUs
-b, --branch-any    sample any taken branches
-B, --no-buildid    do not collect buildids in perf.data
-c, --count <n>     event period to sample
-C, --cpu <cpu>     list of cpus to monitor
-d, --data          Record the sample addresses
-D, --delay <n>     ms to wait before starting measurement after program start (-1: start with events disabled)
-e, --event <event> event selector. use 'perf list' to list available events
... ..
```

```
perf report -h
```

```
-b, --branch-stack  use branch records for per branch histogram filling
-c, --comms <comm[,comm...]>
only consider symbols in these comms
-C, --cpu <cpu>     list of cpus to profile
... ..
```

实例

hardware event：

```
perf record -e instructions -a ls
```

```
perf.data
[ perf record: Woken up 1 times to write data ]
[ perf record: Captured and wrote 0.018 MB perf.data (19 samples) ]
```

可以看到，perf record子命令生成了一个perf.data；继续用perf report解析并展示这个perf.data

```
perf report

# Samples: 19  of event 'instructions'
# Event count (approx.): 767713
#
# Overhead  Command  Shared Object      Symbol
# .....  .....  .....
11.71%  ls      busybox            [.] 0x00000000000018954
11.67%  ls      [kernel.kallsyms] [k] fput
11.64%  ls      [kernel.kallsyms] [k] __might_sleep
10.72%  ls      [kernel.kallsyms] [k] do_page_fault
8.83%   perf    [kernel.kallsyms] [k] free_pages_and_swap_cache
8.27%   perf    [kernel.kallsyms] [k] unmap_page_range
7.54%   perf    [kernel.kallsyms] [k] page_remove_rmap
7.14%   perf    [kernel.kallsyms] [k] __alloc_pages_nodemask
7.02%   perf    [kernel.kallsyms] [k] __asm_copy_to_user
5.85%   perf    [kernel.kallsyms] [k] dput.part.30
4.21%   perf    [kernel.kallsyms] [k] handle_mm_fault
1.78%   perf    [kernel.kallsyms] [k] seq_printf
1.49%   swapper [kernel.kallsyms] [k] finish_task_switch
1.07%   perf    [kernel.kallsyms] [k] _raw_spin_unlock_irqrestore
1.07%   perf    [kernel.kallsyms] [k] fput
0.00%   perf    [kernel.kallsyms] [k] generic_exec_single
```

可以看到，perf report子命令不光展示了cpu执行ls所运行的instruction数，还展示了这些指令数都消耗在哪些进程和函数中。

software event：

```
perf record -e context-switches -a sleep 1

[ perf record: Woken up 1 times to write data ]
[ perf record: Captured and wrote 0.018 MB perf.data (17 samples) ]

# perf report
# Samples: 17  of event 'context-switches'
# Event count (approx.): 36
#
# Overhead  Command      Shared Object      Symbol
# .....  .....  .....
47.22%  swapper      [unknown]          [.] 0000000000000000
11.11%  perf         [unknown]          [.] 0000000000000000
8.33%   kcompactd0  [unknown]          [.] 0000000000000000
8.33%   kworker/0:1-eve [unknown]          [.] 0000000000000000
8.33%   rcu_sched    [unknown]          [.] 0000000000000000
5.56%   kworker/1:0-mm_ [unknown]          [.] 0000000000000000
5.56%   sleep        [unknown]          [.] 0000000000000000
2.78%   migration/0  [unknown]          [.] 0000000000000000
2.78%   ntpd         [unknown]          [.] 0000000000000000
```

可以看到，perf report子命令不光展示了cpu执行sleep 1时发生的上下文切换次数，还展示了这切换都分布在哪些进程中。

hardware cache event：

```
perf record -e L1-dcache-load-misses -a ls

perf.data      perf.data.old
[ perf record: Woken up 1 times to write data ]
[ perf record: Captured and wrote 0.018 MB perf.data (3 samples) ]

# perf report
```

```
# Samples: 3  of event 'L1-dcache-load-misses'
# Event count (approx.): 4098
#
# Overhead  Command  Shared Object      Symbol
# .....
    99.95%  ls          busybox             [.] 0x000000000000c293c
    0.05%  perf        [kernel.kallsyms]  [k] generic_exec_single
```

可以看到，perf report子命令不光展示了cpu执行ls时发生的L1 dcache load miss次数，还展示了这些miss都分布在哪些进程和函数中。

tracepoint event：

```
perf record -e sched:sched_switch -a ls
```

```
perf.data      perf.data.old
[ perf record: Woken up 1 times to write data ]
[ perf record: Captured and wrote 0.019 MB perf.data (10 samples) ]
# perf report
# To display the perf.data header info, please use --header/--header-only options.
#
#
# Total Lost Samples: 0
#
# Samples: 10  of event 'sched:sched_switch'
# Event count (approx.): 10
#
# Overhead  Command      Shared Object      Symbol
# .....
#
    40.00%  swapper      [unknown]          [.] 0000000000000000
    30.00%  perf         [unknown]          [.] 0000000000000000
    10.00%  ls           [unknown]          [.] 0000000000000000
    10.00%  migration/0  [unknown]          [.] 0000000000000000
    10.00%  migration/1  [unknown]          [.] 0000000000000000
```

可以看到，perf report子命令不光展示了cpu执行ls时发生的运行至sched:sched_switch这个tracepoint的次数，还展示了这tracepoint都分布在哪些进程中。

小结一下，对比stat和record，会发现，stat记录的数据是一种counting（计数）类型的数据，而record记录的是一种sampling（采样）类型的数据；record的原理是依赖定期地中断上报来搜集cpu运行状态的信息，所以负荷也会比stat高一些。

perf probe

有的时候，tracepoint不能满足分析过程中的需求，比如在pick_next_task_fair这个函数里没有任何tracepoint，那该如何对这个函数的调用情况进行分析呢？这时候就可以利用perf probe动态地设置一个tracepoint

```
perf probe pick_next_task_fair
```

```
Added new event:
  probe:pick_next_task_fair (on pick_next_task_fair)

You can now use it in all perf tools, such as:

    perf record -e probe:pick_next_task_fair -aR sleep 1
```

```
perf probe --list
```

```
probe:pick_next_task_fair (on pick_next_task_fair)
```

可以看到，perf probe子命令已经设置了一个新的tracepoint，类型是probe，采样点是pick_next_task_fair

接下来可以通过之前介绍过的perf stat和perf record/report来采样和解析以及展示

```
perf stat -e probe:pick_next_task_fair -a sleep 1

Performance counter stats for 'system wide':

                24      probe:pick_next_task_fair

# perf record -e probe:pick_next_task_fair -a sleep 1
[ perf record: Woken up 1 times to write data ]
[ perf record: Captured and wrote 0.020 MB perf.data (28 samples) ]

# perf report
#
# Samples: 28  of event 'probe:pick_next_task_fair'
# Event count (approx.): 28
#
# Overhead  Command          Shared Object      Symbol
# .....  .....
50.00%  swapper             [unknown]          [.] 0xffffffffe000245b6e
 7.14%  kcompactd0         [unknown]          [.] 0xffffffffe000245b6e
 7.14%  perf                [unknown]          [.] 0xffffffffe000245b6e
 7.14%  rcu_sched           [unknown]          [.] 0xffffffffe000245b6e
 7.14%  sleep               [unknown]          [.] 0xffffffffe000245b6e
 3.57%  khungtaskd         [unknown]          [.] 0xffffffffe000245b6e
 3.57%  kworker/0:1-eve     [unknown]          [.] 0xffffffffe000245b6e
 3.57%  kworker/1:0-eve     [unknown]          [.] 0xffffffffe000245b6e
 3.57%  migration/0         [kernel.kallsyms]  [k] pick_next_task_fair
 3.57%  migration/1         [kernel.kallsyms]  [k] pick_next_task_fair
 3.57%  ntpd                [unknown]          [.] 0xffffffffe000245b6e
```

该功能其实依赖了kprobe，这种动态设置的方法实际上是利用的kprobe的功能，只不过perf将其集成为自己的一个功能项（子命令），使perf本身成为了一个入口（前端），使用更加便捷。

perf script

perf script子命令可以展示最原始的采样细节（比perf record & report还要丰富的细节）。

在通过perf record子命令记录了cpu-cycles的事件后，再利用perf script子命令就可以解析出最原始的采样信息，但这种信息不利于人眼解读

```
perf 202 [000] 1724.046093:      1 cycles ffffffff000495664 generic_exec_single+0x88
perf 202 [000] 1724.046109: 4096 cycles ffffffff000495664 generic_exec_single+0x88
perf 202 [000] 1724.046117: 4096 cycles ffffffff000495664 generic_exec_single+0x88
perf 202 [000] 1724.046125:11264 cycles ffffffff000495664 generic_exec_single+0x88
perf 202 [000] 1724.046166:46273 cycles ffffffff000abc7a8 _raw_spin_unlock_irqrest+0x3c
perf 202 [001] 1724.046257:      1 cycles ffffffff000495664 generic_exec_single+0x88
perf 202 [001] 1724.046270: 4096 cycles ffffffff000495664 generic_exec_single+0x88
perf 202 [001] 1724.046279: 4096 cycles ffffffff000495664 generic_exec_single+0x88
perf 202 [001] 1724.046287:13184 cycles ffffffff000495664 generic_exec_single+0x88
perf 202 [001] 1724.046334:57314 cycles ffffffff0007c66c2 strncpy_from_user+0x40
```

火焰图

因此perf script主要被用来生成perf.unfold文件，被交给一个名叫FlameGraph（火焰图）的工具，这个工具会解析perf.unfold数据，然后将其转换成易于人类阅读和分析的图形。

下载安装 FlameGrap 工具：

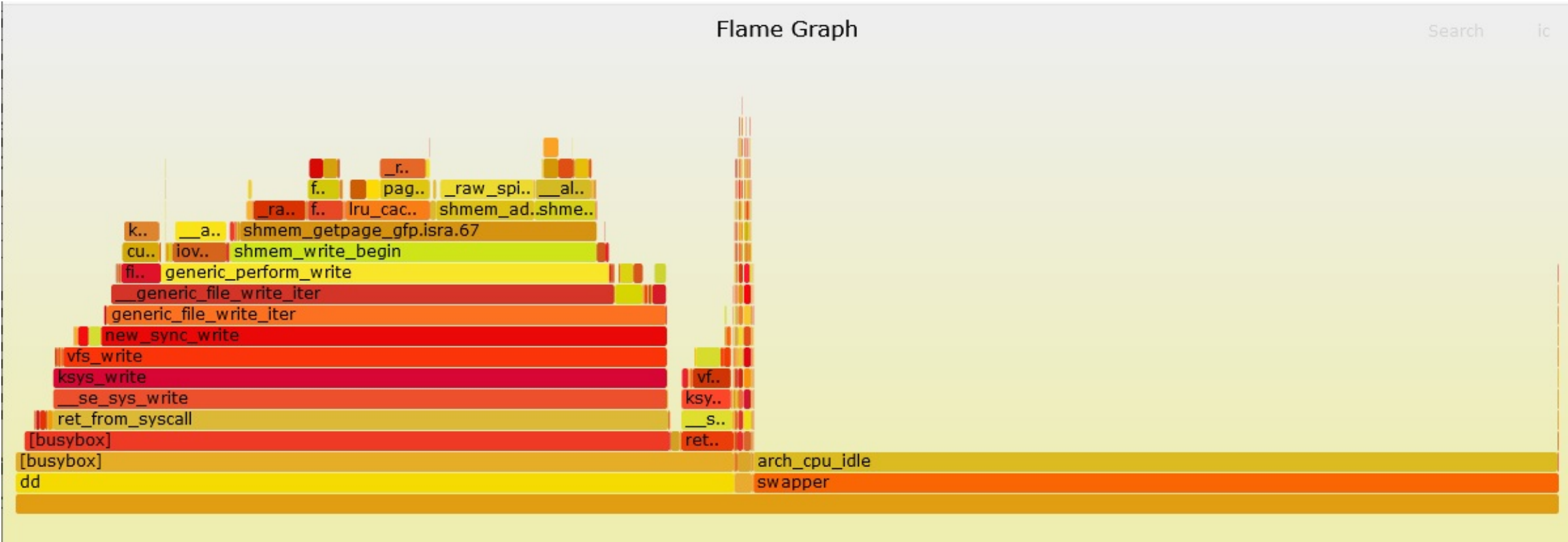
```
wget "https://github.com/brendangregg/FlameGraph/archive/master.zip"
unzip master.zip
```

On CPU火焰图

如果希望了解CPU在一段时间内的都运行了哪些函数以及各函数都消耗了多少时间，就可以使用On CPU火焰图，这种火焰图基于cpu-cycles事件进行采样，然后通过svg图片格式展现出来

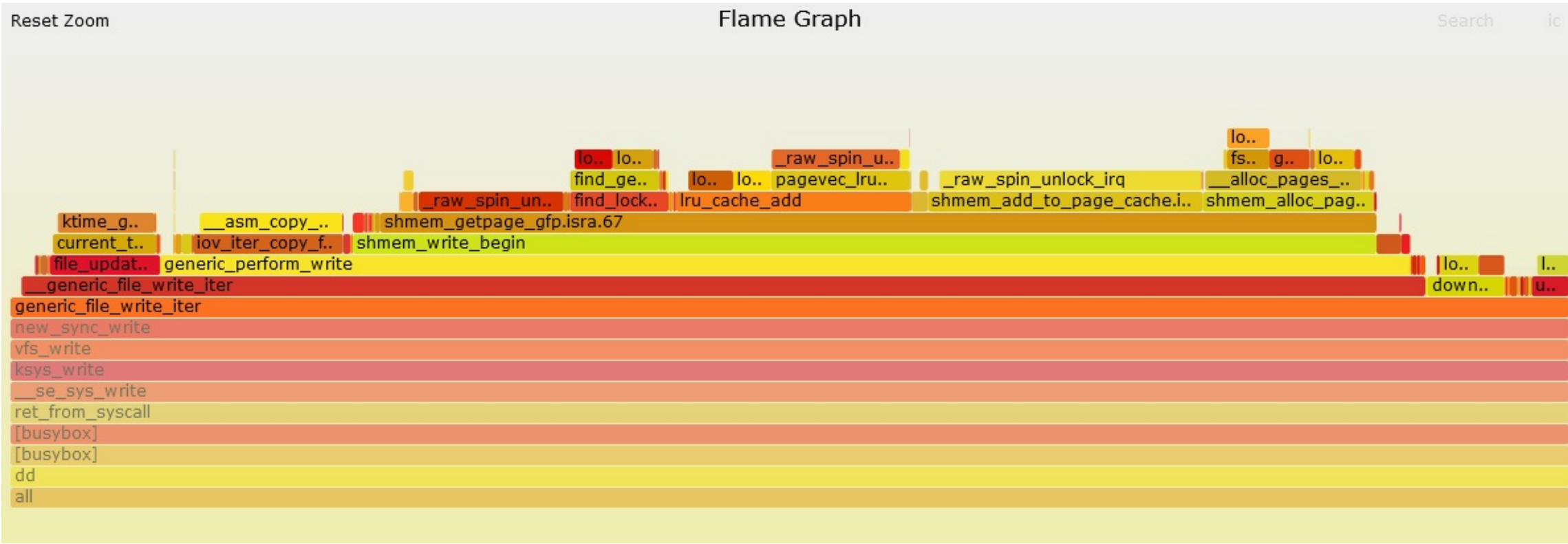
```
dd if=/dev/zero of=/tmp/testfile bs=4K count=102400 &
perf record -e cpu-cycles -a -g sleep 1
perf script > perf.unfold
cd FlameGraph-master
./stackcollapse-perf.pl < ../perf.unfold | ./flamegraph.pl > ../perf.svg
```

首先在后台启动一个dd命令，让它持续运行一段时间，然后开启perf record，记录一秒钟内cpu都运行了多少个cpu-cycles，也就是时间（同时使能-g，就会一并记录运行的函数以及调用关系），再利用perf script命令将perf.data转成perf.unfold，最后利用FlameGraph工具将其转换成一个perf.svg，这是一个图形文件，用浏览器打开后会得到这样一幅图：



此图记录着函数调用关系及其cpu-cycles（时间）占比，就像一缕缕升起的火苗，所以被称之为火焰图。

火焰图还可以通过鼠标点击放大，观察其细节，如下：

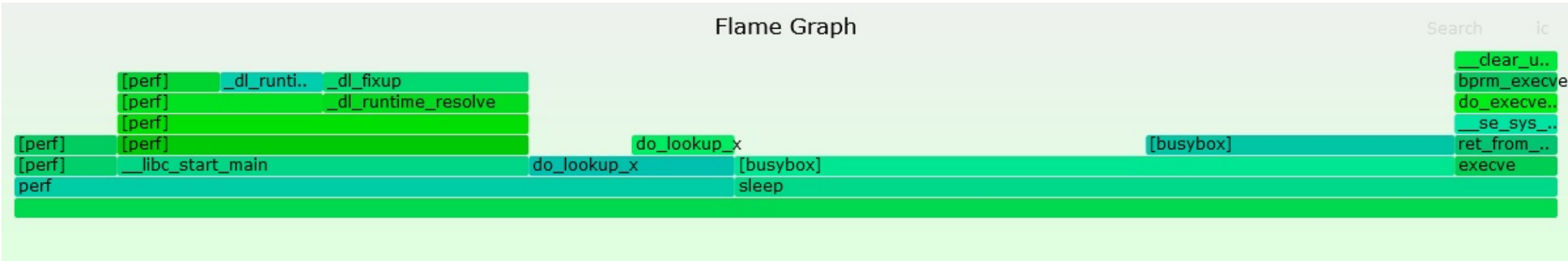


Mem火焰图

火焰图当然也可以基于内存，但目前基于内存的事件玄铁910还未支持，所以暂时用page fault事件来代替

```
perf record -e page-faults -a -g sleep 1
perf script > perf.unfold
cd FlameGraph
./stackcollapse-perf.pl < ../perf.unfold | ./flamegraph.pl --colors=mem > ../perf.svg
```

用浏览器打开perf.svg，就可以看到page fault发生的分布



值得注意的是，Mem火焰图在生成时会使用--colors=mem参数，来生成一个绿色的火焰图，以示和On CPU火焰图的区别。

如果能基于malloc进行采样的话，就可以分析出一个应用程序调用malloc的分布



但可惜现在还有点bug，正在修复中

Off CPU火焰图

如果要看看CPU对IO的使用情况，可以制作Off CPU火焰图。

但目前Off CPU火焰图现在还有些问题，正在修复中。

Diff火焰图

diff火焰图可以将2个unfold文件进行对比，然后生成一个总的Diff火焰图，这样就可以清晰地看出2个火焰图的差异。

如果需要分析诸如”两种配置参数下性能差异的问题“，这将会是非常好的方法。

使用方法和上述方法无异，只是要将生成的fold文件使用difffold.pl进行输出即可

```
./difffolded.pl perf.folded1 perf.folded2 | ./flamegraph.pl > diff.svg
```

perf timechart

火焰图适合分析热点函数，或者指令，但若想分析时序问题（进程/或者cpu在一段时间内都在干什么，例如狂占cpu，或者idle，或者在等待io），就得使用perf timechart子命令。

为了展示timechart的作用，先跑起2个cpu消耗型的测试程序，代码如下：

```
void *a(void *p)
{
    while (1) {
        for (int i = 0; i < 1000000000; i++);
        usleep(200000);
    }
    return NULL;
}

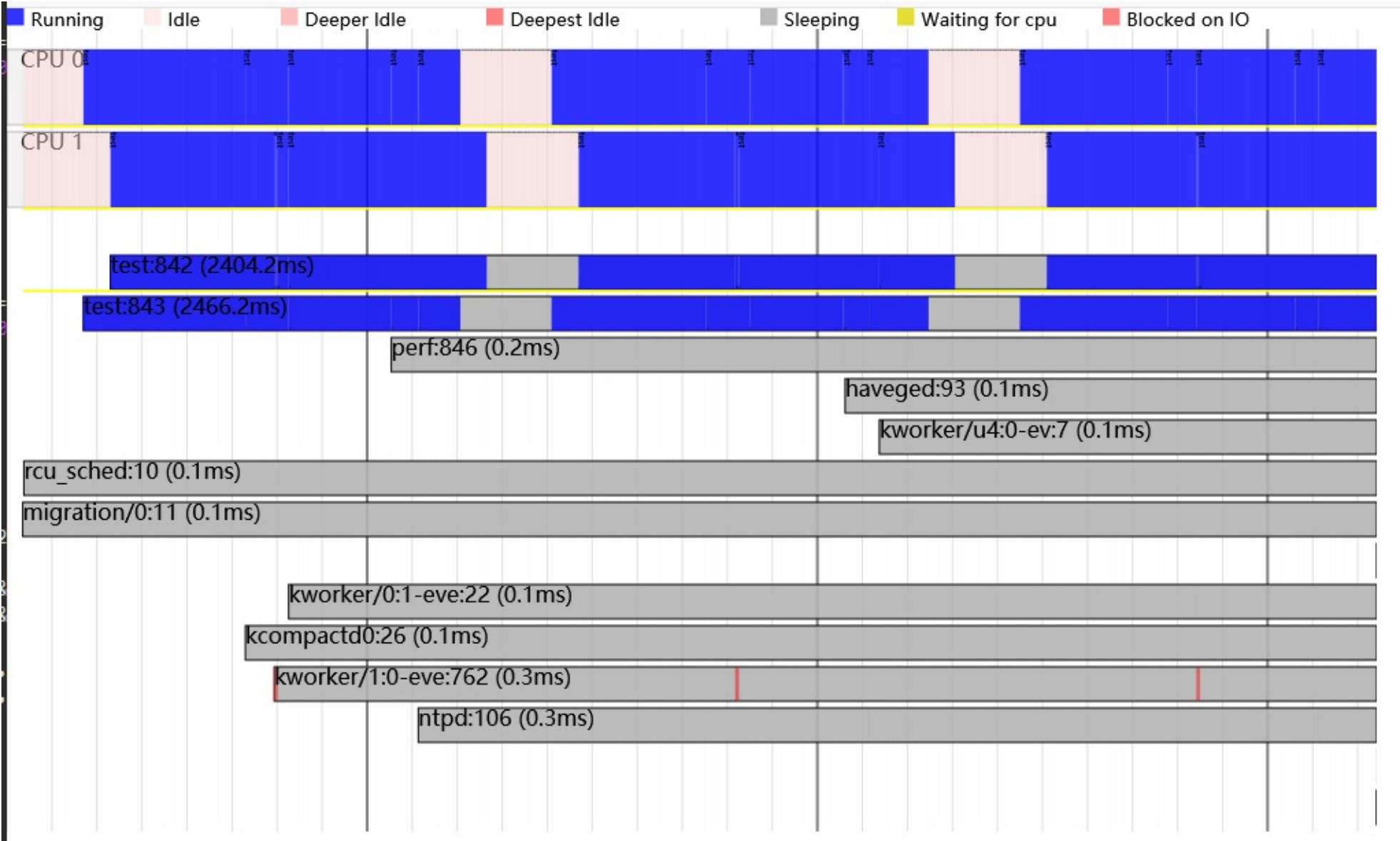
void *b(void *p)
{
    while (1) {
        for (int i = 0; i < 1000000000; i++);
        usleep(200000);
    }
    return NULL;
}

int main(void)
{
    pthread_t t1, t2;
    pthread_create(&t1, NULL, a, NULL);
    pthread_create(&t2, NULL, b, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    return 0;
}
```

```
./test &
perf timechart record -g -a sleep 3
```

```
perf timechart
```

然后启动perf timechart子命令开始记录，最后再用perf timechart子命令生成一个output.svg图，将之用浏览器打开



图中最上方的标记所示解释了每个颜色的含义

接下来两行表示两个cpu的运行状态，如果所示，两个cpu大部分时间里都是处于Running状态的，只有少部分事件是处于Idle

再接下来就是每个进程小号cpu的情况了，如果所示，大部分进程都是灰色的，表示它们都在sleeping，处于一个等待调度的状态，由于系统负载很低，它们即使被调度到了，也不干什么事情，所以看上去就一直处于一个灰色的状态。

综上，perf的基础子命令就介绍完了

总结一下：

- perf的list子命令可以展示所有的事件
- perf可以通过stat子命令直接展示信息
- 也可以通过record子命令生成perf.data，然后利用report展示更多的信息，但script命令可以展示最原始的信息，但并易于阅读，所以可以用来生成火焰图
- perf的stat子命令也可以读取perf.data（这有点特殊，只在perf kmem子命令中被用到）
- perf的probe子命令可以不修改代码地增加观测事件

接下来介绍perf功能型子命令

perf lock

perf lock子命令可以展示一定时间内系统对锁的使用情况

```
perf lock record -a sleep 1
perf report
```

```
# Samples: 38K of event 'lock:lock_acquire'
# Event count (approx.): 38474
89.15% perf [unknown] [.] 0000000000000000
8.35% swapper [unknown] [.] 0000000000000000
1.05% rcu_sched [unknown] [.] 0000000000000000
0.92% sleep [unknown] [.] 0000000000000000
0.18% ksoftirqd/1 [unknown] [.] 0000000000000000
0.11% ntpd [unknown] [.] 0000000000000000
0.06% kcompactd0 [unknown] [.] 0000000000000000
0.05% haveged [unknown] [.] 0000000000000000
0.04% kworker/1:0-mm_ [unknown] [.] 0000000000000000
0.04% migration/1 [unknown] [.] 0000000000000000
0.03% kworker/0:1-mm_ [unknown] [.] 0000000000000000
0.03% migration/0 [unknown] [.] 0000000000000000
```

```
# Samples: 25K of event 'lock:lock_contended'
# Event count (approx.): 25431
 91.47%  perf                [unknown]          [.] 0000000000000000
  6.49%  swapper             [unknown]          [.] 0000000000000000
  1.05%  rcu_sched            [unknown]          [.] 0000000000000000
  0.66%  sleep                [unknown]          [.] 0000000000000000
  0.08%  ntpd                 [unknown]          [.] 0000000000000000
  0.06%  kcompactd0           [unknown]          [.] 0000000000000000
  0.04%  migration/1          [unknown]          [.] 0000000000000000
  0.04%  haveged              [unknown]          [.] 0000000000000000
  0.04%  migration/0          [unknown]          [.] 0000000000000000
  0.02%  kworker/0:1-mm_       [unknown]          [.] 0000000000000000
  0.02%  kworker/1:0-mm_       [unknown]          [.] 0000000000000000
  0.02%  ksoftirqd/1           [unknown]          [.] 0000000000000000

# Samples: 38K of event 'lock:lock_release'
# Event count (approx.): 38448
 89.14%  perf                [unknown]          [.] 0000000000000000
  8.36%  swapper             [unknown]          [.] 0000000000000000
  1.05%  rcu_sched            [unknown]          [.] 0000000000000000
  0.93%  sleep                [unknown]          [.] 0000000000000000
  0.18%  ksoftirqd/1          [unknown]          [.] 0000000000000000
  0.11%  ntpd                 [unknown]          [.] 0000000000000000
  0.06%  kcompactd0           [unknown]          [.] 0000000000000000
  0.05%  haveged              [unknown]          [.] 0000000000000000
  0.04%  migration/1          [unknown]          [.] 0000000000000000
  0.03%  kworker/0:1-mm_       [unknown]          [.] 0000000000000000
  0.03%  migration/0          [unknown]          [.] 0000000000000000
  0.03%  kworker/1:0-mm_       [unknown]          [.] 0000000000000000
```

perf report展示了1秒钟内，系统的锁获取，锁竞争，锁释放的情况

perf lock子命令，其实就是tracepoint event采样事件中的lock事件

```
perf list lock
```

```
... ..
lock:lock_acquire           [Tracepoint event]
lock:lock_acquired          [Tracepoint event]
lock:lock_contended         [Tracepoint event]
lock:lock_release           [Tracepoint event]
... ..
```

像这样的子命令，还有perf kmem和perf sched，但它们又不仅仅是一个简单的wrapper，而是具有更加具有可读性的展示。

perf kmem

perf kmem子命令用于记录内核slab分配器的事件，使用的是tracepoint event中的kmem事件类型

```
perf kmem record ls
perf kmem stat
```

```
SUMMARY (SLAB allocator)
=====
Total bytes requested: 56220
Total bytes allocated: 56824
Total bytes freed:      52968
Net total bytes allocated: 3856
Total bytes wasted on internal fragmentation: 604
Internal fragmentation: 1.062931%
Cross CPU allocations: 0/55
```

简单的stat输出的信息只是一个概要，但如果使用


```
perf kmem --alloc --caller --slab stat
```

就会展示更加丰富的信息

Callsite	Total_alloc	Total_req	Hit	Ping-pong	Frag	

load_elf_phdrs+6e	512/512	280/280	1	0	45.312%	
__alloc_skb+98	1024/1024	576/576	1	0	43.750%	
proc_self_get_link+b6	16/16	11/11	1	0	31.250%	
ext4_readdir+5e8	64/64	48/48	1	0	25.000%	
alloc_bprm+46	1024/512	832/416	2	1	18.750%	
... ..						

0xffffffffe0000c9e00	512/512	280/280	1	0	45.312%	
0xffffffffe00010e000	1024/1024	576/576	1	0	43.750%	
0xffffffffe00201b2b0	16/16	11/11	1	0	31.250%	
0xffffffffe0000dba80	64/64	48/48	1	0	25.000%	
0xffffffffe0000db780	64/64	50/50	1	0	21.875%	
... ..						

其中Callsite记录了调用kmalloc的地方，Hit记录了调用的次数，Total_alloc/Per记录了分配的大小，Frag表示碎片占比，Ping-pong记录了申请内存和释放内存是否在同一个cpu上，这对于cache的性能调优会有所助益，Alloc Ptr由--alloc选项导出，记录了申请的内存的虚拟地址。

perf sched

perf sched子命令用于记录内核调度相关的事件，使用的是tracepoint event中的sched事件类型

```
perf sched record ls
```

在通过record记录了sched事件后，可以通过report和script来查看，这里就不举例了，来看看perf sched子命令有什么强力的拓展

通过dd命令生成一个文件在eMMC上（这样才能看到明显的延迟，DDR上的操作延迟太小）

```
perf sched record dd if=/dev/zero of=./testfile bs=4K count=1024
perf sched latency
```

Task	Runtime ms	Switches	Avg delay	Max delay	Max delay start	Max delay end	

dd:312	68.054 ms	2	0.210 ms	0.420 ms	629.787396 s	629.787816 s	
sshd:272	0.327 ms	1	0.000 ms	0.000 ms	0.000000 s	0.000000 s	
kworker/u4:0-ev:7	0.125 ms	1	0.000 ms	0.000 ms	0.000000 s	0.000000 s	
migration/1:14	0.000 ms	1	0.000 ms	0.000 ms	0.000000 s	0.000000 s	
migration/0:11	0.000 ms	1	0.000 ms	0.000 ms	0.000000 s	0.000000 s	

TOTAL:		68.691 ms		7			

可以看到，所有dd相关的进程的延迟信息都被展现出来了

```
perf sched map
```

```
# perf sched map
*A0      1021.655030 secs A0 => migration/0:11
*.       1021.655100 secs .  => swapper:0
*B0      1021.655570 secs B0 => perf:334
  B0 *.   1021.655880 secs
*C0      1021.658667 secs C0 => rcu_sched:10
```

*B0	.	1021.658719	secs	B0 => dd:334
*C0	.	1021.666567	secs	
*B0	.	1021.666601	secs	
*D0	.	1021.722001	secs	D0 => kworker/u4:2-ev:314
*E0	.	1021.722097	secs	E0 => sshd:322
*B0	.	1021.722419	secs	
*C0	.	1021.722575	secs	
*B0	.	1021.722612	secs	
B0	*F0	1021.722897	secs	F0 => perf:333
*	.	F0	1021.722914	secs
.	*	G0	1021.723142	secs
G0	=>	migration/1:14		
*F0	G0	1021.723200	secs	
F0	*	.	1021.723219	secs
F0	*	F0	1021.723393	secs

perf sched map展示了调度信息的一个总览，*表示发生调度，点表示该cpu处于idle

perf sched timehist

Samples do not have callchains.					
	time	cpu	task name	wait time	sch delay
			[tid/pid]	(msec)	(msec)
					run time
					(msec)
	-----	-----	-----	-----	-----
	1021.655030	[0000]	perf[333]	0.000	0.000
	1021.655100	[0000]	migration/0[11]	0.000	0.025
	1021.655570	[0000]	<idle>	0.000	0.000
	1021.655880	[0001]	perf[333]	0.000	0.000
	1021.658667	[0000]	dd[334]	0.000	0.032
	1021.658719	[0000]	rcu_sched[10]	0.000	0.116
	1021.666567	[0000]	dd[334]	0.052	0.000
	1021.666601	[0000]	rcu_sched[10]	7.848	0.049
	1021.722001	[0000]	dd[334]	0.033	0.000
	1021.722097	[0000]	kworker/u4:2-ev[314]	0.000	0.074
	1021.722419	[0000]	sshd[322]	0.000	0.059
	1021.722575	[0000]	dd[334]	0.418	0.000
	1021.722612	[0000]	rcu_sched[10]	55.974	0.035
	1021.722897	[0001]	<idle>	0.000	0.000
	1021.722914	[0000]	dd[334]	0.037	0.000
	1021.723142	[0001]	perf[333]	67.017	0.046
	1021.723200	[0000]	<idle>	67.344	0.000
	1021.723219	[0001]	migration/1[14]	0.000	0.026
	1021.723393	[0001]	<idle>	0.322	0.000
					0.173

timehist可以展示这一秒钟内，所有进程的调度情况，wait time表示从被唤醒到开始调度的时间，sch delay表示从开始调度到真正运行的时间。

perf top

perf top子命令动态地显示各种采样事件，以下是2个实例

hardware event之instructions

perf top -e instructions

10.87%	[kernel]	[k] format_decode
8.26%	[kernel]	[k] number
6.86%	[kernel]	[k] kallsyms_expand_symbol.constprop.8
5.32%	[kernel]	[k] vsnprintf
3.47%	[kernel]	[k] string_nocheck
2.95%	perf	[.] 0x000000000000948f0
2.78%	libc-2.29.so	[.] memcpy
2.48%	libc-2.29.so	[.] strcmp
2.38%	[kernel]	[k] update_iter
2.20%	libc-2.29.so	[.] strchr

2.10%	libc-2.29.so	[.] __libc_calloc
1.70%	libc-2.29.so	[.] memset
1.45%	perf	[.] 0x000000000000082b78
1.43%	libc-2.29.so	[.] strlen
1.29%	perf	[.] 0x000000000000082c06
1.20%	[kernel]	[k] __softirqentry_text_start
1.15%	[kernel]	[k] _raw_spin_unlock_irqrestore
1.10%	libc-2.29.so	[.] 0x00000000000006247e
1.09%	[kernel]	[k] seq_vprintf
0.93%	perf	[.] 0x000000000000082b68

tracepoint event之sched:sched_wakeup

```
perf top -e sched:sched_wakeup
```

32.60%	comm=perf pid=416 prio=120 target_cpu=001
30.39%	comm=perf pid=412 prio=120 target_cpu=001
17.40%	comm=kworker/0:1H pid=40 prio=100 target_cpu=000
3.31%	comm=ksoftirqd/0 pid=9 prio=120 target_cpu=000
3.04%	comm=kcompactd0 pid=26 prio=120 target_cpu=000
2.76%	comm=ntpd pid=105 prio=120 target_cpu=001
2.49%	comm=kcompactd0 pid=26 prio=120 target_cpu=001
2.49%	comm=kworker/0:1 pid=22 prio=120 target_cpu=000
2.49%	comm=kworker/1:0 pid=16 prio=120 target_cpu=001
2.49%	comm=perf pid=413 prio=120 target_cpu=000
0.28%	comm=haveged pid=93 prio=120 target_cpu=000
0.28%	comm=perf-top-UI pid=417 prio=120 target_cpu=000

如果跟上call-graph功能，perf top子命令将会展示更丰富的函数调用细节

```
perf top -e instructions --call-graph graph
```

16.69%	0.45%	[kernel]	[k] seq_read_iter
	--0.63%	--seq_read_iter	
	--0.98%	--s_next	
		--0.89%	--update_iter
	--0.64%	--s_show	
14.81%	0.18%	[kernel]	[k] s_show
14.25%	0.16%	[kernel]	[k] seq_printf
14.20%	0.48%	[kernel]	[k] seq_vprintf
	--4.51%	--seq_vprintf	
	--5.70%	--vsprintf	
		--4.11%	--pointer
			--3.56%
			--number
		--2.57%	--format_decode
		--2.36%	--string
			--2.10%
			--string_nocheck
	--1.42%	--format_decode	

perf bench

perf bench子命令是perf内部集成的一个benchmark测试程序，可以看一下perf bench支持哪些benchmark

```
perf bench

      sched: Scheduler and IPC benchmarks
      syscall: System call benchmarks
      mem: Memory access benchmarks
      futex: Futex stressing benchmarks
      epoll: Epoll stressing benchmarks
      internals: Perf-internals benchmarks
      all: All benchmarks
```

可见，benchmark有sched，syscall，mem等几大类

```
perf bench mem

      memcpy: Benchmark for memcpy() functions
      memset: Benchmark for memset() functions
      find_bit: Benchmark for find_bit() functions
      all: Run all memory access benchmarks
```

而mem里又包含了3个测试点

```
perf bench mem memcpy

# Running 'mem/memcpy' benchmark:
# function 'default' (Default memcpy() provided by glibc)
# Copying 1MB bytes ...

5.778476 GB/sec


perf bench mem memset

# Running 'mem/memset' benchmark:
# function 'default' (Default memset() provided by glibc)
# Copying 1MB bytes ...

6.300403 GB/sec
```

perf archive

如果想将采集到的perf.data交给其他人分析，此时仅把perf.data丢给他是不够的，这里就要用到perf archive子命令将perf.data以及相关符号信息打包后，再传给其他人，这样别人就也能分析这个perf.data了。

但目前该功能还有些问题，正在修复中。

perf evlist

perf evlist子命令可以分析出perf.data是基于什么事件采样的

```
perf evlist

kmem:kmalloc
kmem:kmalloc_node
kmem:kfree
```

```
kmem:kmem_cache_alloc
kmem:kmem_cache_alloc_node
kmem:kmem_cache_free
dummy:HG
```

可以看到，这个是一个基于kmem事件采样的perf.data

perf kallsyms

perf kallsyms子命令可以用于探测一个函数是否存在于内核符号表中，从而决定它是否可以作为一个perf probe的对象

```
perf kallsyms schedule
```

```
schedule: [kernel] [kernel.kallsyms] 0xffffffffe000abf44a-0xffffffffe000abf52c (0xffffffffe000abf44a-0xffffffffe000abf52c)
```

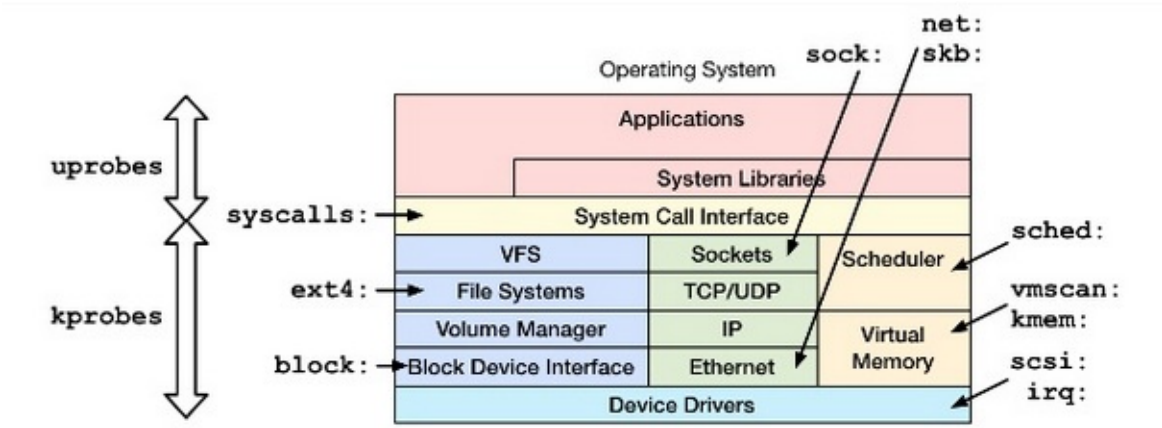
当然，还可以通过

```
cat /proc/kallsyms | grep schedule
```

来判断一个函数是不是一个在内核符号表中。

Kprobe & Uprobe - Linux Tracing

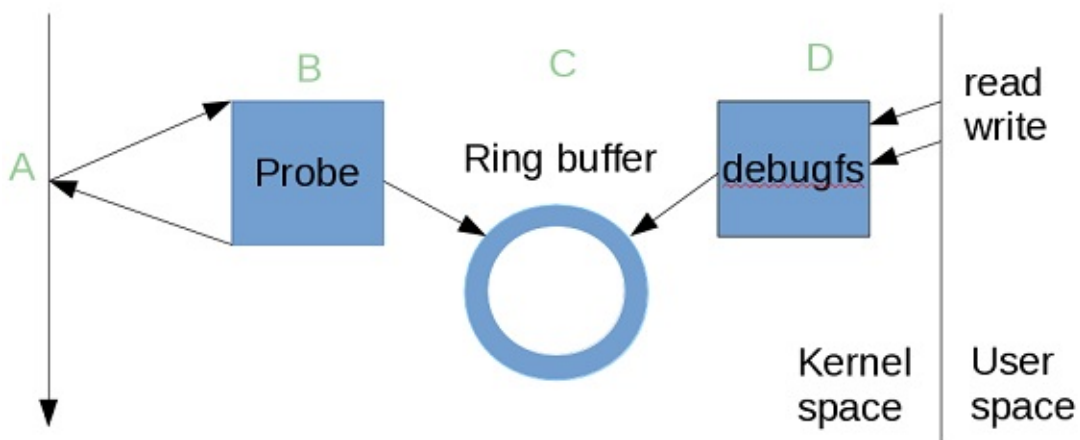
经过长期的发展, kprobes/uprobes 机制在事件(events)的基础上分别为内核态和用户态提供了追踪调试的功能, 这也构成了 tracepoint 机制的基础, 后期的很多工具, 比如 perf_events , ftrace 等都是在其基础上演化而来. 参考由 Brendan Gregg 提供的资料来看, kprobes/uprobes 在 Linux 动态追踪层面起到了基石的作用, 如下所示:



- kprobes: dynamic kernel tracing
 - function calls, returns, line numbers
- uprobes: dynamic user-level tracing

引用: https://blog.arstercz.com/introduction_to_linux_dynamic_tracing/

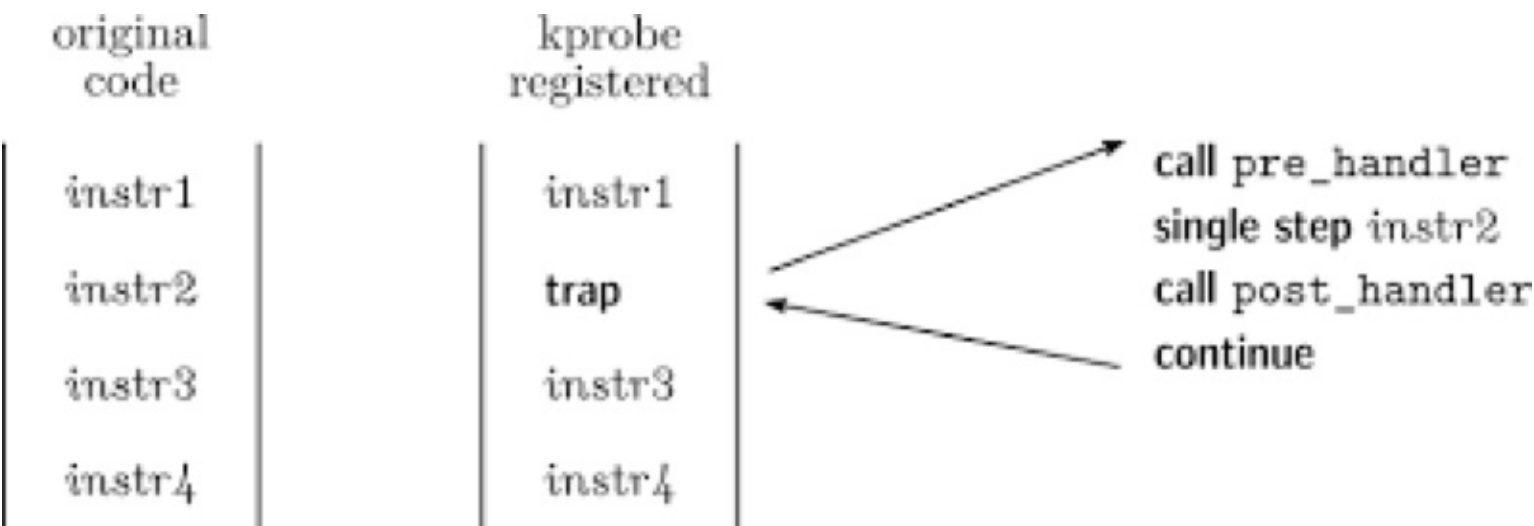
Kprobe 和 Uprobe 均可以通过 ftrace 的 /sys/debug/tracing interface (基于 debugfs 的用户空间层面的 API) 执行各种跟踪和分析，虽然 ftrace 的内部是复杂的, 不过输出的信息却以简单明了为主，更详细的使用示例可以参考 [ftrace-lwn-365835](#), 如下图所示, 大致为 ftrace 的原理:



Krprobe 介绍

kprobe是linux内核的一个重要特性，是其他的内核调试工具（perf, systemtap）的“基础设施”，同时内核 BPF 也依赖 kprobe

它利用指令桩原理，截获指令流，并在指令执行前后插入hook函数：



如果需要知道内核函数是否被调用、被调用上下文、入参以及返回值，比较简单的方法是加printk，但是效率低。

利用kprobe技术，用户可以自定义自己的回调函数，可以再几乎所有的函数中动态插入探测点。

当内核执行流程执行到指定的探测函数时，会调用该回调函数，用户即可收集所需的信息了，同时内核最后还会回到原本的正常执行流程。如果用户已经收集足够的信息，不再需要继续探测，则同样可以动态的移除探测点。

kprobes技术包括的2种探测手段分别时kprobe 和 kretprobe:

- kprobe是最基本的探测方式，是实现后两种的基础，它可以在任意的位置放置探测点（就连函数内部的某条指令处也可以），它提供了探测点的调用前、调用后和内存访问出错3种回调方式，分别是pre_handler、post_handler和fault_handler，其中pre_handler函数将在被探测指令被执行前回调，post_handler会在被探测指令执行完毕后回调（注意不是被探测函数），fault_handler会在内存访问出错时被调用。
- retprobe从名字种就可以看出其用途了，它同样基于kprobe实现，用于获取被探测函数的返回值。

基本使用指南

开启内核：

```
Symbol: FTRACE [=y]
Type   : boolean
Prompt: Tracers
Location:
(5) -> Kernel hacking
Defined at kernel/trace/Kconfig:132
Depends on: TRACING_SUPPORT [=y]

Symbol: KPROBE_EVENT [=y]
Type   : boolean
Prompt: Enable kprobes-based dynamic events
Location:
-> Kernel hacking
(1) -> Tracers (FTRACE [=y])
Defined at kernel/trace/Kconfig:405
Depends on: TRACING_SUPPORT [=y] && FTRACE [=y] && KPROBES [=y] && HAVE_REGS_AND_STACK_ACCESS_API [=y]
Selects: TRACING [=y] && PROBE_EVENTS [=y]

Symbol: HAVE_KPROBES_ON_FTRACE [=y]
Type   : boolean
Defined at arch/Kconfig:183
Selected by: csky [=y]

Symbol: KPROBES_ON_FTRACE [=y]
Type   : boolean
Defined at arch/Kconfig:79
Depends on: KPROBES [=y] && HAVE_KPROBES_ON_FTRACE [=y] && DYNAMIC_FTRACE_WITH_REGS [=y]
```

终端运行:

首先通过 mount 获得 ftrace debug 接口，然后通过 kprobe_evets 注册你需要 probe 的内核函数，在 tracing/events/kprobes// 下可以控制该 kprobe 函数的 开启和关闭

```
# mount -t debugfs nodev /sys/kernel/debug/

# echo 'p:myprobe _do_fork dfd=%a0 filename=%a1 flags=%a2 mode=+4($stack)' > /sys/kernel/debug/tracing/kprobe_events
# echo 'r:myretprobe _do_fork $retval' >> /sys/kernel/debug/tracing/kprobe_events

# echo 1 > /sys/kernel/debug/tracing/events/kprobes/myprobe/enable
# echo 1 > /sys/kernel/debug/tracing/events/kprobes/myretprobe/enable

# cat /sys/kernel/debug/tracing/trace
# tracer: nop
#
# entries-in-buffer/entries-written: 8/8 #P:1
#
# _-----=> irqs-off
# / _-----=> need-resched
# | / _---=> hardirq/softirq
# || / _--=> preempt-depth
# ||| / delay
# TASK-PID CPU#  ||||  TIMESTAMP  FUNCTION
# | | | | ||| | |
swapper/0-1 [000] dn.. 2.488544: Unknown type 599
swapper/0-1 [000] dn.. 2.489270: Unknown type 600
sh-121 [000] d... 408.113780: myprobe: (_do_fork+0x0/0x3ac) dfd=0xbe485f20 filename=0x0 flags=0x0 mode=0x0
sh-121 [000] d... 408.117058: myretprobe: (sys_clone+0xa6/0xac <- _do_fork) arg1=0x82
sh-121 [000] d... 409.816850: myprobe: (_do_fork+0x0/0x3ac) dfd=0xbe485f20 filename=0x0 flags=0x0 mode=0x0
sh-121 [000] dn.. 409.817539: myretprobe: (sys_clone+0xa6/0xac <- _do_fork) arg1=0x83
sh-121 [000] d... 411.202079: myprobe: (_do_fork+0x0/0x3ac) dfd=0xbe485f20 filename=0x0 flags=0x0 mode=0x0
sh-121 [000] d... 411.202750: myretprobe: (sys_clone+0xa6/0xac <- _do_fork) arg1=0x84
```

社区 kprobe 文档非常完善:

<https://www.kernel.org/doc/Documentation/kprobes.txt>

<https://lwn.net/Articles/410200/>

Trace-cmd 和 kernelshark:

Trace-cmd 是一个基于 ftrace 的用户态前端命令行工具，它的仓库在:

<git://git.kernel.org/pub/scm/linux/kernel/git/rostedt/trace-cmd.git>

很多发行版，都有 trace-cmd 的包，完善的 man 手册

具体使用参考:

<https://lwn.net/Articles/410200/>

Uprobe

ftrace 基础使用:

先准备一个应用程序，测试代码:

```
#include <stdio.h>
#include <unistd.h>

static void
print_curr_state_one(void)
{
    printf("This is the print current state one function\n");
}

static void
print_curr_state_two(void)
{
    printf("This is the print current state two function\n");
}

int main() {
    while(1) {
        print_curr_state_one();
        sleep(1);
        print_curr_state_two();
    }
}
```

编译并获取可执行文件和反汇编 (9 系列请使用 riscv-linux-gcc):

```
→ csky-linux-gcc test.c -o test
→ linux-next git:(linux-next-ftrace-kprobe-uprobe-simlutate-insn) x ../artifacts/output_860_next/images/host/bin/csky-linux-objdump -S test

00008518 :
    8518:      1422      subi      sp, sp, 8
    851a:      dd0e2000 st.w      r8, (sp, 0)
    851e:      ddee2001 st.w      r15, (sp, 0x4)
    8522:      6e3b      mov       r8, sp
    8524:      1006      lrw       r0, 0x8698      // 853c
    8526:      eae00007 jsri      0x0      // from address pool at 0x8540
    852a:      6c00      or        r0, r0
    852c:      6fa3      mov       sp, r8
    852e:      d9ee2001 ld.w      r15, (sp, 0x4)
    8532:      d90e2000 ld.w      r8, (sp, 0)
```

```
8536:      1402      addi      sp, sp, 8
8538:      783c      rts
853a:      0000      .short   0x0000
853c:      00008698  .long    0x00008698
8540:      00000000  .long    0x00000000
00008544 :
8544:      1422      subi      sp, sp, 8
8546:      dd0e2000  st.w      r8, (sp, 0)
854a:      ddee2001  st.w      r15, (sp, 0x4)
854e:      6e3b      mov       r8, sp
8550:      100d      lrw       r0, 0x86c8      // 8584
8552:      eae0000e  jsri      0x0      // from address pool at 0x8588
8556:      6c00      or        r0, r0
8558:      6fa3      mov       sp, r8
855a:      d9ee2001  ld.w      r15, (sp, 0x4)
855e:      d90e2000  ld.w      r8, (sp, 0)
8562:      1402      addi      sp, sp, 8
8564:      783c      rts
      ...
00008568
:
8568:      1422      subi      sp, sp, 8
856a:      dd0e2000  st.w      r8, (sp, 0)
856e:      ddee2001  st.w      r15, (sp, 0x4)
8572:      6e3b      mov       r8, sp
8574:      e3ffffd2  bsr       0x8518      // 8518
8578:      3001      movi      r0, 1
857a:      eae00006  jsri      0x0      // from address pool at 0x8590
857e:      e3ffffe3  bsr       0x8544      // 8544
8582:      07f9      br        0x8574      // 8574
```

→ linux-next git:(linux-next-ftrace-kprobe-uprobe-simlutate-insn) x ../artifacts/output_860_next/images/host/bin/csky-linux-readelf -S test
There are 28 section headers, starting at offset 0x1cd8:

Section Headers:

[Nr]	Name	Type	Addr	Off	Size	ES	Flg	Lk	Inf	Al
[0]		NULL	00000000	000000	000000	00		0	0	0
[1]	.interp	PROGBITS	00008134	000134	00000d	00	A	0	0	1
[2]	.note.ABI-tag	NOTE	00008144	000144	000020	00	A	0	0	4
[3]	.hash	HASH	00008164	000164	00003c	04	A	4	0	4
[4]	.dynsym	DYNSYM	000081a0	0001a0	0000a0	10	A	5	1	4
[5]	.dynstr	STRTAB	00008240	000240	000098	00	A	0	0	1
[6]	.gnu.version	VERSYM	000082d8	0002d8	000014	02	A	4	0	2
[7]	.gnu.version_r	VERNEED	000082ec	0002ec	000020	00	A	5	1	4
[8]	.rela.dyn	RELA	0000830c	00030c	00009c	0c	A	4	0	4
[9]	.init	PROGBITS	000083b0	0003b0	000030	00	AX	0	0	16
[10]	.text	PROGBITS	000083e0	0003e0	000282	00	AX	0	0	16

根据程序反汇编编插 uprobe 桩

```
echo 'p:enter_current_state_one /root/test:0x518 arg0=%a0 lr=%lr' >> \
/sys/kernel/debug/tracing/uprobe_events

echo 'r:exit_current_state_one /root/test:0x518 arg0=%a0' >> \
/sys/kernel/debug/tracing/uprobe_events

echo 'p:enter_current_state_two /root/test:0x544 arg0=%a0 lr=%lr' >> \
/sys/kernel/debug/tracing/uprobe_events

echo 'r:exit_current_state_two /root/test:0x544 arg0=%a0' >> \
/sys/kernel/debug/tracing/uprobe_events

echo 1 > /sys/kernel/debug/tracing/events/uprobes/enable

cat /sys/kernel/debug/tracing/trace
```

用户态任意位置都可以设置 uprobe 桩点

Perf probe 使用:

除了 ftrace，我们还可以使用 Perf probe -x 动态跟踪点是一种可观察性功能，可实现以下功能：甚至不需要重新编译就可以插装源代码的任意行。有了程序源代码的副本，跟踪点可以在运行时放置在任何地方，并且每个变量的值都可以转储时间执行通过跟踪点。这是一项非常强大的技术，用于检测其他人编写的复杂系统或代码。

```
perf probe -x /lib/libc.so.6 memcpy
perf record -e probe_libc:memcpy -aR ls
perf report
```

社区有大量 Uprobe 应用案例，和文档:

<http://www.brendangregg.com/blog/2015-06-28/linux-ftrace-uprobe.html>

<https://www.kernel.org/doc/Documentation/trace/uprobetracer.txt>

<https://www.kernel.org/doc/html/latest/trace/uprobetracer.html>

The Kernel Address Sanitizer (KASAN)

KernelAddressSANitizer (KASAN) 是动态内存错误检测器。它提供了一种快速而全面的解决方案，以查找无用后使用和越界错误。

KASAN使用编译时工具来检查每个内存访问，因此您将需要GCC 4.9.2或更高版本。需要GCC 5.0或更高版本才能检测对堆栈或全局变量的越界访问。

使用方法

在内核中开启 KASAN:

```
CONFIG_KASAN = y
```

并在CONFIG_KASAN_OUTLINE和CONFIG_KASAN_INLINE之间选择。 Outline和inline是编译器检测类型。 前者产生较小的二进制文件，而后者则快1.1-2倍。 内联检测需要GCC 5.0或更高版本。

KASAN可与SLUB和SLAB内存分配器一起使用。 为了更好地检测错误和更好地报告，请启用CONFIG_STACKTRACE。

要禁用特定文件或目录的检测，请在相应的内核Makefile中添加类似于以下内容的行：

- 对于单个文件 (譬如: main.o)

```
KASAN_SANITIZE_main.o := n
```

- 对于一个目录中的所有文件

```
KASAN_SANITIZE := n
```

出错报告

典型的越界访问报告如下所示：

```
=====
BUG: AddressSanitizer: out of bounds access in kmalloc_oob_right+0x65/0x75 [test_kasan] at addr ffff8800693bc5d3
Write of size 1 by task modprobe/1689
=====
BUG kmalloc-128 (Not tainted): kasan error
-----

Disabling lock debugging due to kernel taint
INFO: Allocated in kmalloc_oob_right+0x3d/0x75 [test_kasan] age=0 cpu=0 pid=1689
__slab_alloc+0x4b4/0x4f0
kmem_cache_alloc_trace+0x10b/0x190
kmalloc_oob_right+0x3d/0x75 [test_kasan]
init_module+0x9/0x47 [test_kasan]
do_one_initcall+0x99/0x200
load_module+0x2cb3/0x3b20
SyS_finit_module+0x76/0x80
system_call_fastpath+0x12/0x17
INFO: Slab 0xfffffea0001a4ef00 objects=17 used=7 fp=0xfffff8800693bd728 flags=0x1000000000004080
INFO: Object 0xfffff8800693bc558 @offset=1368 fp=0xfffff8800693bc720

Bytes b4 ffff8800693bc548: 00 00 00 00 00 00 00 00 5a 5a 5a 5a 5a 5a 5a 5a  ....ZZZZZZZ
Object ffff8800693bc558: 6b 6b 6b 6b 6b 6b 6b 6b 6b 6b 6b 6b 6b 6b 6b 6b  kkkkkkkkkkkkkkkk
Object ffff8800693bc568: 6b 6b 6b 6b 6b 6b 6b 6b 6b 6b 6b 6b 6b 6b 6b 6b  kkkkkkkkkkkkkkkk
Object ffff8800693bc578: 6b 6b 6b 6b 6b 6b 6b 6b 6b 6b 6b 6b 6b 6b 6b 6b  kkkkkkkkkkkkkkkk
Object ffff8800693bc588: 6b 6b 6b 6b 6b 6b 6b 6b 6b 6b 6b 6b 6b 6b 6b 6b  kkkkkkkkkkkkkkkk
Object ffff8800693bc598: 6b 6b 6b 6b 6b 6b 6b 6b 6b 6b 6b 6b 6b 6b 6b 6b  kkkkkkkkkkkkkkkk
Object ffff8800693bc5a8: 6b 6b 6b 6b 6b 6b 6b 6b 6b 6b 6b 6b 6b 6b 6b 6b  kkkkkkkkkkkkkkkk
Object ffff8800693bc5b8: 6b 6b 6b 6b 6b 6b 6b 6b 6b 6b 6b 6b 6b 6b 6b 6b  kkkkkkkkkkkkkkkk
Object ffff8800693bc5c8: 6b 6b 6b 6b 6b 6b 6b 6b 6b 6b 6b 6b 6b 6b a5  kkkkkkkkkkkkkkkk.
```

```
Redzone ffff8800693bc5d8: cc cc cc cc cc cc cc cc          .....
Padding ffff8800693bc718: 5a 5a 5a 5a 5a 5a 5a 5a          ZZZZZZZZ
CPU: 0 PID: 1689 Comm: modprobe Tainted: G    B            3.18.0-rc1-mm1+ #98
 ffff8800693bc000 0000000000000000 ffff8800693bc558 ffff88006923bb78
 ffffffff81cc68ae 00000000000000f3 ffff88006d407600 ffff88006923bba8
 ffffffff811fd848 ffff88006d407600 ffffea0001a4ef00 ffff8800693bc558
Call Trace:
[<ffffffff81cc68ae>] dump_stack+0x46/0x58
[<ffffffff811fd848>] print_trailer+0xf8/0x160
[<ffffffffffa00026a7>] ? kmem_cache_oob+0xc3/0xc3 [test_kasan]
[<ffffffff811ff0f5>] object_err+0x35/0x40
[<ffffffffffa0002065>] ? kmalloc_oob_right+0x65/0x75 [test_kasan]
[<ffffffff8120b9fa>] kasan_report_error+0x38a/0x3f0
[<ffffffff8120a79f>] ? kasan_poison_shadow+0x2f/0x40
[<ffffffff8120b344>] ? kasan_unpoison_shadow+0x14/0x40
[<ffffffff8120a79f>] ? kasan_poison_shadow+0x2f/0x40
[<ffffffffffa00026a7>] ? kmem_cache_oob+0xc3/0xc3 [test_kasan]
[<ffffffff8120a995>] __asan_store1+0x75/0xb0
[<ffffffffffa0002601>] ? kmem_cache_oob+0x1d/0xc3 [test_kasan]
[<ffffffffffa0002065>] ? kmalloc_oob_right+0x65/0x75 [test_kasan]
[<ffffffffffa0002065>] kmalloc_oob_right+0x65/0x75 [test_kasan]
[<ffffffffffa00026b0>] init_module+0x9/0x47 [test_kasan]
[<ffffffff810002d9>] do_one_initcall+0x99/0x200
[<ffffffff811e4e5c>] ? __vunmap+0xec/0x160
[<ffffffff81114f63>] load_module+0x2cb3/0x3b20
[<ffffffff8110fd70>] ? m_show+0x240/0x240
[<ffffffff81115f06>] SyS_finit_module+0x76/0x80
[<ffffffff81cd3129>] system_call_fastpath+0x12/0x17
Memory state around the buggy address:
 ffff8800693bc300: fc fc fc fc fc fc fc fc fc fc fc fc fc fc fc
 ffff8800693bc380: fc fc 00 00 00 00 00 00 00 00 00 00 00 00 00 fc
 ffff8800693bc400: fc fc fc fc fc fc fc fc fc fc fc fc fc fc fc
 ffff8800693bc480: fc fc fc fc fc fc fc fc fc fc fc fc fc fc fc
 ffff8800693bc500: fc fc fc fc fc fc fc fc fc fc fc fc 00 00 00 00
>ffff8800693bc580: 00 00 00 00 00 00 00 00 00 00 03 fc fc fc fc fc
                        ^
 ffff8800693bc600: fc fc fc fc fc fc fc fc fc fc fc fc fc fc fc
 ffff8800693bc680: fc fc fc fc fc fc fc fc fc fc fc fc fc fc fc
 ffff8800693bc700: fc fc fc fc fb fb fb fb fb fb fb fb fb fb
 ffff8800693bc780: fb fb fb fb fb fb fb fb fb fb fb fb fb fb fb
 ffff8800693bc800: fb fb fb fb fb fb fb fb fb fb fb fb fb fb fb
=====
```

该报告的标题描述了发生了哪种错误以及导致这种错误的访问方式。后面是对访问的slub对象的描述（有关详细信息，请参见 Documentation/vm/slub.txt 中的“SLUB调试输出”部分）以及对访问的内存页面的描述。

在最后一部分中，报告显示访问地址周围的内存状态。阅读此部分需要对KASAN的工作方式有一些了解。

存储器的每8个对齐字节的状态被编码在一个影子字节中。这8个字节可以访问，部分访问，释放或为redzone。对于每个影子字节，我们使用以下编码：0表示相应存储区的所有8个字节均可访问；N（1 <= N <= 7）表示前N个字节是可访问的，而其他（8-N）个字节则不可访问；任何负值表示整个8字节字不可访问。我们使用不同的负值来区分不同类型的不可访问的内存，例如Redzone或释放的内存（请参见mm / kasan / kasan.h）。

在报告上方的箭头指向影子字节03，这意味着被访问的地址可以部分访问。

本文参考：<https://www.kernel.org/doc/html/v4.14/dev-tools/kasan.html>

KGDB 内核调试指南

内核有两个不同的调试器前端（kdb和kgdb），它们连接到调试核心。如果在编译和运行时正确配置了内核，则可以使用两个调试器前端中的任何一个并在它们之间动态转换。

Kdb是简单化的shell风格的界面，可以在带有键盘或串行控制台的系统控制台上使用。您可以使用它来检查内存，寄存器，进程列表，dmesg，甚至设置断点以停在特定位置。尽管您可以设置断点并执行一些基本的内核运行控制，但Kdb并不是源代码级调试器。Kdb主要旨在进行一些分析，以帮助开发或诊断内核问题。如果代码是使用CONFIG_KALLSYMS构建的，则可以按名称访问内核内置文件或内核模块中的某些符号。

Kgdb旨在用作Linux内核的源代码级调试器。它与gdb一起用于调试Linux内核。期望gdb可用于“侵入”内核以检查内存，变量并查看调用堆栈信息，类似于应用程序开发人员使用gdb调试应用程序的方式。可以在内核代码中放置断点并执行一些有限的执行步骤。

使用kgdb需要两台机器。其中一台机器是开发机器，另一台是目标机器。要调试的内核在目标计算机上运行。开发机器针对包含符号的vmlinux文件运行gdb实例（不是启动映像，如bzImage，zImage，uImage...）。在gdb中，开发人员指定连接参数并连接到kgdb。开发人员与gdb建立的连接类型取决于在测试计算机内核中编译为内置模块或可加载内核模块的kgdb I / O模块的可用性。

编译内核

在 menuconfig 中开启 CONFIG_KGDB

```
Symbol: KGDB [=y]
Type   : bool
Defined at lib/Kconfig.kgdb:11
  Prompt: KGDB: kernel debugger
  Depends on: HAVE_ARCH_KGDB [=y] && DEBUG_KERNEL [=y]
  Location:
    -> Kernel hacking
(1)    -> Generic Kernel Debugging Instruments
```

LOCKDEP

简介

验证程序操作的基本对象是锁的类型。

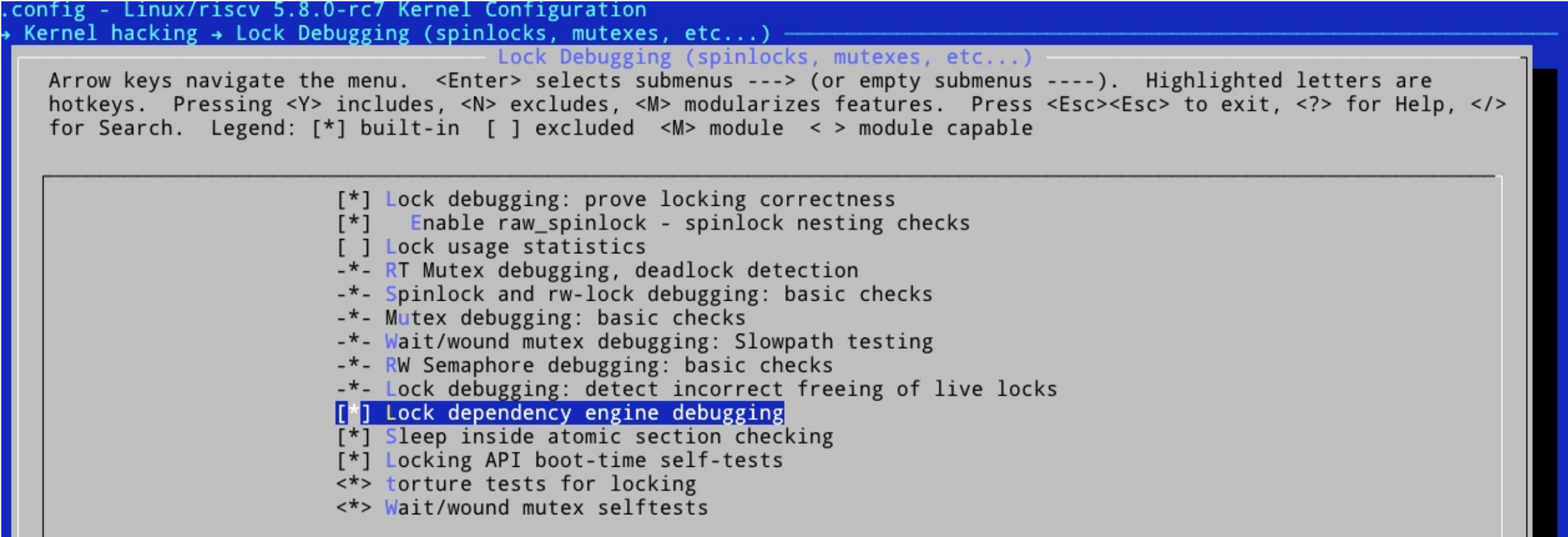
一类锁是一组在锁规则上逻辑上相同的锁，即使锁可能具有多个（可能是数万个）实例化。 例如，inode结构中的锁是一个类，而每个inode具有该锁类的自己的实例化。

验证器跟踪锁类的“状态”，并跟踪不同锁类之间的依赖关系。 验证器保持状态和依赖关系正确的滚动证明。

锁实例化不同，锁类本身永远不会消失：在启动后首次使用锁类时，它将被注册，并且该锁类的所有后续使用都将附加到该锁类上

使用介绍

在 Kernel hacking -> Lock Debugging 中开启 LOCK_DEP 测试，(注意不要开启 LOCK_STAT)



运行后，会启动自动测试

```
[ 0.000424] Lock dependency validator: Copyright (c) 2006 Red Har
[ 0.000443] ... MAX_LOCKDEP_SUBCLASSES: 8
[ 0.000460] ... MAX_LOCK_DEPTH: 48
[ 0.000476] ... MAX_LOCKDEP_KEYS: 8192
[ 0.000492] ... CLASSHASH_SIZE: 4096
[ 0.000508] ... MAX_LOCKDEP_ENTRIES: 32768
[ 0.000524] ... MAX_LOCKDEP_CHAINS: 65536
[ 0.000540] ... CHAINHASH_SIZE: 32768
[ 0.000556] memory used by lock dependency info: 6301 kB
[ 0.000572] memory used for stack traces: 4224 kB
[ 0.000588] per task-struct memory footprint: 1920 bytes
[ 0.000604] -----
[ 0.000620] | Locking API testsuite:
[ 0.000636] -----
[ 0.000651] | spin |wlock |rloc|
[ 0.000667] -----
[ 0.000698] A-A deadlock: ok | ok | ok|
[ 0.009065] A-B-B-A deadlock: ok | ok | ok|
[ 0.018270] A-B-B-C-C-A deadlock: ok | ok | ok|
[ 0.028441] A-B-C-A-B-C deadlock: ok | ok | ok|
[ 0.038546] A-B-B-C-C-D-D-A deadlock: ok | ok | ok|
[ 0.049592] A-B-C-D-B-D-D-A deadlock: ok | ok | ok|
[ 0.060594] A-B-C-D-B-C-D-A deadlock: ok | ok | ok|
[ 0.071571] double unlock: ok | ok | ok|
[ 0.079825] initialize held: ok | ok | ok|
[ 0.087588] -----
[ 0.087606] recursive read-lock: | ok|
[ 0.090094] recursive read-lock #2: | ok|
[ 0.092472] mixed read-write-lock: | ok|
[ 0.094807] mixed write-read-lock: | ok|
[ 0.097203] mixed read-lock/lock-write ABBA: | ok|
```

[	0.099780]	mixed read-lock/lock-read ABBA:				FAIL
[	0.102444]	mixed write-lock/lock-write ABBA:				ok
[	0.105050]	-----				
[	0.105079]	hard-irqs-on + irq-safe-A/12:	ok		ok	ok
[	0.108638]	soft-irqs-on + irq-safe-A/12:	ok		ok	ok
[	0.112183]	hard-irqs-on + irq-safe-A/21:	ok		ok	ok
[	0.115749]	soft-irqs-on + irq-safe-A/21:	ok		ok	ok
[	0.119282]	sirq-safe-A => hirqs-on/12:	ok		ok	ok
[	0.122826]	sirq-safe-A => hirqs-on/21:	ok		ok	ok
[	0.126384]	hard-safe-A + irq-s-on/12:	ok		ok	ok
[	0.129957]	soft-safe-A + irq-s-on/12:	ok		ok	ok
[	0.133488]	hard-safe-A + irq-s-on/21:	ok		ok	ok
[	0.137056]	soft-safe-A + irq-s-on/21:	ok		ok	ok
[	0.140635]	hard-safe-A + unsafe-B #1/123:	ok		ok	ok
[	0.144589]	soft-safe-A + unsafe-B #1/123:	ok		ok	ok
[	0.148522]	hard-safe-A + unsafe-B #1/132:	ok		ok	ok
[	0.152483]	soft-safe-A + unsafe-B #1/132:	ok		ok	ok
[	0.156465]	hard-safe-A + unsafe-B #1/213:	ok		ok	ok
[	0.160438]	soft-safe-A + unsafe-B #1/213:	ok		ok	ok
[	0.164415]	hard-safe-A + unsafe-B #1/231:	ok		ok	ok
[	0.168325]	soft-safe-A + unsafe-B #1/231:	ok		ok	ok

Linux 系统性能分析工具集

Linux 系统中提供众多的性能分析工具，常见的工具如下表：

软件包名	简介
latencytop	是一个监测工具，用于报告系统和应用程序中与等待时间相关的统计信息。
memstat	分析了应用层内存使用的情况，并从匿名页和文件页两个角度给出了详尽的数据。
vmtouch	是一个控制文件系统page cache的小工具，可以将文件纳入page cache中，也可以驱逐出去。
fio	是一个测试IO性能的工具，从各个方面测试了IO的性能数据。
blktrace	是一个用于跟踪Block Layer函数调用以及耗费时间的工具。
iostat	是一个查看当前各block设备IO状态的工具。
iozone	可以对block设备上的文件系统进行测试。
lmbench	是一个全面测试系统性能的工具。
nmon	是一个综合性的系统性能观测工具。
iperf3	是一个用于测量网络性能的工具。
pv	是一个 用于显示命令在执行时的进度条的工具。
dieharder	是一个用于测试随机数生成能力的工具。

上述工具都可以通过apt install的方式安装，详情请见 [包管理](#)

如果apt install安装失败，可以尝试apt update后，再apt install

latencytop

LatencyTOP 是一个监测工具，用于报告系统和应用程序中与等待时间相关的统计信息。此工具报告系统中以及系统上正在运行的应用程序中等待时间的发生位置和类型的相关统计信息。可以根据该统计信息来消除所查明的等待时间，从而提高应用程序和系统的性能吞吐量。

此工具定期分析系统活动，并在输出窗口中显示数据。将跟踪两种类型的等待时间：进入休眠状态和从休眠状态唤醒的 LWP，以及调整顺序以获取同步对象的 LWP。此工具使用 Solaris DTrace 框架收集与系统和应用程序 LWP 的这两种非活动情况相对应的统计信息。

输出窗口分成两部分。上部分显示系统范围的统计信息，而下部分显示有关各个进程的统计信息。用户可使用向左方向键和向右方向键在进程列表中导航，并选择感兴趣的列表。然后此工具会在窗口下部分中显示该选定进程的相关统计信息。如果按下 t 或 T 键，此工具将显示该选定进程的特定于 LWP 的视图。可使用 t 或 T 键在进程视图和线程视图之间进行切换。

执行期间，用户可通过按下 r 或 R 键强制刷新分析结果。间隔时间将恢复为缺省值或指定值（如果使用 -t）。要退出应用程序，用户必须按下 q 或 Q 键。

内核需要开启 **CONFIG_LATENCYTOP**

```
apt install -y latencytop
```

latencytop 可以观察一定时间内，每一个进程在运行过程中发生的调度的次数和原因分布

LatencyTOP version 0.5 (C) 2008 Intel Corporation		
Cause	Maximum	Percentage
Waiting for a process to die	3.8 msec	29.4 %
Waiting for event (select)	3.7 msec	53.6 %
Waiting for TTY to finish sending	1.3 msec	16.0 %
[__schedule]	0.2 msec	0.5 %
Process latencytop (1007)	Total: 9.1 msec	
Waiting for a process to die	3.8 msec	42.3 %
Waiting for TTY to finish sending	1.3 msec	55.7 %
[__schedule]	0.2 msec	1.9 %
kworker/1:1-mm_percpu_wq kworker/0:1H-mmc_complete haveged ntpd latencytop		

上面一部分是内核态的，下面一部分对应的是每一个进程


```
LatencyTOP version 0.5      (C) 2008 Intel Corporation

Cause                      Maximum      Percentage
waiting for a process to die 3.9 msec    33.4 %
waiting for event (select)   3.8 msec    48.9 %
waiting for TTY to finish sending 1.3 msec    16.7 %
[__schedule]                 0.1 msec    0.5 %


Process latencytop (1132)   Total: 9.2 msec
waiting for a process to die 3.9 msec    42.9 %
waiting for TTY to finish sending 1.3 msec    55.6 %
[__schedule]                0.1 msec     1.5 %


kworker/0:1H-mmc_complete  kworker/1:1H-kblockd  haveged  ntpd  latencytop
```

高亮的latencytop表示latencytop这个程序自身在运行过程中发生的进程切换次数和原因分布

memstat

memstat是一个用于观测系统内存使用状况的工具

安装

```
apt install -y memstat
```

memstat通过遍历/proc下所有进程，然后解析内存使用情况。通过对比可以发现，memstat不会统计内核线程的内存使用情况，同时memstat也不会统计所有设备使用的内存。memstat 详尽地分析了应用层内存使用的情况，并从匿名页和文件页两个角度给出了详尽的数据。memstat使用：

```
memstat - 显示整个系统内存使用情况。
memstat -p pid - 显示进程pid的内存使用情况。
```

显示示例如下：

```
# memstat
288k: PID 1 (/bin/busybox)
288k: PID 69 (/bin/busybox)
288k: PID 73 (/bin/busybox)
1752k: PID 78 (/bin/busybox)
5988k: PID 94 (/usr/sbin/haveged)
66536k: PID 106 (/usr/sbin/ntpd)
572k: PID 111 (/usr/sbin/sshd)
288k: PID 116 (/bin/busybox)
672k: PID 117 (/bin/bash)
300k: PID 894 (/usr/bin/memstat)
536k( 516k): /bin/bash 117
1284k( 1224k): /bin/busybox 1 69 73 78 116 1 69 73 78 116
144k( 92k): /lib/ld-2.29.so 78 94 106 111 117 894 78 94 106 111 117 ...
40k( 16k): /lib/libatomic.so.1.2.0 106 111 106 111 106 111
1196k( 1028k): /lib/libc-2.29.so 78 94 106 111 117 894 78 94 106 111 11...
36k( 24k): /lib/libcrypt-2.29.so 111
32k( 8k): /lib/libdl-2.29.so 106 111 117 106 111 117 106 111 117
76k( 68k): /lib/libgcc_s.so.1 106
528k( 520k): /lib/libm-2.29.so 106
24k( 16k): /lib/libnss_dns-2.29.so 106
64k( 32k): /lib/libnss_files-2.29.so 78 106 111 117 78 106 111 117 ...
96k( 80k): /lib/libpthread-2.29.so 106 111 106 111 106 111
76k( 52k): /lib/libresolv-2.29.so 106 111 106 111 106 111
16k( 8k): /lib/libutil-2.29.so 111
16k( 8k): /usr/bin/memstat 894
1868k( 1516k): /usr/lib/libcrypto.so.1.1 106 111 106 111 106 111
92k( 84k): /usr/lib/libhavege.so.1.1.0 94
32k( 24k): /usr/lib/libhistory.so.8.0 117
264k( 240k): /usr/lib/libncursesw.so.6.1 117
236k( 200k): /usr/lib/libreadline.so.8.0 117
456k( 344k): /usr/lib/libssl.so.1.1 106 111 106 111 106 111
68k( 60k): /usr/lib/libz.so.1.2.11 111
24k( 16k): /usr/sbin/haveged 94
572k( 536k): /usr/sbin/ntpd 106
```



```
520k( 504k): /usr/sbin/sshd 111
```

vmtouch

vmtouch 是一个可以操纵page cache的工具，它可以查看一个文件占用了多少 page cache，可以让一个文件完全浸泡在 page cache 里，也可以让一个文件完全从 page cache 中剥离出来

安装

```
apt install -y vmtouch
```

```
# vmtouch uImage
    Files: 1
  Directories: 0
Resident Pages: 0/3433 0/13M 0%
    Elapsed: 0.000499 seconds

# free -m
      total        used        free      shared  buff/cache   available
Mem:      3448          67       3371          0           9       3368
Swap:          0           0           0
```

uImage 文件存放在 eMMC 上，通过 vmtouch 工具可见，uImage 大小为3433个pages，约13M，此时并没有占用 page cache

同时可以看到，此时的 free 内存为3371M

然后使用 vmtouch 将 uImage 完全纳入 page cache

```
# vmtouch -t uImage
    Files: 1
  Directories: 0
Touched Pages: 3433 (13M)
    Elapsed: 0.61707 seconds

# free -m
      total        used        free      shared  buff/cache   available
Mem:      3448          67       3358          0          22       3363
Swap:          0           0           0

# vmtouch uImage
    Files: 1
  Directories: 0
Resident Pages: 3433/3433 13M/13M 100%
    Elapsed: 0.001754 seconds
```

可以看到，uImage 已经完全浸泡在 page cache 中了，并且 free 内存也恰好少了13M

使用

```
# vmtouch -e uImage
    Files: 1
  Directories: 0
Evicted Pages: 3433 (13M)
    Elapsed: 0.004634 seconds

# vmtouch uImage
    Files: 1
  Directories: 0
Resident Pages: 0/3433 0/13M 0%
    Elapsed: 0.000446 seconds
```

可以将 uImage 从 page cache 中打捞出来

fio

fio是一个用于测量文件系统挂载有文件系统的block设备的性能的工具

安装

```
apt install -y fio
```

```
fio -filename=/dev/emcpowerb \  
-iodepth 16 -thread -rw=randread \  
-ioengine=psync -bs=4k -size=1G \  
-numjobs=1 -runtime=180 \  
-group_reporting -name=test1
```

输出：

```
test1: (g=0): rw=randread, bs=(R) 4096B-4096B, (W) 4096B-4096B, (T) 4096B-4096B, ioengine=psync, iodepth=16  
fio-3.9  
Starting 1 thread  
Jobs: 1 (f=1): [r(1)][100.0%][r=330MiB/s][r=84.4k IOPS][eta 00m:00s]  
test1: (groupid=0, jobs=1): err= 0: pid=167: Thu Jan  1 10:31:36 1970  
  read: IOPS=84.9k, BW=332MiB/s (348MB/s)(1024MiB/3087msec)  
    clat (usec): min=4, max=329, avg= 6.61, stdev= 3.91  
      lat (usec): min=5, max=330, avg= 7.58, stdev= 4.26  
    clat percentiles (usec):  
      |  1.00th=[   6],  5.00th=[   6], 10.00th=[   6], 20.00th=[   7],  
      | 30.00th=[   7], 40.00th=[   7], 50.00th=[   7], 60.00th=[   7],  
      | 70.00th=[   7], 80.00th=[   7], 90.00th=[   8], 95.00th=[   8],  
      | 99.00th=[   8], 99.50th=[   9], 99.90th=[  70], 99.95th=[ 122],  
      | 99.99th=[ 151]  
    bw (  KiB/s): min=334910, max=340294, per=99.46%, avg=337844.33, stdev=2086.47, samples=6  
    iops        : min=83727, max=85073, avg=84460.50, stdev=521.51, samples=6  
  lat (usec)    : 10=99.57%, 20=0.25%, 50=0.02%, 100=0.10%, 250=0.06%  
  lat (usec)    : 500=0.01%  
  cpu           : usr=29.16%, sys=69.51%, ctx=772, majf=0, minf=1  
  IO depths     : 1=100.0%, 2=0.0%, 4=0.0%, 8=0.0%, 16=0.0%, 32=0.0%, >=64=0.0%  
    submit      : 0=0.0%, 4=100.0%, 8=0.0%, 16=0.0%, 32=0.0%, 64=0.0%, >=64=0.0%  
    complete    : 0=0.0%, 4=100.0%, 8=0.0%, 16=0.0%, 32=0.0%, 64=0.0%, >=64=0.0%  
    issued rwts: total=262144,0,0,0 short=0,0,0,0 dropped=0,0,0,0  
    latency     : target=0, window=0, percentile=100.00%, depth=16  
  
Run status group 0 (all jobs):  
  READ: bw=332MiB/s (348MB/s), 332MiB/s-332MiB/s (348MB/s-348MB/s), io=1024MiB (1074MB), run=3087-3087msec
```

可以看到，平均带宽是 332MB/s，平均延迟在 10us，更多参数的设置和解读参见：<https://tobert.github.io/post/2014-04-17-fio-output-explained.html>

blktrace

内核需要开启CONFIG_BLK_DEV_IO_TRACE

blktrace是一个用于观测和跟踪系统IO状况的工具

安装

```
apt install -y blktrace
```

```
# dd if=/dev/mmcb1k0p3 of=/dev/zero bs=512K count=1024000 &  
# blktrace /dev/mmcb1k0p3 -o - | blkparse -i -  
179,3    0        1      0.0000000000      9  C  RA 1164800 + 1536 [0]  
179,0    0        2      0.000552333      136 A  RA 1166336 + 512 <- (179,3) 1039360  
179,3    0        3      0.000554333      136 Q  RA 1166336 + 512 [dd]  
179,3    0        4      0.000570000      136 G  RA 1166336 + 512 [dd]  
179,3    0        5      0.000574000      136 P   N [dd]  
179,3    0        6      0.000578000      136 U   N [dd] 1  
179,3    0        7      0.000581333      136 I  RA 1166336 + 512 [dd]
```

```
179,3      0      8      0.000627000      45  D  RA 1166336 + 512 [kworker/0:2H]
179,0      0      9      0.001208000      136 A  RA 1166848 + 1024 <- (179,3) 1039872
179,3      0     10      0.001209000      136 Q  RA 1166848 + 1024 [dd]
179,3      0     11      0.001217333      136 G  RA 1166848 + 1024 [dd]
179,3      0     12      0.001220333      136 P   N [dd]
... ..
179,3      0    103      0.171033333      45  D  RA 1174528 + 512 [kworker/0:2H]
CPU0 (179,3):
  Reads Queued:      13,      4352KiB  Writes Queued:      0,      0KiB
  Read Dispatches:   13,      4352KiB  Write Dispatches:   0,      0KiB
  Reads Requeued:     0                Writes Requeued:    0
  Reads Completed:   12,      4608KiB  Writes Completed:   0,      0KiB
  Read Merges:        0,      0KiB    Write Merges:        0,      0KiB
  Read depth:         2                Write depth:         0
  IO unplugs:         13                Timer unplugs:      0

Throughput (R/W): 26947KiB/s / 0KiB/s
Events (179,3): 103 entries
Skips: 0 forward (0 - 0.0%)
```

其中第六个字段非常有用：每一个字母都代表了 IO 请求所经历的某个阶段

- Q – 即将生成 IO 请求
- G – IO 请求生成
- I – IO 请求进入 IO Scheduler 队列
- D – IO 请求进入 driver
- C – IO 请求执行完毕

而第四个字段表示从走到这一个阶段的时间戳，通过这一信息，可以详尽地分析一个 IO 完整的流程，并分析出瓶颈在哪里

最后，还可以通过 btt 来绘制流程图，这里就不表了

iotstat

内核需要开启 **CONFIG_TASK_IO_ACCOUNTING**

iotstat是一个用于观测当前系统IO速度和负载的工具

安装

```
apt install -y iostat

# iostat -h
iostat v2.2, (C) 1999-2005 by Greg Franks, Zlatko Calusic, Rick Lindsley, Arnaud Desitter
Distributed under the terms of the GPL (see LICENSE file)
Usage: iostat [-cdDpPxh] [disks...] [interval [count]]
options:

    c - print cpu usage info
    d - print basic disk info
    D - print disk utilization info
    p - print partition info also
    P - print partition info only
    x - print extended disk info
    h - this help

# iostat -cdDP
mmcb1k0      mmcb1k0p3      cpu
r/s  w/s  %b   r/s  w/s  %b   us  sy  wt  id
  5    0   2    5    0   2    0   0   7  92
```

iozone

iozone是一个测量block设备上文件系统性能的测试工具

安装

```
apt install -y iozone
```

iozone -a 可以给 ICE 上基于 eMMC 的文件系统进行一个全面测试

```
# iozone -a
Iozone: Performance Test of File I/O
Version $Revision: 3.483 $
Compiled for 64 bit mode.
Build: linux

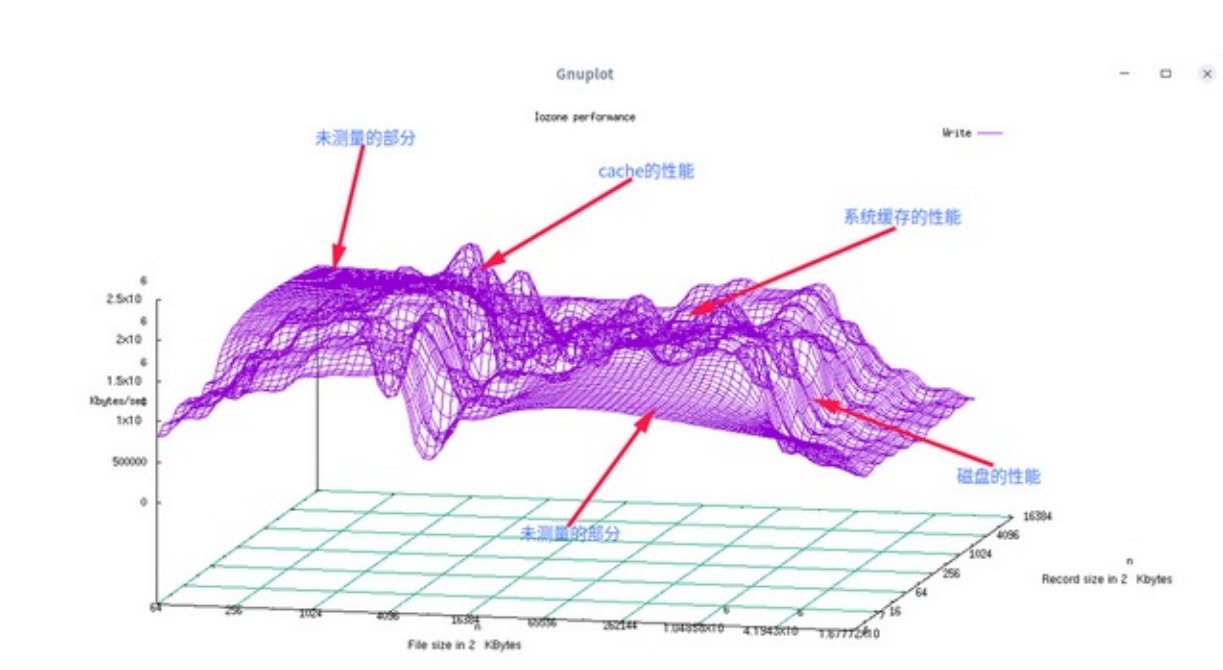
Contributors:William Norcott, Don Capps, Isom Crawford, Kirby Collins
Al Slater, Scott Rhine, Mike Wisner, Ken Goss
Steve Landherr, Brad Smith, Mark Kelly, Dr. Alain Cyr,
Randy Dunlap, Mark Montague, Dan Million, Gavin Brebner,
Jean-Marc Zucconi, Jeff Blomberg, Benny Halevy, Dave Boone,
Erik Habbinga, Kris Strecker, Walter Wong, Joshua Root,
Fabrice Bacchella, Zhenghua Xue, Qin Li, Darren Sawyer,
Vangel Bojaxhi, Ben England, Vikentsi Lapa,
Alexey Skidanov, Sudhir Kumar.

Run began: Thu Jan 1 09:39:29 1970

Auto Mode
Command line used: iozone -a
Output is in kbytes/sec
Time Resolution = 0.000001 seconds.
Processor cache size set to 1024 kbytes.
Processor cache line size set to 32 bytes.
File stride size set to 17 * record size.

      kB  reclen   write  rewrite   read   reread   random   random   bkwd   record   stride
      64     4  158024  566371  955223  1122807  1066666  537815  914285  646464  1103448  516129  561403  1066666  1306122
      64     8  202531  659793  1488372  1422222  1488372  640000  1163636  810126  1523809  659793  673684  1391304  1599999
      64    16  213333  688172  1641025  1560975  1684210  673684  1391304  853333  1729729  771084  810126  1684210  1882352
      64    32  217687  719101  1729729  1641025  1882352  703296  1523809  842105  1882352  1015873  1207547  1828571  2064516
      64    64  219178  752941  1600000  2285714  2133333  744186  1641025  761904  1999999  771084  771084  1999999  2206896
     128     4  197836  606635  1174311  1174311  1163636  522448  1024000  707182  1103448  551724  547008  1230769  1292929
     128     8  212271  688172  1641025  1438202  1505882  663212  1319587  727272  1580246  656410  663212  1505882  1662337
     128    16  222222  744186  1828571  1599999  1828571  723163  1523809  962406  1828571  766467  766467  1753424  1828571
     128    32  226950  780487  2031746  1729729  1999999  643216  1777777  992248  1753424  934306  941176  1969230  2133333
     128    64  226950  785276  1855072  1969230  2133333  785276  1684210  876712  2098360  810126  1333333  2064516  2245614
     128   128  228980  795031  1753424  2327272  2327272  739884  1969230  805031  1523809  795031  805031  2245614  2370370
     256     4  198449  568888  1190697  1196261  1122807  573991  1003921  742028  1098712  537815  563876  1273631  1299492
```

iozone 本身可以生成 excel , 结合 gnuplot 之类的画图工具还可以生成更复杂的剖面图



lmbench

lmbench 是一个综合性的性能测量工具集合

安装

```
apt install -y lmbench
```

下面介绍一些典型的功能

bw_mem

bw_mem用于测量内存带宽

```
root@thead-910:~# bw_mem 1M rd
1.00 9565.06
```

表示测试 1MB数据量的读 , 速度为 9565MB/s

```
root@thead-910:~# bw_mem 10M cp
10.00 1218.92
```

表示测试 10MB数据量的拷贝 , 速度为 1218MB/s

lat_mem_rd

lat_mem_rd用于测量读内存的延迟

```
root@thead-910:~# lat_mem_rd 1M
"stride=64
0.00049 2.501
0.00098 2.501
0.00195 2.501
0.00293 2.501
0.00391 2.501
0.00586 2.501
0.00781 2.502
0.01172 2.503
0.01562 2.502
0.02344 2.503
0.03125 2.504
0.04688 3.790
0.06250 6.100
0.09375 6.198
0.12500 6.650
0.18750 6.741
0.25000 6.730
0.37500 6.710
0.50000 6.701
0.75000 6.695
1.00000 6.690
```

左边一列是读到的数据量（MB），右边一列是读的延时（ns）

mhz

mhz用于测量当前CPU的频率

```
root@thead-910:~# mhz
1199 MHz, 0.8340 nanosec clock
```

表示 cpu 运行的频率是 1.2GHz

当然，lmbench 还有很多测试用例，所以可以使用一个来自 ltp 的 lmbench 测试集 lmbench_test.sh（默认在apt install的时候安装），该测试集涵盖了 lmbench 所有的测试用例，跑一遍下来，所有性能参数一目了然，如下：

```
root@thead-910:~# lmbench_test.sh
|TRACE LOG| ***** STARTING LMBENCH SCRIPT ***** |
|TRACE LOG| CREATING THE FILE OF 16MB SIZE FOR BENCHMARKING|
dd: error writing 'test1.txt': No space left on device
9+0 records in
8+0 records out
|TRACE LOG| ***** STARTING BANDWIDTH BENCHMARKS ***** |
|TRACE LOG| MEMORY BANDWIDTH BENCHMARKS |
|TEST START|bw_mem|
|TRACE LOG| Parameters      : |
|TRACE LOG| Operation       - rd|
|TRACE LOG| Memory Blk Size - 1M|
1.00 9563.10
|TEST RESULT|PASS|bw_mem|
|TEST END|bw_mem|
|TEST START|bw_mem|
|TRACE LOG| Parameters      : |
|TRACE LOG| Operation       - rd|
|TRACE LOG| Memory Blk Size - 2M|
2.00 9172.56
|TEST RESULT|PASS|bw_mem|
|TEST END|bw_mem|
|TEST START|bw_mem|
|TRACE LOG| Parameters      : |
```



```
|TRACE LOG| Operation          - rd|
|TRACE LOG| Memory Blk Size - 4M|
... ..
AF_UNIX sock stream bandwidth: 1903.41 MB/sec
|TEST RESULT|PASS|bw_unix|
|TEST END|bw_unix|
|TRACE LOG| BANDWIDTH OF FILE READ|
|TEST START|bw_file_rd|
|TRACE LOG| Parameters          : |
|TRACE LOG| Size                - 1MB|
|TRACE LOG| operation           - open2close|
|TRACE LOG| File                - test1.txt|
1.00 1658.93
|TEST RESULT|PASS|bw_file_rd|
|TEST END|bw_file_rd|
|TEST START|bw_file_rd|
|TRACE LOG| Parameters          : |
|TRACE LOG| Size                - 1MB|
|TRACE LOG| operation           - io_only|
|TRACE LOG| File                - test1.txt|
1.00 1652.89
... ..
"size=128k ovr=26.95
4 7.87
|TEST RESULT|PASS|lat_ctx|
|TEST END|lat_ctx|
|TEST START|lat_ctx|
|TRACE LOG| Parameters          : |
|TRACE LOG| procs               - 4|
|TRACE LOG| size_in_kbytes      - 256K|
|TRACE LOG| repetitions        - 100|
... ..
```

nmon

nmon 是一个综合性的性能观测工具

安装

```
apt install -y nmon
```

输入 nmon 会进入这个画面

```
+nmon-16g-----Hostname=buildroot----Refresh= 0secs ---09:01.14--+
|
| -----
|      _ _ _ _ _          For help type H or ...
| | '_ \ | '_ ` _ \ / _ \ | '_ \       nmon -?  - hint
| | | | | | | | | (_ ) | | | |       nmon -h  - full details
| | _ | |_ | | _ | |_ \|_ \|/_ \| |_ |
|
|                               To stop nmon type q to Quit
| -----
|
|
|
| Use these keys to toggle statistics on/off:
|   c = CPU           l = CPU Long-term       - = Faster screen updates
|   C = " WideView    U = Utilisation         + = Slower screen updates
|   m = Memory        V = Virtual memory     j = File Systems
|   d = Disks         n = Network             . = only busy disks/procs
|   r = Resource      N = NFS                 h = more options
|   k = Kernel        t = Top-processes      q = Quit
+-----+
```

再次输入“cUd”可以观测 cpu 使用率，io 使用率等性能参数

```
+nmon-16g-----[H for help]---Hostname=buildroot----Refresh= 2secs ---09:03.30-+
| CPU Utilisation -----|
|-----+-----+|
|CPU User%   Sys% Wait%   Idle|0           |25           |50           |75           |100||
| 1   21.9    2.0    0.0   76.1|UUUUUUUUUU   >           ||
| 2    0.5    3.0    0.0   96.5|s           >           ||
|-----+-----+|
|Avg   11.0    2.2    0.0   86.8|UUUUUs >           ||
|-----+-----+|
| CPU Utilisation Stats -----|
|ALL      21.9    0.0    4.5  173.3    0.0    0.0    0.0    0.0    0.0    0.0   |
|CPU      User%   Nice%    Sys% Idle%   Wait% HWirq% SWirq% Steal% Guest% GuestNice%|
| 1      21.9    0.0    2.0   76.2    0.0    0.0    0.0    0.0    0.0    0.0   |
| 2       0.5    0.0    3.0   96.6    0.0    0.0    0.0    0.0    0.0    0.0   |
| Disk I/O --/proc/diskstats----mostly in KB/s-----Warning:contains duplicates-|
|DiskName Busy   Read WriteKB|0           |25           |50           |75           |100||
|mmcblk0    0%    0.0    0.0|>           ||
|mcb1k0p3   0%    0.0    0.0|>           ||
|Totals Read-MB/s=0.0      Writes-MB/s=0.0      Transfers/sec=0.0      |
|-----|
+-----+
```

iperf3

iperf 用于测量网络性能

安装

```
apt install -y iperf3
```

在主机端启动一个 iperf3 server

```
Microsoft Windows [版本 10.0.18363.1256]
(c) 2019 Microsoft Corporation。保留所有权利。

D:\公司\工具\iperf-3.1.3-win64>iperf3.exe -s
-----
Server listening on 5201
-----

Accepted connection from 192.111.111.11, port 58022
[ 5] local 192.111.111.123 port 5201 connected to 192.111.111.11 port 58024
[ ID] Interval            Transfer      Bandwidth
[ 5]  0.00-1.00    sec  11.3 MBytes  94.8 Mbits/sec
[ 5]  1.00-2.00    sec  11.1 MBytes  92.9 Mbits/sec
[ 5]  2.00-3.00    sec  11.3 MBytes  94.4 Mbits/sec
[ 5]  3.00-4.00    sec  11.3 MBytes  94.9 Mbits/sec
[ 5]  4.00-5.00    sec  11.3 MBytes  94.9 Mbits/sec
[ 5]  5.00-6.00    sec  11.3 MBytes  94.9 Mbits/sec
[ 5]  6.00-7.00    sec  11.3 MBytes  94.9 Mbits/sec
[ 5]  7.00-8.00    sec  11.3 MBytes  94.9 Mbits/sec
[ 5]  8.00-9.00    sec  11.3 MBytes  94.9 Mbits/sec
[ 5]  9.00-10.00   sec  11.3 MBytes  94.9 Mbits/sec
```

在板端启动一个 iperf3 测试程序

```
# iperf3 -c 192.111.111.123
Connecting to host 192.111.111.123, port 5201
[ 5] local 192.111.111.11 port 58024 connected to 192.111.111.123 port 5201
[ ID] Interval            Transfer      Bitrate      Retr  Cwnd
[ 5]  0.00-1.00    sec  11.6 MBytes  96.8 Mbits/sec    0   92.7 KBytes
[ 5]  1.00-2.00    sec  11.0 MBytes  92.5 Mbits/sec    0   92.7 KBytes
[ 5]  2.00-3.00    sec  11.3 MBytes  94.6 Mbits/sec   54   94.1 KBytes
[ 5]  3.00-4.00    sec  11.4 MBytes  95.6 Mbits/sec    0   98.4 KBytes
```

[	5]	4.00-5.00	sec	11.3	MBytes	94.6	Mbits/sec	0	115	KBytes
[	5]	5.00-6.00	sec	11.3	MBytes	95.1	Mbits/sec	0	115	KBytes
[	5]	6.00-7.00	sec	11.4	MBytes	95.6	Mbits/sec	0	115	KBytes
[	5]	7.00-8.00	sec	11.3	MBytes	94.6	Mbits/sec	0	137	KBytes
[	5]	8.00-9.00	sec	11.4	MBytes	95.6	Mbits/sec	0	137	KBytes
[	5]	9.00-10.00	sec	11.4	MBytes	95.6	Mbits/sec	0	137	KBytes

就可以测试网络性能了

pv

pv可以用来显示命令在执行时的进度条

安装

```
apt install -y pv
```

例如拷贝文件（把testfile1拷贝为testfile2），就可以用pv命令来代替：

```
root@thead-910:~# dd if=/dev/zero of=testfile1 bs=1M count=128
root@thead-910:~# pv testfile1 > testfile2
27.6KiB 0:00:00 [ 130MiB/s] [=====>] 100%
```

而且pv还能限制传输的速度，例如1KB/s

```
root@thead-910:~# pv -L 1k testfile1 > testfile2
10KiB 0:00:10 [1011 B/s] [=====>] 36% ETA 0:00:17
```

或者dd：

```
root@thead-910:~# pv < /dev/mmcblk0p3 | dd of=/tmp/testfile bs=4M
662MiB 0:00:02 [ 198MiB/s] [=====>] 17% ETA 0:00:09
```

dieharder

dieharder 是一款用于测试随机数生成能力的测试工具

安装

```
apt install -y dieharder
```

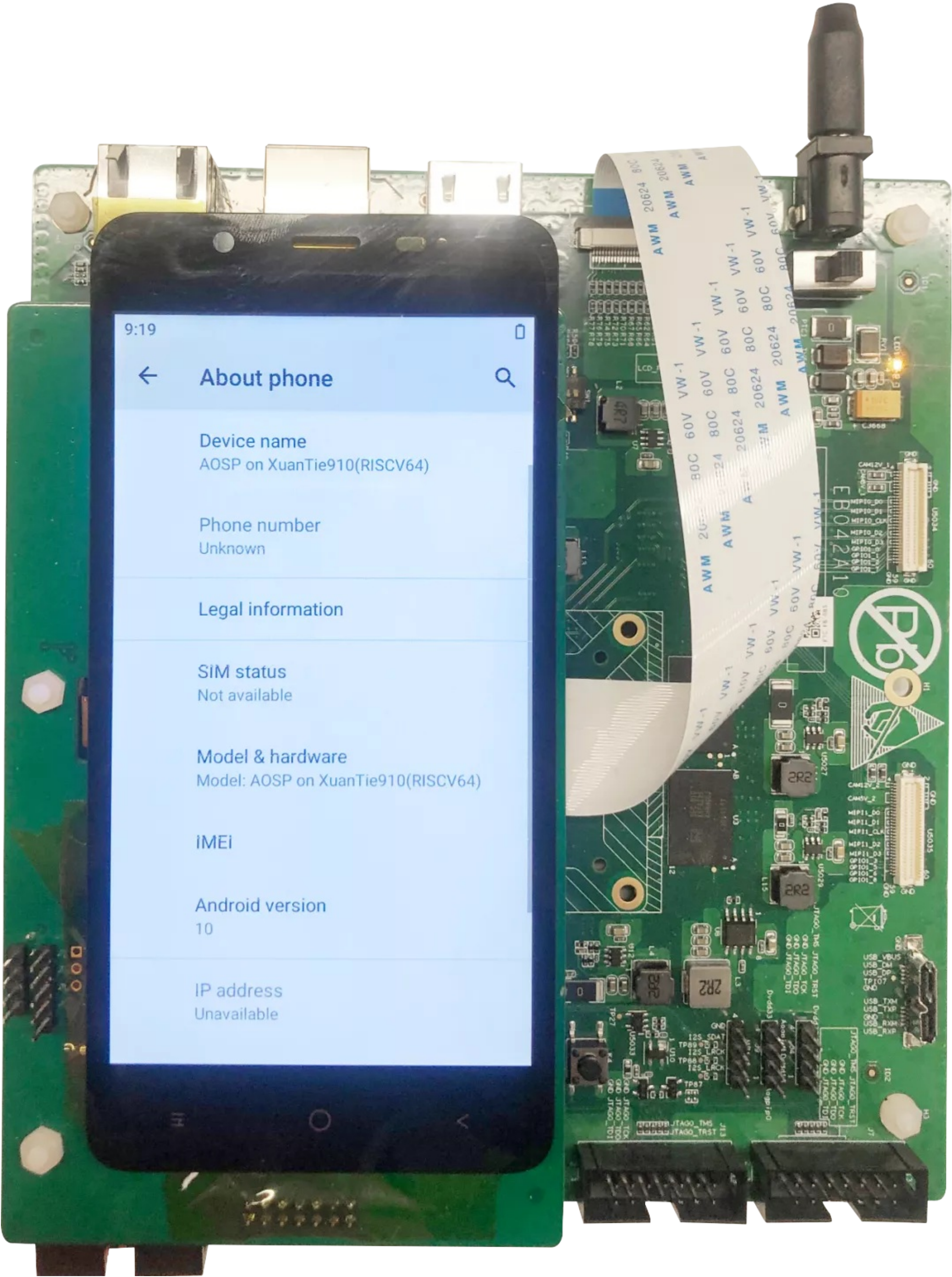
```
root@thead-910:~# dieharder -a
#####
#           dieharder version 3.31.1 Copyright 2003 Robert G. Brown           #
#####
# rng_name      |rands/second|   Seed   |
#   mt19937|   3.34e+07  | 273137300|
#####
# test_name     |ntup| tsamples |psamples|  p-value |Assessment
#####
# diehard_birthdays|   0|      100|    100|0.08654260| PASSED
#   diehard_operm5|   0|   1000000|    100|0.76268865| PASSED
# diehard_rank_32x32|   0|     40000|    100|0.09896061| PASSED
#   diehard_rank_6x8|   0|    100000|    100|0.79845879| PASSED
# diehard_bitstream|   0|   2097152|    100|0.90976807| PASSED
#   diehard_opso|   0|   2097152|    100|0.90149754| PASSED
#   diehard_oqso|   0|   2097152|    100|0.60943983| PASSED
#   diehard_dna|   0|   2097152|    100|0.28147819| PASSED
# diehard_count_1s_str|   0|    256000|    100|0.87571086| PASSED
# diehard_count_1s_byt|   0|    256000|    100|0.32803443| PASSED
# diehard_parking_lot|   0|     12000|    100|0.43004782| PASSED
```


diehard_2dsphere	2	8000	100 0.91557461	PASSED
diehard_3dsphere	3	4000	100 0.10831747	PASSED
diehard_squeeze	0	100000	100 0.58947448	PASSED
diehard_sums	0	100	100 0.03804515	PASSED
diehard_runs	0	100000	100 0.16852247	PASSED
diehard_runs	0	100000	100 0.16901521	PASSED
diehard_craps	0	200000	100 0.28860890	PASSED
diehard_craps	0	200000	100 0.41000082	PASSED
marsaglia_tsang_gcd	0	10000000	100 0.94040087	PASSED
marsaglia_tsang_gcd	0	10000000	100 0.98666060	PASSED
sts_monobit	1	100000	100 0.99649756	WEAK
sts_runs	2	100000	100 0.06977154	PASSED
...	...			

Android on RISC-V

平头哥基于安卓开源项目(AOSP)实现了对 RISC-V 架构的支持。安卓软件栈主要包括系统内核、硬件抽象、运行时、框架层、应用五个层次的近千个软件包，其中涉及到处理器架构相关移植工作主要包含本地库与ART支持、Linux内核支持、build系统支持三大部分：

- 本地库与ART支持。完成 Bionic、ART、Clang/LLVM、V8、NDK、VNDK、OpenGL 等软件包的 RISC-V 架构支持：为 Bionic 添加动态链接、系统调用、浮点数学库的支持；基于 ART 实现了 RISC-V 架构的 DEX 实时解释执行、dex2oat、JNI 调用以及 JIT 编译优化，极大地提高了 JAVA 程序在 RISC-V 平台上执行的效率。
- Linux内核支持。完善了 Clang/LLVM 对 Linux 内核的编译支持，修复了大量 Clang/LLVM 的问题，首次将RISC-V 架构的 Linux 内核与安卓系统进行了适配。
- Build系统支持。安卓的编译框架主要由 blueprint 和 soong 构成，平头哥整合编译框架、预编译的工具链、本地方法库、模拟器，应用和服务等模块，首次实现了 RISC-V 架构对安卓 build 系统的支持。



RISC-V 作为一个开源处理器架构，从 2011 年诞生之初就为物联网时代 CPU 的发展提供了非常大的想象空间。近几年，主流的软件生态陆续实现了对 RISC-V 架构的支持，唯独在安卓生态系统方面进展缓慢，主要的原因包括：

- 移植代码量大，涉及模块多，整个工程包含727个子工程和多个预编译工程；

2. 架构级优化难度大，为了让安卓可以更加流畅的运行，需要在汇编指令层面实现对本地方法库、音视频库、加解密库等的性能优化；
3. 开发语言多种多样，包括 C/C++，汇编，JAVA，go，RS，rust等。

基于现有的工作成果，平头哥将开源全部的相关代码并与全球开发者一起推动 **AOSP on RISC-V** 的代码完善、性能优化与应用落地。

安卓作为第一大移动设备操作系统，支撑了全球数十亿部手机、电视与可穿戴设备，拥有数百万应用开发者。在碎片化的物联网时代，经历了时间检验的安卓系统也将是开发者和应用解决方案商的重要解决方案之一。安卓与RISC-V 的强强联合，将极大降低物联网设备和应用开发的门槛，推动百花齐放的产业局面。

目前，基于 RISC-V 64 架构的安卓开源仓库已在 OCC 上线，复制链接 <https://occ.t-head.cn/development/series/Index?id=3872715572777455616&type=kind&softPlatformType=8>

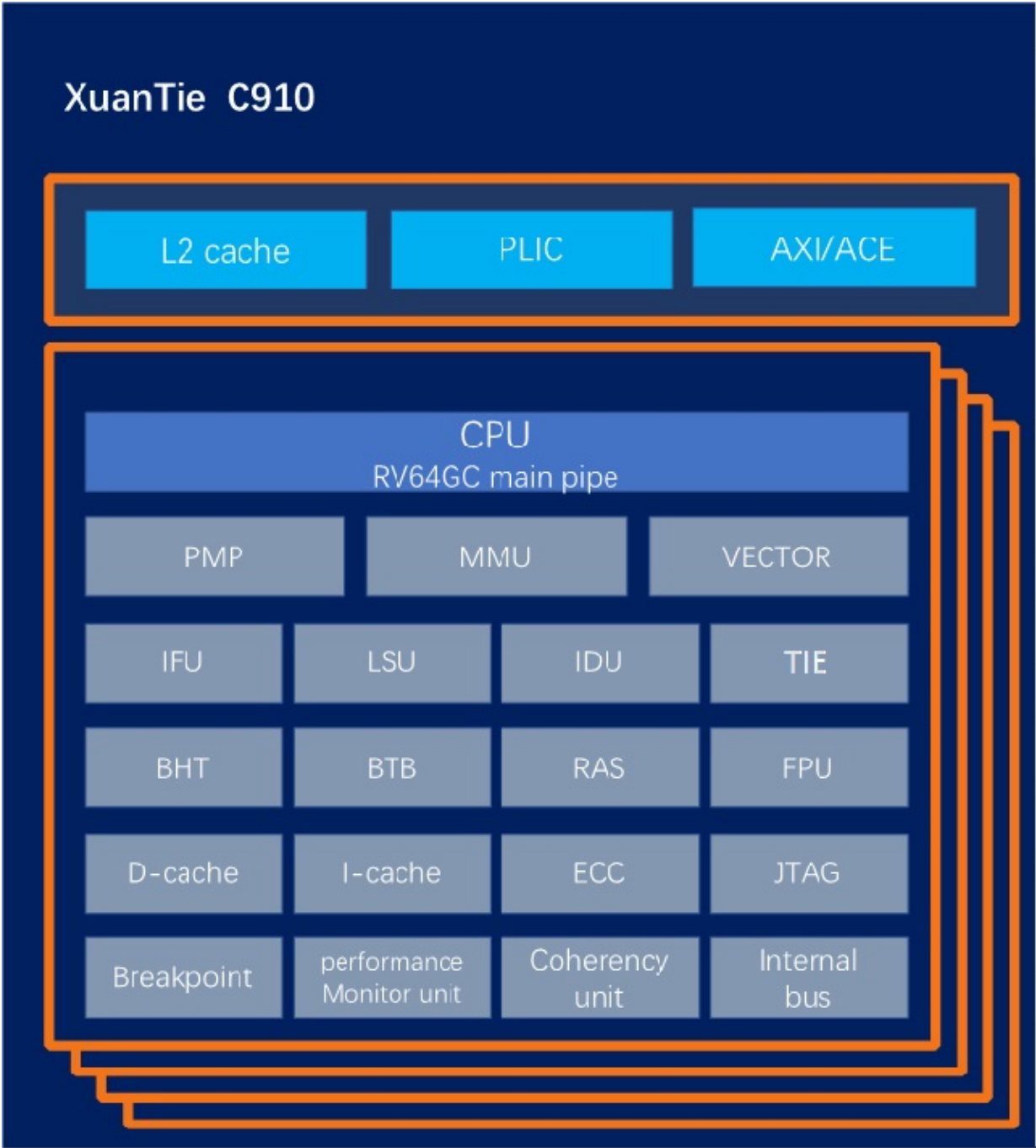
开源网址：

- <https://github.com/T-head-Semi/aosp-riscv>

RISC-V AOSP 开源项目

平头哥基于AOSP 10版本完成了RISC-V架构的支持，并基于模拟器和XuanTie 910芯片添加了相关设备和板级支持；用户可基于现有模拟器和板级支持开发基于RISC-V架构和产品。AOSP 是一个针对多种不同设备类型打造的开放源代码软件堆栈项目。

AOSP on XuanTie 910板级平台基于ICE开发板进行构建；搭载了两颗XuanTie C910 CPU核, C910采用12级超标量流水线，兼容RISC-V架构且针对算术运算、内存访问以及多核同步等方面进行了增强采用3发射、8执行的深度乱序执行架构，配有单/双精度浮点引擎，适用于5G、人工智能等对性能要求很高的应用领域。此外ICE SOC平台支持包括GPU，DPU，以太网，USB 3.0 Device，红外，i2s输入，hdmi，SDHC等外设IP; 适用于数据处理、图形渲染、语音识别等场景的开发使用。



Feature	Description
Architecture	RV64GCV
SMP	Up to 4 cores in each cluster, up to 4 clusters
Pipeline	12-stages (integer)
T-Head extension	T-Head instruction extension (TIE) T-Head memory attributes extension (TMAE)
Floating point Unit	Support RISC V F、D instruction extension Support IEEE 754-2008 standard
Vector Unit	Support RISC V V instruction extension(optional) vector register width 128bit element size support FP16/FP32/FP64/INT8/INT16/INT32/INT64
Bus interface	AXI4-128 master or ACE-128 master(optional)
Device coherence port	AXI4-128 slave (optional)
Instruction Cache	Up to 64KB with parity check (optional)
Data Cache	Up to 64KB with ECC (optional)
L2 Cache	Up to 8MB with ECC (optional) Supporting parallel access with multi-bank
Memory Management Unit (MMU)	Sv39 virtual memory translation Up to 2048 entry TLB
PMP	Up to 16 regions
Interrupt Controller	Flexibly configurable Platform-Level Interrupt Controller (PLIC)
Debug	RV debug

开源网址：

- <https://github.com/T-head-Semi/aosp-riscv>

烧写 Android 镜像

对于第一次使用 ICE-EVB 开发板的开发者，需要根据以下几个步骤完成开发板的镜像烧写：

- 第一步：烧录前准备：安装烧录工具(thead-tools、fastboot)、检查开发板与电脑的串口连接情况
- 第二步：包括下载镜像文件(ice_images)
- 第三步：通过 CCT 将 u-boot 烧写开发板的 eMMC 存储器
- 第四步：使用 u-boot、fastboot 等工具，烧写 Android 镜像
- 第五步：重启开发板，即可以进入Android

第一步：安装烧写工具

如果已经安装过 thead-tools 与 fastboot 可以跳过第一步。

thead-tools

通过 pip 命令来安装 yoctools 到你的系统中，yoctools 支持 python2.7+、python3.6+，建议使用 python2 作为默认 python。

```
sudo pip install thead-tools

# 如果使用官方地址下载过慢，可使用国内清华镜像源加速
sudo pip install -i https://pypi.tuna.tsinghua.edu.cn/simple thead-tools
```

如果未找到 pip 命令，请先安装 pythonp-pip，如： sudo apt install python-pip 。

thead-tools 使用

通过 thead-tools 命令可以烧写 uboot，更详细的 thead-tools 使用说明可以参见：thead-tools 使用说明，通过命令: thead cct --help 查使用说明：

```
root@linux > thead cct --help
Usage: thead cct <uart|list|download> [param]

Options:
  -h, --help            show this help message and exit
  -u UART                CCT serial port device
  -f FILE                Device start address
  -o OFFSET              Device start address
  -b BLOCK, --block=BLOCK
  -d DEVICE, --device=DEVICE
                        Device name
  -c, --compress
  -D, --debug            Enable debug trace info
```

安装 fastboot

发行版可通过包管理器安装 fastboot 工具，如 ubuntu下通过 sudo apt install fastboot 安装，或者从官网下载二进制文件，下载地址：<https://developer.android.com/studio/releases/platform-tools>。

第二步：下载镜像

```
wget "https://yoc.docs.t-head.cn/tools/android_images.tar.bz2"
tar xf android_images.tar.bz2
cd android_images
ls -l
```

需要用到的，是如下五个文件：


```
-rw-r--r-- 1 root root 23975004 Jan 13 14:24 boot.ext4
-rw-r--r-- 1 root root 683602236 Jan 13 14:19 system.img
-rw-r--r-- 1 root root 487692 Jan 5 10:04 userdata.img
-rw-r--r-- 1 root root 26996812 Jan 13 14:19 vendor.img
-rw-r--r-- 1 root root 599192 Dec 20 16:39 u-boot-with-spl.bin
```

- **u-boot-with-spl.bin**：uboot 引导镜像文件
- **boot.ext4**：boot 分区文件系统，ext4格式，包含 fw_jump.bin、uImage、ice_evb_c910.dtb
- **system.img**：Android 系统分区，ext4 格式
- **userdata.img**：Android 用户分区，ext4 格式
- **vendor.img**：Android 供应商分区，ext4 格式

第三步：烧写 uboot

查看电脑串口

参考 [开发板接线](#) 完成开发板的串口连接，通过 CCT 烧写 uboot 需要使用开发板的 CCT 烧写串口，通过命令 `thead cct uart` 命令看到电脑上安装的串口列表，并确认电脑上串口与开发板串的对应关系：

```
root@linux > thead cct uart
uart device list:
/dev/ttyUSB0 - USB-Serial Controller
/dev/ttyUSB1 - USB-Serial Controller
```

查找开发板存储器列表

通过命令：`thead cct -u /dev/ttyUSB0 list` 可以查看开发板支持的烧写存储器列表，如下：

```
root@linux > thead cct -u /dev/ttyUSB0 list
Wait .....
CCT Version: 2
memory device list:
dev = ram0 , size = 256.0KB
dev = emmc0 , size = 2.0MB
dev = emmc1 , size = 2.0MB
dev = emmc2 , size = 3.7GB
```

注意：使用该命令时，先确定串口的连接是否正确，运行命令前先关闭开发板电源后再运行该命令，等到出现 `Wait .....` 信息后再开启开发板电源，正常情况下电源开启后在1秒内会出现如下信息：

```
CCT Version: 2
memory device list:
dev = ram0 , size = 256.0KB
dev = emmc0 , size = 2.0MB
dev = emmc1 , size = 2.0MB
dev = emmc2 , size = 3.7GB
```

如果未出现上述打印信息，同时电脑是接有多个串口，请更新另一个串再尝试一次，如：`thead cct -u /dev/ttyUSB1 list`。如果所有串口都尝试后均未出现上述打印机信息，请检查串口与开发板的连接是否符合，详细连接参见：[开发板接线](#)。

烧写 uboot

通过命令：`thead cct -u /dev/ttyUSB0 download -f u-boot-with-spl.bin -d emmc0` 将 uboot 烧写到 eMMC 的 0号分区，烧写过程信息如下：

```
CCT Version: 2
Send file 'u-boot-with-spl.bin' to 21:0 ...
Writing at 0x00009800... (3%)
```


待打印 `File u-boot-with-spl.bin download success.` 后，烧写成功后，可以通过开发板的 Console 串口看到正确的启动信息：

```
U-Boot 2020.01-g6cc5d59b0d (Dec 20 2020 - 08:37:37 +0000)

CPU:   rv64imafdcvsu
Model: T-HEAD c910 ice
DRAM:  4 GiB
GPU ChipDate is:0x20151217
GPU Frequency is:500000KHz
NPU ChipDate is:0x20190514
DPU ChipDate is:0x20161213
MMC:   mmc0@3fffb0000: 0
Loading Environment from MMC... OK
In:     serial@3fff73000
Out:    serial@3fff73000
Err:    serial@3fff73000
Net:

Warning: ethernet@3fffc0000 (eth0) using random MAC address - e6:e2:ea:7a:30:ce
eth0: ethernet@3fffc0000
Hit any key to stop autoboot:  1
```

第四步：烧写 Android 镜像

开发板 uboot 配置

开发板重新开机后进入 uboot 命令模式，当串口出现如下提示时，按任键即可进行命令模式。

```
Warning: ethernet@3fffc0000 (eth0) using random MAC address - a6:7d:bc:02:7d:4d
eth0: ethernet@3fffc0000
Hit any key to stop autoboot:  3
```

通过 `gpt` 命令配置 eMMC 分区大小，命令如下：

```
# 恢复默认值
env default -a

# 配置 eMMC 分区
setenv partitions "name=sparse,size=2031kb"
setenv partitions "$partitions;name=bootpart,size=60MiB"
setenv partitions "$partitions;name=system,size=1500MiB"
setenv partitions "$partitions;name=vendor,size=100MiB"
setenv partitions "$partitions;name=cache,size=256MiB"
setenv partitions "$partitions;name=userdata,size=-"

gpt write mmc 0 $partitions

# 配置网络

# !!!!!!!!!!!!!!!!!!!!!!! 特别注意 !!!!!!!!!!!!!!!!!!!!!!!
# 如果网络中接入多块 ICE-EVB 开发板，每块开发板的 ethaddr 要唯一
setenv ethaddr 00:a0:a0:a0:a0:a1 # 如果网络中接入多块 ICE-EVB 开发板，ethaddr 要唯一
setenv ipaddr 192.168.1.100 # 开发板 IP 地址
setenv netmask 255.255.255.0
ping 192.168.1.1 # ping 主机的IP，检验网络是否正常

# 配置内核启动参数 bootargs

setenv bootargs "console=ttyS0,115200"
setenv bootargs "$bootargs root=PARTUUID=$uuid_rootfs rootfstype=ext4"
setenv bootargs "$bootargs earlycon=sbi rdinit=/init security=selinux"
setenv bootargs "$bootargs crashkernel=256M-:128M c910_mmu_v1 printk.devkmsg=on"
setenv bootargs "$bootargs androidboot.selinux=permissive"
setenv bootargs "$bootargs androidboot.hardware=ranchu"
```



```
setenv bootargs "$bootargs androidboot.super_partition"

# 配置 bootcmd
setenv bootcmd "ext4load mmc 0:2 $opensbi_addr fw_jump.bin"
setenv bootcmd "$bootcmd;ext4load mmc 0:2 $dtb_addr ice_evb_c910.dtb"
setenv bootcmd "$bootcmd;ext4load mmc 0:2 $kernel_addr uImage"
setenv bootcmd "$bootcmd;ext4load mmc 0:2 0x02000000 ramdisk.img"
setenv bootcmd "$bootcmd;bootm $kernel_addr - $dtb_addr"

# 环境变量保存到 eMMC 中
saveenv                                # 将配置保存到 eMMC,下次启动时无需再配置

# 开启fastboot模式
fastboot udp
```

烧写镜像

在电脑上使用使用 fastboot 命令完成镜像烧写，192.168.1.100 是开发板的运行 u-boot 时配置的 IP 地址，使用 fastboot 命令前确认电脑与开发板在同一个子网。可在板子上使用 `ping 192.168.1.1` 检查网络情况。

```
# PC 烧写镜像
fastboot -s udp:192.168.1.100 flash bootpart boot.ext4
fastboot -s udp:192.168.1.100 flash system system.img
fastboot -s udp:192.168.1.100 flash vendor vendor.img
fastboot -s udp:192.168.1.100 flash userdata userdata.img
```

第五步：重启进入Android

复位 ICE-EVB 开发板，自动进入 Android 界面。

FAQ

1. 在 Linux 电脑上使用 minicom 登录到开发板的串口控制台上，会出现显示不一致的现场，如何设置？
2. 《ICE SDK 使用户手册》有没有提供在线文档？
3. ICE 开发板上的软件可以从哪里下载？

提问：在 Linux 电脑上使用 **minicom** 登录到开发板的串口控制台上，会出现显示不一致的现场，如何设置？

解答：在使用 minicom 命令之后，先通过 `stty size` 查看当前主机上控制台的行宽大小，如：

```
xxxxx@thead-pc:~ > stty size
50 220
```

可以看到当前控制台是每行220字符，有50行，再用 `minicom` 登录到开发板后，输入如下命令 `stty rows 50 cols 220`：

```
root@thead-910:~# stty rows 50 cols 220
root@thead-910:~# stty size
50 220
```

提问：《ICE SDK 使用户手册》有没有提供在线文档？

解答：《ICE SDK 使用户手册》提供了在线文档，通过 <https://yoc.docs.t-head.cn/icebook/> 网页访问，在线文档提供了最新和内容，还还提供了最新的 PDF/MOBI/EPUB 格式的离线文档下载。

- PDF 格式：https://yoc.docs.t-head.cn/icebook/ice_book.pdf
- MOBI格式：https://yoc.docs.t-head.cn/icebook/ice_book.mobi
- EPUB格式：https://yoc.docs.t-head.cn/icebook/ice_book.epub

提问：ICE 开发板上的软件可以从哪里下载？

解答：平头哥提供对 RISC-V 架构的优化软件包的 debian 仓库，地址：<http://mirrors.aliyun.com/thead/>。可以在 RISC-V Linux 中使用，根据本文档提供的 debian 镜像制作，会默认增加该仓库。平头哥还提供了 debian 官方源的镜像：<http://mirrors.aliyun.com/debian-ports/>